



Grado en Ingeniería Informática - Ingeniería del Software

TRABAJO FIN DE GRADO

**Clasificación multiclase de tumores cerebrales en imágenes de
resonancia magnética mediante aprendizaje profundo**

Autor:

Joaquín González Rodríguez

Tutor:

José Cristóbal Riquelme Santos

Cotutor:

Manuel Carranza García

Departamento de Lenguajes y Sistemas Informáticos

Sevilla, octubre de 2026

Resumen

Este Trabajo Fin de Grado aborda la clasificación experimental de tumores cerebrales —glioma, meningioma y tumor pituitario— a partir de imágenes de resonancia magnética, mediante redes neuronales convolucionales (CNN) y técnicas de transfer learning, dentro del ámbito de la Ingeniería del Software aplicada al Deep Learning.

El proyecto comenzó explorando una tarea binaria (presencia/ausencia de tumor) sobre datos sin identificador de paciente fiable. La reconstrucción del conjunto con trazabilidad real por paciente permitió detectar que esa tarea binaria combinaba, en su diseño original, pacientes con tumor procedentes de una fuente y sujetos sin tumor procedentes de otra, lo que constituye un sesgo de procedencia: la etiqueta de clase quedaba asociada de forma sistemática al dataset de origen, comprometiendo cualquier resultado favorable como evidencia de capacidad de clasificación tumoral. Esta observación motivó la reorientación del proyecto hacia el núcleo final: la clasificación multiclase dentro de una única cohorte de referencia (Cheng et al., 233 pacientes), eliminando esa asociación determinista entre etiqueta y procedencia externa.

A partir de esa reorientación, el trabajo desarrolló un pipeline reproducible a nivel de procedimiento, con trazabilidad patient-level, particiones agrupadas por paciente entre entrenamiento y evaluación, controles automáticos de separación por paciente y de duplicados, comparación homogénea de varias arquitecturas mediante transfer learning, fijación por escrito del protocolo de evaluación antes de abrir el conjunto de test, y un análisis cualitativo de explicabilidad mediante Grad-CAM. La ejecución de los experimentos se automatizó en un clúster de computación de altas prestaciones (HPC) gestionado con SLURM.

El modelo final seleccionado, ResNet50 con ajuste fino (ResNet50-FT), fue el candidato con mejor balanced accuracy sobre las predicciones out-of-fold empleadas en la selección (88,54 %). Evaluado una única vez sobre un conjunto de holdout reservado, alcanzó una balanced accuracy del 93,89 %. Una validación cruzada posterior de cinco particiones sobre el conjunto completo de la cohorte caracterizó la variabilidad interna del modelo, con una balanced accuracy media entre particiones del $89,44 \% \pm 4,64$ puntos porcentuales y una balanced accuracy del 88,68 % calculada sobre las predicciones agregadas de todas las particiones.

Los resultados muestran un rendimiento favorable dentro de la cohorte estudiada, con una variabilidad entre particiones no despreciable que sitúa el resultado del holdout cerca del extremo superior del rango observado. El trabajo no incluye

validación externa ni validación clínica: el sistema desarrollado constituye un artefacto de clasificación experimental, evaluado dentro de los límites de una única cohorte de referencia y del protocolo experimental documentado, y no una herramienta de diagnóstico ni un dispositivo médico.

Palabras clave: tumores cerebrales, resonancia magnética, Deep Learning, redes neuronales convolucionales, transfer learning, clasificación a nivel de paciente, reproducibilidad experimental.

Índice general

1. Introducción.....	1
1.1 Contexto y motivación.....	1
1.2 Planteamiento del problema.....	2
1.3 Evolución metodológica del proyecto.....	2
1.4 Contribución desde la Ingeniería del Software.....	4
1.5 Alcance del trabajo.....	5
1.6 Estructura de la memoria.....	6
2. Objetivos y alcance.....	7
2.1 Objetivo general.....	7
2.2 Objetivos específicos.....	7
2.3 Alcance del proyecto.....	9
2.4 Aspectos fuera de alcance.....	10
2.5 Criterios técnicos y de calidad.....	11
3. Estado del arte.....	12
3.1 Clasificación de tumores cerebrales mediante CNN.....	12
3.2 Transfer learning y arquitecturas de referencia.....	13
3.3 El dataset Cheng/figshare en la literatura.....	15
3.4 Patient-level frente a image-level: independencia, leakage y duplicados	18
3.5 Explicabilidad mediante Grad-CAM.....	21
3.6 Limitaciones de la comparación con la literatura.....	21
4. Planificación y gestión del proyecto.....	26

4.1 Enfoque de planificación.....	26
4.2 Evolución temporal del proyecto.....	27
4.3 EDT — Estructura de Descomposición del Trabajo.....	30
4.4 Estimación de esfuerzo por paquetes de trabajo.....	32
4.5 Planificación temporal y diagrama de Gantt.....	32
4.6 Seguimiento real del esfuerzo.....	34
4.7 Análisis de desviaciones.....	35
4.8 Estimación de costes.....	36
4.8.1 Coste de personal (simulado).....	37
4.8.2 Coste de recursos técnicos (hardware y cloud).....	38
4.8.3 Coste teórico equivalente total del proyecto.....	40
4.9 Hitos y feedback durante el desarrollo.....	40
4.10 Síntesis.....	42
5. Análisis y diseño de la aplicación.....	42
5.1 Introducción y objetivo.....	42
5.2 Evolución desde ScanIA al demostrador actual.....	43
5.3 Arquitectura cliente-servidor.....	45
5.4 Cliente de escritorio.....	46
5.5 Backend de inferencia.....	46
5.6 Funcionalidades.....	47
5.7 Requisitos funcionales.....	48
5.8 Requisitos no funcionales y grado de cumplimiento.....	49
5.9 Reglas de negocio.....	50

5.10 Casos de uso.....	51
5.11 Historias de usuario.....	53
5.12 Diagramas de secuencia.....	54
5.13 Interfaz y capturas.....	56
5.14 Limitaciones del demostrador.....	58
5.15 Relación con infraestructura y despliegue.....	59
5.16 Vídeo demostrativo.....	60
5.17 Síntesis.....	60
6. Datos y reconstrucción metodológica.....	60
6.1 Fuentes de datos iniciales.....	60
6.2 Problema de trazabilidad en la primera versión.....	61
6.3 Reconstrucción con identificación real de paciente.....	62
6.4 Manifiestos y control de duplicados.....	64
6.5 Controles de fuga de información implementados.....	66
6.6 La tarea binaria tumor/no tumor y el sesgo de procedencia.....	67
6.7 Reorientación hacia la clasificación multiclase.....	68
6.8 Dataset multiclase definitivo.....	69
7. Diseño y arquitectura software del pipeline.....	71
7.1 Visión global del sistema experimental.....	71
7.2 Arquitectura del pipeline.....	71
7.3 Componentes y responsabilidades.....	73
7.4 Gestión de configuración.....	75
7.5 Modelo de datos y manifiestos.....	76

7.6 Trazabilidad de experimentos y artefactos.....	76
7.7 Integridad, hashes y persistencia.....	78
7.8 Validaciones automáticas y gestión de errores.....	79
7.9 Automatización de experimentos mediante HPC y SLURM.....	81
7.10 Reproducibilidad y empaquetado de resultados.....	82
7.11 Síntesis de las decisiones de diseño.....	83
8. Metodología de Deep Learning.....	84
8.1 Preprocesamiento de las imágenes.....	84
8.2 Arquitecturas consideradas y aprendizaje por transferencia.....	84
8.3 Evolución desde el baseline histórico hasta el modelo final.....	86
8.4 Configuración de entrenamiento y fine-tuning.....	87
8.5 Evaluación a nivel de paciente y métricas.....	89
8.6 Explicabilidad mediante Grad-CAM.....	90
8.7 Síntesis de la metodología.....	91
9. Diseño experimental y reproducibilidad del protocolo.....	92
9.1 Visión general del diseño experimental.....	92
9.2 Selección del modelo mediante validación cruzada a nivel de paciente.....	93
9.3 Evaluación sobre el holdout.....	94
9.4 Validación cruzada de cinco folds para estimar variabilidad interna.....	95
9.5 Métricas y controles del protocolo.....	97
9.6 Alcance y limitaciones del diseño experimental.....	99
9.7 Síntesis del protocolo.....	99
10. Resultados.....	101

10.1 Resultados históricos y controles metodológicos.....	101
10.2 Selección de arquitectura.....	102
10.3 Evaluación sobre el holdout final.....	104
10.4 Análisis cualitativo mediante Grad-CAM.....	106
10.5 Validación cruzada de cinco folds.....	108
10.6 Síntesis de resultados.....	111
11. Discusión.....	112
11.1 Lectura global.....	112
11.2 Del problema binario al núcleo multiclase.....	112
11.3 Selección del modelo y efecto observado del fine-tuning.....	113
11.4 Holdout y validación cruzada de cinco folds.....	114
11.5 Comportamiento por clases y análisis del Fold 2.....	116
11.6 Interpretabilidad mediante Grad-CAM.....	117
11.7 Comparabilidad con la literatura.....	117
11.8 Limitaciones metodológicas de la interpretación.....	118
11.9 Síntesis de la discusión.....	119
12. Viabilidad y recursos.....	123
12.1 Recursos técnicos utilizados.....	123
12.2 Infraestructura software y entorno de ejecución.....	123
12.3 Ejecución en HPC mediante SLURM.....	124
12.4 Recursos computacionales observados por etapa.....	125
12.5 Viabilidad técnica.....	126
12.6 Viabilidad económica y limitaciones de estimación.....	126

12.7 Síntesis.....	127
12.8 Despliegue operativo del servicio y material demostrativo.....	130
13. Consideraciones éticas y limitaciones.....	132
13.1 Procedencia de los datos y consideraciones éticas.....	132
13.1.1 Dataset Cheng.....	132
13.1.2 Dataset IXI.....	133
13.2 Sesgos y limitaciones metodológicas.....	134
13.3 Límites de interpretación y aplicabilidad.....	136
13.4 Síntesis.....	136
14. Conclusiones y trabajo futuro.....	138
14.1 Conclusiones.....	138
14.2 Cumplimiento de los objetivos.....	139
14.3 Principales aportaciones del trabajo.....	141
14.4 Trabajo futuro.....	142
14.5 Cierre.....	143
Referencias.....	145
Anexos.....	146

Índice de figuras

Figura 3.1 — Esquema del transfer learning aplicado en este TFG.....	14
Figura 3.2 — Arquitecturas de referencia consideradas en la fase frozen.....	15
Figura 3.3 — Ejemplos reales del dataset Cheng por clase (tarea multiclase). 16	
Figura 3.4 — Ejemplos reales de las fuentes empleadas en la tarea binaria histórica.....	17
Figura 4.1 — Estructura de Descomposición del Trabajo (EDT).....	30
Figura 4.2 — Diagrama de Gantt retrospectivo del proyecto.....	33
Figura 4.3 — Dedicación real por paquete de trabajo y mes (mapa de calor). 34	
Figura 4.4 — Horas previstas frente a horas reales por paquete de trabajo.....	35
Figura 5.1 — Línea temporal: de ScanIA al demostrador actual.....	43
Figura 5.2 — Diagrama de componentes/módulos.....	45
Figura 5.3 — Diagrama de casos de uso.....	51
Figura 5.4 — Diagrama de secuencia: análisis binario 2D.....	54
Figura 5.5 — Diagrama de secuencia: análisis Cheng multiclase 2D.....	55
Figura 5.6 — Diagrama de secuencia: análisis de volumen NIfTI y visor 3D....	55
Figura 5.7 — Pantalla principal, con el aviso de uso experimental y el selector de modelo.....	56
Figura 5.8 — Selector de modelo ("Cheng Brain Tumor Classification" activo) y botón de selección de archivo.....	57
Figura 5.9 — Modal de resultado del modelo Cheng: clase predicha, probabilidad, distribución por clase y mapa Grad-CAM.....	57
Figura 5.10 — Visor volumétrico 3D en el navegador, con el resultado Cheng superpuesto (meningioma, 58,7 %).....	58

Figura 6.1 — Reconstrucción de los datos con identificación real de paciente.	64
Figura 7.1 — Arquitectura del pipeline experimental.....	73
Figura 7.2 — Cadena de trazabilidad de artefactos.....	78
Figura 9.1 — Cronología del protocolo experimental.....	93
Figura 10.1 — Balanced accuracy OOF durante la selección de arquitectura	103
Figura 10.2 — Matriz de confusión del holdout final.....	105
Figura 10.3 — Grad-CAM de casos representativos.....	107
Figura 10.4 — Balanced accuracy por fold en la 5CV.....	109
Figura 10.5 — Matriz de confusión agregada OOF de la 5CV.....	110
Figura 11.1 — Holdout frente al rango de variabilidad de la 5CV.....	116

Índice de tablas

Tabla 3.1 — Estudios representativos verificados.....	23
Tabla 4.1 — Dedicación real por mes (resumen).....	28
Tabla 4.2 — Evolución del proyecto: problema/objetivo, decisión y consecuencia.....	29
Tabla 4.3 — EDT con horas previstas (estimación de referencia reconstruida a posteriori), horas reales y desviación.....	30
Tabla 4.4 — Coste de personal simulado, por rol.....	37
Tabla 4.5 — Coste real frente a coste teórico equivalente de los recursos técnicos.....	38
Tabla 4.6 — Cuestión metodológica planteada y respuesta incorporada al proyecto.....	41
Tabla 5.1 — Funcionalidades confirmadas del demostrador.....	47
Tabla 5.2 — Requisitos funcionales (RF).....	48
Tabla 5.3 — Requisitos no funcionales (RNF) y grado de cumplimiento.....	50
Tabla 5.4 — Reglas de negocio (RB).....	50
Tabla 5.5 — Casos de uso: identificación y contexto.....	51
Tabla 5.6 — Casos de uso: comportamiento.....	52
Tabla 6.1 — Evolución de los datos utilizados en el proyecto.....	63
Tabla 6.2 — Dataset Cheng multiclase: distribución final por clase.....	69
Tabla 7.1 — Componentes del pipeline.....	73
Tabla 7.2 — Mecanismos de trazabilidad/validación.....	80
Tabla 8.1 — Arquitecturas consideradas.....	86
Tabla 8.2 — Configuración de entrenamiento.....	88

Tabla 8.3 — Métricas utilizadas.....	90
Tabla 9.1 — Etapas del protocolo.....	97
Tabla 9.2 — Controles del diseño experimental.....	98
Tabla 10.1 — Resultados históricos y controles.....	101
Tabla 10.2 — Selección de arquitectura.....	104
Tabla 10.3 — Holdout final.....	105
Tabla 10.4 — Resultados por fold.....	108
Tabla 10.5 — Resultados OOF agregados (233 pacientes).....	109
Tabla 11.1 — Hallazgos, interpretación permitida y límite de esa interpretación	121
Tabla 12.1 — Resumen de jobs SLURM documentados.....	128
Tabla 12.2 — Recursos por categoría.....	129
Tabla 13.1 — Estado documental de los datos.....	133
Tabla 13.2 — Limitaciones principales.....	135
Tabla 14.1 — Cumplimiento de los objetivos.....	140

1. Introducción

1.1 Contexto y motivación

El uso de técnicas de Deep Learning para analizar imágenes médicas ha convertido la evaluación de modelos de clasificación en un problema que no depende únicamente de la arquitectura empleada. La procedencia de los datos, la unidad de partición, la independencia entre muestras y la trazabilidad del proceso experimental condicionan la interpretación de cualquier resultado obtenido.

Este TFG aborda la clasificación de tumores cerebrales a partir de imágenes de resonancia magnética (RM) mediante redes neuronales convolucionales (CNN), dentro del ámbito del Deep Learning aplicado al análisis de imagen. En este tipo de modelos, las representaciones utilizadas para la clasificación se aprenden durante el entrenamiento a partir de los datos, en lugar de definirse manualmente como un conjunto cerrado de características. Una CNN puede entrenarse, en concreto, para asignar a un corte de RM una etiqueta de tipo tumoral —por ejemplo, glioma, meningioma o tumor pituitario— a partir de un conjunto de imágenes ya etiquetadas.

Sin embargo, evaluar correctamente un sistema de este tipo resulta más complejo de lo que sugiere ese planteamiento inicial. Cuando un mismo paciente aporta varios cortes de RM, o cuando las etiquetas de clase están asociadas de forma sistemática a datasets de procedencia diferentes, un resultado numérico elevado puede estar condicionado por factores distintos de la información que se pretende estudiar. La clasificación de imágenes médicas es, por eso, un dominio técnicamente exigente: factores como la dependencia entre muestras de un mismo sujeto, la procedencia de los datasets utilizados, la trazabilidad de cada imagen hasta su origen y la reproducibilidad del proceso experimental completo pueden alterar de forma sustancial la interpretación de un resultado, incluso cuando ese resultado en sí mismo parece favorable.

Este tipo de problemas metodológicos —y no solo la elección de una arquitectura concreta— es lo que motiva el interés de este trabajo, que se sitúa en el ámbito de Ingeniería Informática, especialidad de Ingeniería del Software, y no en un contexto clínico o de investigación biomédica. El interés del proyecto no reside, por tanto, únicamente en obtener un clasificador con un rendimiento razonable, sino en construir y documentar un procedimiento experimental suficientemente trazable para que sus resultados puedan interpretarse dentro de los límites reales de los datos utilizados.

1.2 Planteamiento del problema

El problema abordado en este TFG tiene dos dimensiones que conviene distinguir desde el principio.

La primera es un problema de aprendizaje automático propiamente dicho: clasificar imágenes de RM, y en última instancia pacientes, dentro de un conjunto reducido de clases tumorales. La segunda dimensión, menos habitual en un enfoque centrado exclusivamente en el modelo, es un problema de ingeniería y de metodología experimental. Implica garantizar que los datos empleados sean trazables hasta su origen, que las particiones entre entrenamiento y evaluación respeten la identidad real de cada paciente, que los experimentos puedan reconstruirse a partir de los artefactos generados, y que un resultado numérico pueda relacionarse de forma inequívoca con la configuración, los datos y el modelo que lo produjeron.

En conjuntos de imágenes médicas en los que un mismo paciente aporta varios cortes, esta segunda dimensión adquiere una relevancia particular: la forma en que se reparten las imágenes entre los distintos subconjuntos puede alterar sustancialmente la interpretación de un resultado, con independencia de la arquitectura utilizada. Una separación que no tenga en cuenta la identidad del paciente puede hacer que el modelo se beneficie, durante la evaluación, de haber visto en el entrenamiento otras imágenes muy similares del mismo sujeto, lo que reduce la independencia real entre los conjuntos y dificulta interpretar un resultado favorable como evidencia sólida de capacidad de generalización. Este capítulo no desarrolla aquí las implicaciones completas de esa distinción —que se retoman con detalle en los capítulos 3, 6 y 9— pero la señala como uno de los ejes que atraviesan todo el proyecto.

El origen concreto de esta preocupación metodológica se remonta a una fase temprana del proyecto. El desarrollo comenzó explorando la clasificación de tumores cerebrales a partir de conjuntos de imágenes derivados de plataformas públicas de tipo Kaggle, sin identificador de paciente propiamente dicho. Un análisis posterior de esos datos puso de manifiesto que no era posible reconstruir de forma fiable la identidad real del paciente ni la procedencia exacta de cada imagen a partir de la información disponible en esas fuentes. Esta limitación —no la elección de una arquitectura concreta— fue la que motivó la reconstrucción del conjunto de datos a partir de fuentes con identificación real de sujeto o paciente, descrita con detalle en el capítulo 6.

1.3 Evolución metodológica del proyecto

La reconstrucción de los datos con identificación real de paciente no fue el único punto de inflexión del proyecto. Tras esa reconstrucción, una auditoría de los

datos resultantes permitió detectar que la tarea binaria explorada inicialmente — distinguir entre presencia y ausencia de tumor— combinaba, en su diseño original, imágenes de pacientes con tumor procedentes de una fuente y de sujetos sin tumor procedentes de una fuente distinta. Esta asociación entre la etiqueta de clase y el dataset de procedencia constituye un sesgo de procedencia: un modelo entrenado sobre esos datos puede alcanzar un rendimiento elevado aprovechando diferencias sistemáticas asociadas a las fuentes de datos. La evidencia disponible permite demostrar la existencia de información discriminativa de bajo nivel asociada a esa separación, pero no determinar qué parte de dicha información está relacionada o no con características tumorales.

Esta observación no invalidó el interés del proyecto, pero sí condicionó de forma decisiva su orientación final: la tarea binaria tumor/no-tumor pasó a considerarse un antecedente histórico del trabajo, cuyo resultado no puede emplearse como evidencia central de capacidad de clasificación tumoral. El núcleo final del TFG se reorientó hacia la clasificación multiclase dentro de una única cohorte de referencia — glioma, meningioma y tumor pituitario—, eliminando así la asociación determinista entre etiqueta y dataset externo presente en la tarea binaria, aunque sin que ello permita descartar otros sesgos residuales dentro de la propia cohorte.

Esta distinción entre sesgo de procedencia y otros problemas metodológicos con los que suele confundirse, como la fuga de información entre entrenamiento y evaluación, resulta importante para no simplificar en exceso lo ocurrido. La propia composición del conjunto de datos hacía posible que un rendimiento numérico elevado se explicara, al menos parcialmente, por esa asociación estructural entre clase y procedencia, sin que de ello puedan derivarse afirmaciones concretas sobre qué información utilizó internamente el modelo. Esta precisión conceptual se retoma con mayor detalle en el capítulo 13.

A partir de esa reorientación, el trabajo dejó de estar centrado exclusivamente en entrenar un modelo y pasó a construir, de forma explícita, un proceso experimental completo. Este proceso incluye identificación real de cada paciente, particiones agrupadas por paciente entre entrenamiento y evaluación, controles de duplicados exactos y perceptualmente similares, controles orientados a detectar solapes entre pacientes y dependencias no deseadas entre subconjuntos, comparación de varias arquitecturas bajo un protocolo común, fijación por escrito del protocolo de evaluación final antes de abrir el conjunto de test, una evaluación sobre ese conjunto reservado y, posteriormente, una validación cruzada interna para caracterizar la variabilidad del modelo seleccionado, junto con un análisis cualitativo de explicabilidad mediante Grad-CAM. Esta evolución —de un experimento de clasificación a un proceso experimental

trazable y documentado— es la que permite presentar este TFG como un trabajo de Ingeniería del Software aplicado a un problema de Deep Learning experimental, y es también la idea que estructura el resto de esta memoria.

1.4 Contribución desde la Ingeniería del Software

Este TFG pertenece al Grado en Ingeniería Informática, especialidad de Ingeniería del Software, y esa procedencia no es incidental: una parte central de su aportación no reside en el modelo de clasificación en sí, sino en cómo se construyó, documentó y validó el proceso que condujo a él.

Esa aportación se concreta en varios elementos. Por un lado, en el diseño del pipeline experimental como un conjunto de responsabilidades separadas y reutilizables, con una configuración explícita y centralizada de los parámetros relevantes, en lugar de un script monolítico ajustado manualmente en cada ejecución. Por otro, en la trazabilidad de los artefactos generados a lo largo del proyecto —datos, particiones, modelos entrenados y resultados— mediante mecanismos de versionado por huella criptográfica, que permiten confirmar que un resultado concreto corresponde exactamente a una configuración y un conjunto de datos determinados. También en la incorporación de validaciones automáticas y comprobaciones de regresión que verifican, antes o durante la ejecución, condiciones como la ausencia de solapes entre pacientes o la consistencia de un resultado con una referencia histórica, en lugar de confiar en una inspección manual posterior. Se suma la automatización de la ejecución de los experimentos mediante infraestructura de computación de altas prestaciones, que permite automatizar y registrar las distintas ejecuciones experimentales, conservando logs y artefactos cuando corresponde y reduciendo la intervención manual durante su ejecución. Y, por último, la gestión explícita de la evolución del propio proyecto a lo largo de su desarrollo: las decisiones de reorientación metodológica descritas en el apartado anterior no se dejaron implícitas, sino que quedaron documentadas y relacionadas con los artefactos y hallazgos que las motivaron, de manera que puede reconstruirse por qué el proyecto cambió de rumbo en cada punto de inflexión.

En conjunto, la aportación central del TFG consiste en transformar un experimento de Deep Learning inicialmente orientado al entrenamiento de un modelo en un proceso experimental trazable, reproducible y auditable, en el que los datos, las particiones, la configuración, los modelos y los resultados pueden relacionarse entre sí mediante artefactos persistentes. Esta reproducibilidad se plantea a nivel de procedimiento: los artefactos conservados permiten verificar las principales decisiones, configuraciones y resultados, pero no garantizan un determinismo numérico exacto

entre ejecuciones distintas, cuestión que se retoma con más detalle en el capítulo 7. Con posterioridad, en la fase final del proyecto, se integraron los modelos entrenados en un demostrador software experimental —descrito en el capítulo 5— como complemento al núcleo científico ya descrito: una aportación adicional de Ingeniería del Software, no planificada desde el inicio y no equiparable en alcance a la aportación científica principal, que permite ejecutar inferencias sobre los modelos entrenados y visualizar sus resultados.

1.5 Alcance del trabajo

El alcance de este TFG queda delimitado tanto por lo que aborda como, de forma explícita, por lo que no pretende demostrar. El proyecto desarrolla y evalúa, de forma experimental, un sistema de clasificación multiclase de tumores cerebrales entre glioma, meningioma y tumor pituitario a partir de imágenes de RM, sobre una única cohorte de referencia. Ese desarrollo incluye la construcción de un pipeline de datos con trazabilidad real por paciente, la comparación de varias arquitecturas de Deep Learning bajo un protocolo experimental común, una evaluación patient-level tanto sobre un conjunto reservado como mediante validación cruzada interna, y un análisis cualitativo de explicabilidad mediante Grad-CAM como complemento visual de las predicciones del modelo.

El proyecto no aborda, en cambio, diagnóstico automático, validación clínica, certificación como dispositivo médico, segmentación tumoral, estimación del volumen tumoral ni localización espacial validada del tumor: Grad-CAM se emplea aquí exclusivamente como herramienta de inspección cualitativa y post-hoc, y en ningún caso equivale a una segmentación ni a una prueba de localización. Tampoco se ha realizado una validación externa sobre una cohorte independiente distinta de la empleada en el desarrollo y la evaluación del modelo, por lo que no puede establecerse la generalización externa del sistema a partir de este trabajo.

Además, aunque el proyecto tuvo históricamente un origen ligado a la idea de “detección” de tumores, la tarea final implementada y evaluada es de clasificación, no de detección espacial ni de segmentación. Por eso, esta memoria emplea de forma consistente los términos “clasificación experimental” y “clasificación multiclase” para describir lo que el sistema realmente hace.

Del mismo modo, la evaluación patient-level empleada en este trabajo —tanto sobre el conjunto reservado como mediante la validación cruzada interna— se limita a caracterizar el comportamiento del modelo dentro de la propia cohorte de referencia; no debe leerse como una demostración de robustez frente a condiciones de

adquisición, equipos o poblaciones de pacientes distintas de las representadas en esa cohorte.

1.6 Estructura de la memoria

El resto de esta memoria se organiza en trece capítulos adicionales. Tras esta introducción, el capítulo 2 concreta los objetivos específicos y el alcance detallado del proyecto, y el capítulo 3 sitúa el trabajo dentro de la literatura relevante sobre clasificación de tumores cerebrales mediante CNN, transfer learning y explicabilidad. El capítulo 4 documenta la planificación y la gestión del proyecto —incluyendo su evolución temporal, la estructura de descomposición del trabajo, el seguimiento real del esfuerzo dedicado y una estimación de costes—, y el capítulo 5 documenta, como aportación complementaria de Ingeniería del Software, el demostrador software desarrollado en una fase posterior del proyecto. Los capítulos 6 y 7 describen, respectivamente, la reconstrucción y trazabilidad de los datos empleados y el diseño de la arquitectura software del pipeline experimental, mientras que los capítulos 8 y 9 detallan la metodología de Deep Learning aplicada y el diseño experimental orientado a la trazabilidad y reproducibilidad del protocolo.

Los capítulos 10 y 11 presentan y discuten los resultados obtenidos, incluyendo la caracterización de la variabilidad interna mediante validación cruzada y el análisis cualitativo de explicabilidad. El capítulo 12 aborda la viabilidad del proyecto y los recursos empleados, y el capítulo 13 recoge las consideraciones éticas y las limitaciones del trabajo, incluyendo las relativas a los datos utilizados y al propio protocolo experimental. La memoria se cierra con un capítulo de conclusiones y trabajo futuro, seguido de las referencias; los anexos técnicos (Anexos A a J) se recogen en un documento complementario independiente, tal como se indica al final de esta memoria.

Esta organización refleja la evolución descrita en los apartados anteriores: los capítulos centrales no presentan el proyecto como si hubiera seguido, desde el principio, una línea recta hacia la clasificación multiclase final, sino que documentan también las decisiones metodológicas que fueron condicionando esa trayectoria, de manera que el lector pueda evaluar tanto el resultado final como el proceso que condujo a él.

2. Objetivos y alcance

2.1 Objetivo general

El objetivo general de este Trabajo Fin de Grado es diseñar, implementar y evaluar un pipeline reproducible de clasificación multiclase de tumores cerebrales (glioma, meningioma y tumor pituitario) a partir de imágenes de resonancia magnética, abordado desde una perspectiva de Ingeniería del Software. El proyecto no se limita a obtener un modelo de Deep Learning que clasifique correctamente las imágenes, sino que busca construir todo el proceso que rodea a ese modelo: la trazabilidad de los datos utilizados, la partición de los conjuntos de entrenamiento, validación y test a nivel de paciente (patient-level), el diseño experimental que permite comparar arquitecturas de forma homogénea, la evaluación rigurosa de los resultados obtenidos, la reproducibilidad de cada ejecución y la incorporación de un análisis de explicabilidad post-hoc sobre las predicciones del modelo final.

Este planteamiento responde a una limitación detectada durante las primeras fases del propio proyecto. La versión inicial de los datos no permitía establecer con suficiente fiabilidad la correspondencia entre cada imagen y un paciente real, lo que dificultaba asegurar una separación adecuada entre los conjuntos de entrenamiento, validación y prueba. A partir de esta limitación, la trazabilidad a nivel de paciente y los controles de separación pasaron a ocupar un papel central en el desarrollo del trabajo.

También conviene aclarar qué no es este proyecto. No se plantea como el desarrollo de una aplicación que diagnostique tumores cerebrales de forma autónoma. El resultado central de este trabajo sigue siendo el núcleo experimental multiclase —el pipeline de datos, el proceso de selección y evaluación de arquitecturas y el modelo resultante—, junto con toda la evidencia que permite reproducir y verificar cada paso, si bien el capítulo 5 documenta un demostrador software experimental, desarrollado y actualizado en fases posteriores del proyecto, que permite interactuar con los modelos entrenados sin constituir en sí mismo el objetivo científico del trabajo.

2.2 Objetivos específicos

Para alcanzar el objetivo general, el trabajo se articula en torno a un conjunto de objetivos específicos que cubren tanto la vertiente de datos e Ingeniería del Software como la vertiente de Deep Learning propiamente dicha.

En el ámbito de los datos y la trazabilidad, los objetivos son:

- Garantizar la trazabilidad patient-level de cada imagen utilizada, de forma que en todo momento sea posible identificar a qué paciente real pertenece cada

corte, en lugar de depender de identificadores que no se corresponden con pacientes distintos.

- Establecer controles de fuga de información entre los conjuntos de entrenamiento, validación y test, verificando de forma automática que no existan pacientes ni imágenes compartidas entre ellos.
- Construir un pipeline reproducible, en el que cada resultado quede acompañado de la configuración, los datos y el código que lo generaron, de modo que una misma ejecución pueda repetirse y verificarse posteriormente.
- Reconstruir y preparar el conjunto de datos multiclase a partir de las fuentes originales, dejando documentado el proceso de depuración y organización de las imágenes.

En el ámbito del modelado y la evaluación, los objetivos son:

- Desarrollar la clasificación multiclase sobre el dataset Cheng como núcleo experimental del trabajo, dejando en un segundo plano la tarea binaria explorada en fases anteriores del proyecto.
- Comparar varias arquitecturas de Deep Learning bajo un mismo protocolo, de manera que la elección del modelo final no dependa de una única prueba aislada.
- Seleccionar el modelo final mediante una evaluación basada en predicciones out-of-fold (OOF), evitando que la decisión se apoye en el mismo conjunto que después se usará como test final.
- Evaluar el modelo seleccionado sobre un conjunto de prueba reservado (holdout) que permanezca completamente inalterado hasta el momento de la evaluación final.
- Estudiar la variabilidad interna del modelo mediante una validación cruzada de cinco particiones (5CV), con el fin de analizar hasta qué punto el rendimiento observado depende del particionado de los datos.
- Incorporar Grad-CAM como técnica de explicabilidad post-hoc, que permita observar en qué regiones de la imagen se apoya el modelo al realizar sus predicciones.

De forma complementaria, en el ámbito del demostrador software, se plantea:

Integrar los modelos desarrollados en un demostrador software que permita ejecutar inferencias y visualizar los resultados obtenidos, incorporado como

aportación complementaria durante la evolución del proyecto y no como parte de la especificación formal inicial.

Por último, en el ámbito de la automatización, se plantea automatizar la ejecución de los experimentos mediante el uso de un clúster de computación de altas prestaciones (HPC) y el gestor de trabajos SLURM, reduciendo la intervención manual durante el entrenamiento y la evaluación de los modelos.

Estos objetivos no deben leerse como una lista de resultados ya alcanzados, sino como el conjunto de metas que orientaron las decisiones tomadas a lo largo del proyecto. Los resultados obtenidos al perseguir cada uno de ellos se presentan más adelante, en el capítulo dedicado específicamente a ese fin.

2.3 Alcance del proyecto

El alcance de este trabajo queda delimitado por el conjunto de decisiones tomadas sobre qué tarea abordar, con qué datos y bajo qué metodología.

La tarea central es la clasificación multiclase de tumores cerebrales en tres categorías —glioma, meningioma y tumor pituitario— a partir de imágenes bidimensionales derivadas de resonancia magnética. El proyecto emplea técnicas de transfer learning, partiendo de arquitecturas preentrenadas y adaptándolas al problema mediante una fase de entrenamiento de la cabeza de clasificación seguida de un ajuste fino de las últimas capas de la red. Sobre esta base se comparan varias arquitecturas candidatas antes de seleccionar una configuración final.

La evaluación se realiza siempre a nivel de paciente, agregando las predicciones de los distintos cortes correspondientes a un mismo paciente antes de calcular las métricas. Dado el desequilibrio en el número de pacientes de cada clase, la métrica principal considerada es la balanced accuracy, sin renunciar a reportar también otras métricas complementarias. El protocolo de evaluación combina un conjunto de holdout, reservado hasta el final del proceso de selección, con una evaluación de variabilidad interna mediante 5CV que permite observar el comportamiento del modelo bajo distintos particionados de los mismos datos.

También forma parte del alcance el análisis de explicabilidad mediante Grad-CAM sobre una selección de casos, así como la construcción de un pipeline reproducible en el que los principales artefactos experimentales —conjuntos de datos, modelos entrenados, resultados— quedan acompañados de la información necesaria para verificar su origen. La ejecución de los experimentos se automatiza mediante un entorno HPC gestionado con SLURM, lo que permite documentar de forma sistemática cuándo y en qué condiciones se llevaron a cabo las principales ejecuciones experimentales.

2.4 Aspectos fuera de alcance

De forma simétrica a lo anterior, es necesario precisar qué queda fuera de este trabajo, no porque se haya descartado por falta de interés, sino porque excede los objetivos y los recursos disponibles para un Trabajo Fin de Grado de estas características.

Quedan fuera de alcance:

- El uso del sistema como herramienta de diagnóstico clínico o su empleo en un contexto asistencial real.
- Cualquier proceso de certificación como producto sanitario.
- La validación clínica del modelo, entendida como su evaluación en un entorno médico real con supervisión especializada.
- La validación externa sobre una cohorte de pacientes independiente de la utilizada en este trabajo.
- La segmentación tumoral como objetivo del proyecto: el trabajo aborda un problema de clasificación, no de delimitación espacial del tumor.
- La clasificación a partir de volúmenes tridimensionales completos; el proyecto trabaja sobre cortes bidimensionales.
- Cualquier afirmación de generalización a poblaciones, escáneres o protocolos de adquisición distintos de los presentes en los datos utilizados.
- El desarrollo de una aplicación clínica certificada, o de cualquier producto sanitario, a partir del demostrador software descrito en el capítulo 5.

Sobre este último punto conviene una aclaración. Durante etapas previas se exploró un prototipo de interfaz denominado ScanIA; no obstante, tras la reorientación metodológica del proyecto hacia la clasificación multiclase, dicho prototipo quedó desacoplado del núcleo experimental multiclase y no se considera el producto final de este trabajo. Su origen y su relación con la evolución posterior del proyecto se explican brevemente en el capítulo 4 (planificación y gestión del proyecto). Con posterioridad, y de forma independiente a esa reorientación, se desarrolló y actualizó un demostrador software de escritorio que sí se integra con el backend de inferencia y con el modelo multiclase final; su origen, su alcance y sus limitaciones se documentan en el capítulo 5, sin que ello modifique el carácter experimental —no clínico— de este trabajo ni sus resultados científicos.

Ninguno de los puntos anteriores implica que las capacidades correspondientes carezcan de interés, sino que quedaron fuera del alcance finalmente acotado para este trabajo.

2.5 Criterios técnicos y de calidad

Los criterios técnicos y de calidad que guían este trabajo no proceden de un proceso formal de ingeniería de requisitos definido antes de comenzar el proyecto. Se trata, más bien, de criterios que se fueron consolidando a medida que el proyecto avanzaba, en respuesta a problemas concretos detectados durante su desarrollo.

La necesidad de trazabilidad patient-level, por ejemplo, surgió al identificar que el identificador utilizado en una versión anterior del dataset no permitía establecer con fiabilidad la correspondencia con pacientes reales. De forma similar, la exigencia de reproducibilidad y la necesidad de establecer controles de fuga de información entre los distintos conjuntos se convirtieron en criterios centrales conforme se identificaban riesgos concretos asociados a su ausencia. Por reproducibilidad se entiende aquí la capacidad de reconstruir y repetir un experimento a partir de la configuración, los datos, el código y el protocolo registrados, junto con mecanismos de integridad y trazabilidad de los artefactos mediante hashes. Las validaciones automáticas incorporadas en el código, que comprueban condiciones como la disjunción de pacientes entre conjuntos antes de continuar la ejecución, son la traducción práctica de estos criterios.

En conjunto, estos criterios pueden resumirse en cuatro ideas: que cada dato utilizado sea trazable hasta su origen, que cada resultado sea reproducible en el sentido anterior, que la separación entre los conjuntos de datos se compruebe mediante controles explícitos y automáticos a nivel de paciente, y que la evaluación del modelo final se apoye en más de una prueba aislada. La historia completa de cómo fueron surgiendo estos criterios, y de las decisiones que motivaron cada uno de ellos, se desarrolla en el capítulo dedicado a la gestión y evolución del proyecto.

3. Estado del arte

Este capítulo sitúa el TFG dentro de la literatura relevante para la clasificación de tumores cerebrales mediante Deep Learning sobre resonancia magnética (RM). La revisión que sustenta este capítulo procede de una búsqueda bibliográfica propia realizada en Scopus, con acceso institucional de la Universidad de Sevilla, restringida a los campos “Article title, Abstract, Keywords” y ejecutada en cuatro búsquedas distintas. Tras consolidar los registros recuperados y eliminar solapes entre búsquedas, se obtuvieron 39 estudios distintos, orientados especialmente a extraer, de cada uno, el dataset empleado, la arquitectura, la unidad de partición utilizada y los controles de independencia entre muestras. Esta revisión no sigue un protocolo formal de revisión sistemática (no se ha aplicado, por ejemplo, un esquema PRISMA), por lo que a lo largo de este capítulo se hace referencia a ella como revisión bibliográfica estructurada, no como revisión sistemática.

De los 39 estudios recopilados en la revisión bibliográfica específica sobre aplicaciones a tumores cerebrales, cuatro se utilizan y citan por nombre en el cuerpo como estudios aplicados directamente analizados: Bulla et al., Nalawade et al., Hassan et al. y Arghistani et al., verificados directamente contra su fuente original (artículo completo en la página oficial del editor o repositorio institucional). Los trabajos originales de las arquitecturas de referencia y de Grad-CAM (apartados 3.2 y 3.5) se citan por separado como fuentes metodológicas primarias y no forman parte de ese subconjunto de cuatro estudios aplicados. El resto de los estudios de la revisión Scopus no se utiliza como evidencia individual verificada en el cuerpo mientras permanezca pendiente la revisión de su artículo original; se recogen en el Anexo I del documento de anexos técnicos, identificando individualmente su estado de verificación. Este criterio —citar por nombre solo lo verificado contra la fuente original— se mantiene de forma consistente en todo el capítulo.

3.1 Clasificación de tumores cerebrales mediante CNN

La clasificación automática de tumores cerebrales a partir de imágenes de RM mediante redes neuronales convolucionales (CNN) es una línea de trabajo extensamente explorada en la literatura reciente. El planteamiento habitual consiste en entrenar una CNN para asignar una etiqueta de tipo tumoral (por ejemplo, glioma, meningioma o tumor pituitario) a partir de cortes bidimensionales de RM, generalmente en secuencia T1 con contraste, que es también el tipo de imagen empleado en este TFG.

Un aspecto que conviene señalar desde el principio, y que se retoma con más detalle en el apartado 3.4, es la necesidad de distinguir entre la unidad de entrada de

la red (habitualmente un corte o imagen individual) y la unidad real de evaluación del sistema (que puede ser ese mismo corte, o bien el paciente o sujeto del que proceden varios cortes). En la literatura revisada se encuentran trabajos que entrenan y evalúan sus modelos a nivel de corte sin explicitar si los cortes de un mismo paciente pueden repartirse entre entrenamiento y evaluación, lo cual condiciona cómo deben interpretarse sus resultados.

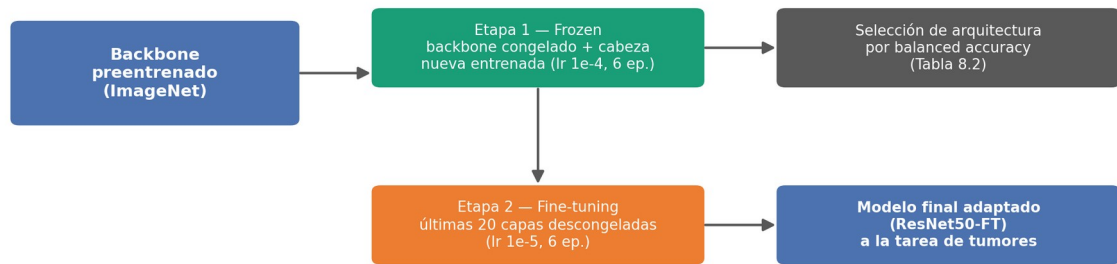
En la revisión bibliográfica realizada aparecen tanto arquitecturas convolucionales entrenadas desde cero como arquitecturas preentrenadas adaptadas mediante *transfer learning*. Esta segunda familia de enfoques, que incluye las arquitecturas descritas en el apartado 3.2, es también la seguida en este TFG. Junto a las CNN convolucionales clásicas, parte de la literatura reciente explora arquitecturas híbridas (por ejemplo, combinando CNN con clasificadores clásicos o con mecanismos de atención) y, en menor medida, arquitecturas basadas en transformers aplicadas a imagen médica. Este capítulo no pretende recorrer exhaustivamente esa evolución técnica, que puede consultarse en trabajos de revisión dedicados a Deep Learning aplicado a imagen médica, sino situar específicamente los elementos metodológicos —dataset, arquitectura, unidad de partición y controles de independencia— que resultan relevantes para interpretar y comparar los resultados de este TFG con los de la literatura.

Dentro de esta línea de trabajo, varios de los estudios recopilados se centran en la misma tarea de tres clases abordada en este TFG y emplean el dataset Cheng/figshare descrito en el apartado 3.3 o variantes combinadas con otros conjuntos de imágenes. Esto convierte a esta tarea y a este dataset en un punto de referencia habitual dentro de la literatura reciente, lo que resulta favorable para situar este TFG, pero no exime de examinar, para cada estudio citado, los elementos metodológicos que condicionan la comparabilidad de sus resultados, desarrollados en los apartados 3.4 y 3.6.

3.2 Transfer learning y arquitecturas de referencia

El *transfer learning* consiste en reutilizar una red ya entrenada sobre una tarea de origen —habitualmente clasificación de imágenes naturales sobre un banco de datos de gran tamaño— como punto de partida para una tarea distinta, en este caso la clasificación de tumores cerebrales. En la práctica, esto permite aprovechar las representaciones visuales aprendidas por la red y adaptarlas mediante entrenamiento adicional (parcial o completo) sobre el conjunto de imágenes médicas disponible, que suele ser considerablemente más pequeño que los bancos de datos usados en el entrenamiento original de la arquitectura.

Figura 3.1 — Esquema del transfer learning aplicado en este TFG



En ambas etapas la capa de BatchNormalization permanece congelada. La cabeza de clasificación es siempre nueva (no preentrenada); solo el backbone reutiliza pesos de ImageNet. Optimizador Adam y batch size 32 en las dos etapas (ver Tabla 6.2).

Figura 3.1 — Esquema del transfer learning aplicado en este TFG

Cinco arquitecturas preentrenadas resultan directamente relevantes para este TFG, por ser las evaluadas bajo un protocolo común en los capítulos 8 y 10: VGG16, ResNet50, MobileNetV2, EfficientNetB0 e InceptionV3. Cada una representa una familia arquitectónica distinta dentro de la literatura de visión por computador.

VGG16 (Simonyan y Zisserman) (2014) es una arquitectura convolucional profunda de diseño relativamente simple, construida apilando de forma homogénea bloques de convoluciones pequeñas seguidas de operaciones de *pooling*, hasta alcanzar una profundidad considerable para su época. Representa, dentro de las cinco arquitecturas consideradas, el enfoque más directo de “profundidad mediante apilamiento”.

ResNet50 (He, Zhang, Ren y Sun) (2016) introduce conexiones residuales: cada bloque de la red aprende una función residual respecto a su entrada, en lugar de una transformación completa, y esa entrada se suma directamente a la salida del bloque. Este mecanismo, motivado originalmente por las dificultades de entrenar redes muy profundas, es la característica que distingue a esta familia arquitectónica del resto.

MobileNetV2 (Sandler, Howard, Zhu, Zhmoginov y Chen) (2018) pertenece a una familia de arquitecturas orientadas a la eficiencia computacional, mediante bloques residuales invertidos y convoluciones separables en profundidad (*depthwise separable*), que reducen el número de parámetros y operaciones respecto a una convolución estándar equivalente.

EfficientNetB0 (Tan y Le) (2019) parte de un planteamiento distinto: en lugar de diseñar manualmente la arquitectura, propone un método de escalado compuesto que

ajusta de forma conjunta y proporcional la profundidad, la anchura y la resolución de entrada de la red, buscando un compromiso sistemático entre coste computacional y capacidad del modelo.

InceptionV3 (Szegedy, Vanhoucke, Ioffe, Shlens y Wojna) (2016) pertenece a la familia de arquitecturas con bloques Inception, que combinan en paralelo, dentro de un mismo bloque, convoluciones de distinto tamaño de filtro y operaciones de *pooling*, factorizando además las convoluciones grandes en combinaciones de convoluciones más pequeñas para reducir el coste computacional.

Estas cinco familias no se describen aquí con el detalle de sus hiperparámetros o de su configuración concreta de entrenamiento; ese nivel de detalle, específico de este TFG, se desarrolla en el capítulo 8. Tampoco se anticipa en este capítulo qué arquitectura resultó finalmente seleccionada como configuración final del proyecto, decisión que se presenta y justifica en el capítulo 10. Estas familias constituyen la base de las arquitecturas posteriormente evaluadas bajo un protocolo común en este trabajo, cuya configuración y resultados se desarrollan en los capítulos 8 y 10.

Figura 3.2 — Arquitecturas de referencia consideradas en la fase frozen

Comparación de tamaño (parámetros totales, escala pública de cada arquitectura original) y año de publicación de las cinco arquitecturas evaluadas en la fase frozen (Tabla 6.1). No representa resultados de accuracy de este TFG, solo el orden de magnitud de cada red antes de aplicar transfer learning.

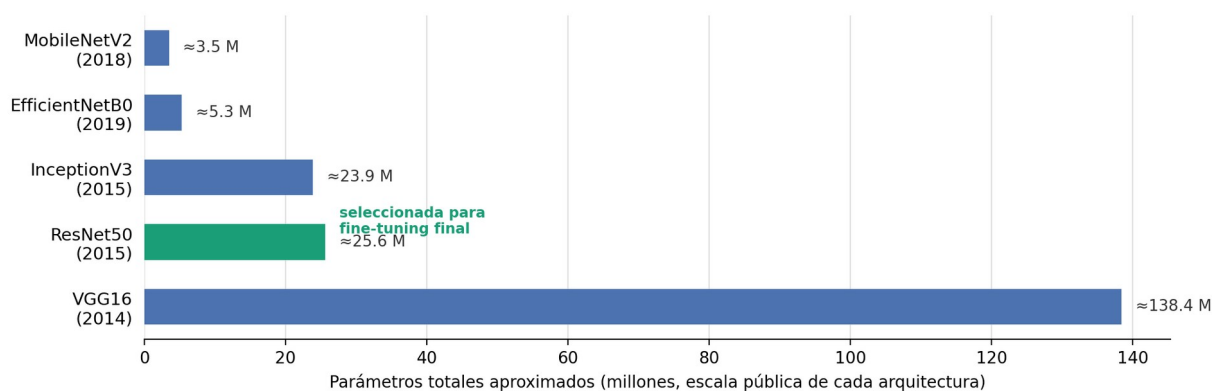


Figura 3.2 — Arquitecturas de referencia consideradas en la fase frozen

3.3 El dataset Cheng/figshare en la literatura

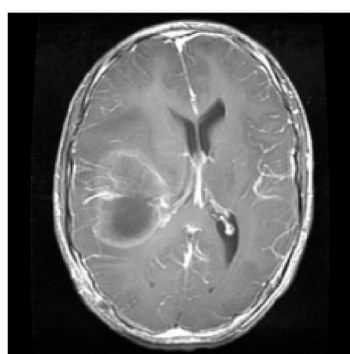
El dataset de Cheng et al., depositado públicamente en Figshare, aparece de forma recurrente en la revisión bibliográfica realizada sobre clasificación de tumores cerebrales mediante Deep Learning. Conviene distinguir el artículo original de Cheng et al. (2015), publicado en PLOS ONE, que describe la procedencia clínica y la tarea original para la que se recopilaron las imágenes, del depósito de datos en Figshare propiamente dicho (Cheng, 2024), que es el repositorio desde el que se distribuyen las imágenes bajo la licencia correspondiente (ya tratada en el capítulo 13). El dataset

contiene 3.064 cortes de RM en secuencia T1 con contraste, correspondientes a 233 pacientes distintos, distribuidos entre tres clases: glioma, meningioma y tumor pituitario.

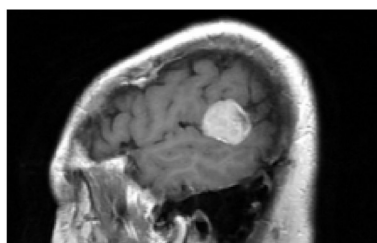
Este dataset es especialmente relevante para este TFG porque coincide exactamente con la cohorte final empleada en la tarea multiclase (véase el capítulo 6): los mismos 3.064 cortes y los mismos 233 pacientes. Esta coincidencia permite identificar en la literatura estudios que trabajan sobre la misma base de datos, lo cual resulta más informativo para la comparabilidad metodológica que comparar cifras entre datasets distintos.

Figura 3.3 — Ejemplos reales del dataset Cheng por clase (tarea multiclase)

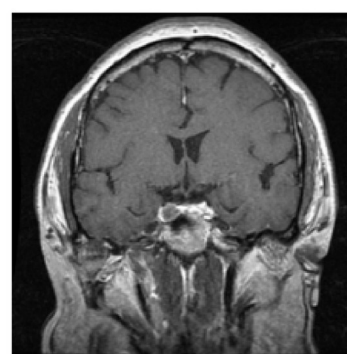
Cortes reales verificados, ya utilizados en la evaluación Grad-CAM (Anexo E); identificador de caso anonimizado del dataset (no es un dato personal identificable). El corte sagital de meningioma se ha recortado para eliminar el perfil facial, conservando únicamente cráneo, cerebro y tumor.



Glioma
caso CHENG-MR033389B (anonimizado)



Meningioma
caso CHENG-111077 (anonimizado)



Tumor pituitario
caso CHENG-105936 (anonimizado)

Figura 3.3 — Ejemplos reales del dataset Cheng por clase (tarea multiclase)

Dentro de los estudios que emplean este dataset, el trabajo de Bulla, Anantha y Peram (2020), publicado en *Traitement du Signal*, constituye un comparador metodológicamente relevante por utilizar la misma cohorte de referencia y explicitar una evaluación agrupada por paciente. Los autores comparan explícitamente varios esquemas de evaluación sobre el mismo conjunto de 3.064 imágenes y 233 pacientes: un holdout simple, una validación cruzada de 10 particiones a nivel de imagen, una versión estratificada de la misma, y una validación cruzada de 10 particiones agrupada por paciente, empleando una arquitectura InceptionV3 adaptada mediante *transfer learning*. El esquema agrupado por paciente es el que los propios autores destacan como el más adecuado para evitar que cortes del mismo paciente aparezcan simultáneamente en entrenamiento y evaluación, y para ese esquema los autores reportan una exactitud (*accuracy*) del 99,82 %. Esta cifra corresponde a *accuracy* y no a *balanced accuracy*, y que el esquema experimental de Bulla et al. —en particular, el tratamiento de la variante recortada o no recortada de las imágenes del dataset— no

está descrito con el detalle suficiente como para darlo por idéntico al protocolo seguido en este TFG. Por estas razones, la cifra de Bulla et al. no debe leerse como una comparación directa de rendimiento con los resultados de este trabajo, que se presentan y discuten en los capítulos 10 y 11; su valor aquí es principalmente metodológico, al mostrar que la propia literatura reconoce la diferencia entre evaluar a nivel de imagen y evaluar a nivel de paciente sobre esta misma base de datos.

Además de Bulla et al., la revisión bibliográfica propia recoge un número considerable de estudios adicionales que emplean este mismo dataset (o variantes combinadas con otros conjuntos de imágenes) bajo arquitecturas y protocolos heterogéneos, varios de ellos con cifras de exactitud elevadas. Dado que en la mayoría de esos estudios no se especifica con precisión suficiente la unidad de partición empleada, no resulta posible, a partir de la síntesis bibliográfica disponible, determinar en qué medida esas cifras son comparables entre sí o con los resultados de este TFG; esos estudios se recogen en el Anexo I del documento de anexos técnicos con su estado de verificación pendiente, en lugar de citarse por nombre en este apartado.

Como complemento a la Figura 3.3, la Figura 3.4 muestra un ejemplo real de cada una de las dos fuentes que componían la tarea binaria histórica tumor/no tumor explorada en las primeras fases del proyecto (véase la sección 6.6): un corte con tumor procedente del conjunto Cheng y un corte axial de un sujeto sin tumor procedente del conjunto IXI.

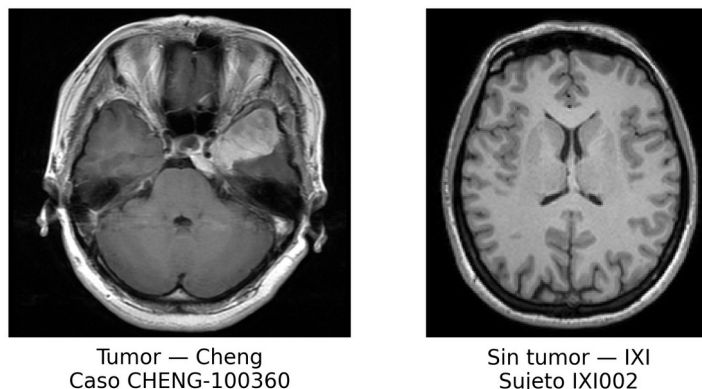


Figura 3.4 — Ejemplos reales de las fuentes empleadas en la tarea binaria histórica

A la izquierda se muestra un caso tumoral procedente del conjunto Cheng y, a la derecha, un sujeto sin tumor del conjunto IXI. Las imágenes se incluyen únicamente para ilustrar la composición de la tarea binaria explorada durante las primeras fases del proyecto. Dado que las dos clases procedían de fuentes distintas, esta tarea quedó afectada por el sesgo de procedencia analizado en el capítulo 6 y no constituye el resultado central del TFG.

El ejemplo Cheng se ha reconstruido directamente a partir del fichero 1.mat incluido en el paquete oficial del dataset distribuido mediante Figshare (DOI 10.6084/m9.figshare.1512427), cuyo campo PID corresponde al identificador de caso CHENG-100360 registrado en el manifiesto del proyecto. El ejemplo IXI corresponde al sujeto IXI002 (adquisición IXI002-Guys-0828-T1) del conjunto T1 del dataset IXI. Debido a que el servidor oficial de IXI no permitía, en el momento de preparar esta figura, descargar el paquete original IXI-T1.tar, el volumen utilizado se obtuvo de una copia de terceros alojada en Hugging Face, identificada con el mismo sujeto y adquisición y declarada por su autor como reorientada. Esta copia se ha empleado exclusivamente con finalidad ilustrativa: no forma parte del conjunto de datos experimental ni interviene en ningún resultado reportado en esta memoria. La trazabilidad completa de ambos paneles (ficheros fuente, procedencia y huellas SHA-256) se recoge en el Anexo A del documento de anexos técnicos.

3.4 Patient-level frente a image-level: independencia, leakage y duplicados

Esta distinción es, probablemente, el elemento metodológico más relevante que aporta la revisión bibliográfica para interpretar los resultados de este TFG. En datasets donde cada paciente aporta varios cortes de RM, existe más de una forma de repartir las imágenes entre entrenamiento, validación y evaluación. Una separación aleatoria realizada a nivel de imagen —es decir, sin agrupar previamente los cortes por paciente— puede dar lugar a que distintos cortes de un mismo paciente aparezcan simultáneamente en el conjunto de entrenamiento y en el conjunto de evaluación. En ese caso, el modelo puede beneficiarse, durante la evaluación, de haber visto durante el entrenamiento otros cortes muy similares del mismo paciente, lo que reduce la independencia real entre los conjuntos.

A lo largo de este capítulo se emplean tres términos con un significado técnico preciso: partición a nivel de imagen (*image-level*), en la que la unidad de reparto es el corte individual; partición a nivel de paciente (*patient-level*), en la que la unidad de reparto es el paciente completo, de modo que todos sus cortes quedan en el mismo subconjunto; y partición a nivel de sujeto (*subject-level*), empleado en algunos estudios con un significado equivalente a *patient-level* cuando la unidad clínica de interés no es estrictamente un “paciente” en sentido asistencial (por ejemplo, en estudios sobre voluntarios sanos). En el resto del capítulo se utilizan directamente los términos en inglés (*patient-level*, *image-level*, *subject-level*), ya asentados en la literatura y en el resto de esta memoria.

Es importante precisar el alcance de esta observación: que una separación sea a nivel de imagen no demuestra, por sí sola, que un estudio concreto tenga fuga de

información entre entrenamiento y evaluación, del mismo modo que una cifra de exactitud elevada tampoco demuestra por sí sola la existencia de dicha fuga. Lo que puede afirmarse es que, cuando el nivel de partición no está descrito con suficiente precisión en el artículo revisado, no puede determinarse el grado de independencia real entre las muestras de entrenamiento y evaluación, y que esa indeterminación limita la comparabilidad de la cifra reportada con resultados obtenidos bajo un protocolo *patient-level* explícito.

La síntesis bibliográfica propia, elaborada antes de la verificación individual de los estudios citados en este apartado, había agrupado provisionalmente los 39 estudios revisados en dos bloques según el nivel de partición y los controles de independencia descritos en cada resumen: un primer bloque, mayoritario, formado por estudios en los que el nivel de partición no se especificaba con precisión suficiente o parecía realizarse a nivel de imagen; y un segundo bloque, más reducido, formado por estudios con partición a nivel de paciente explícita o con algún otro control adicional de independencia (deduplicación de imágenes o validación externa). La síntesis bibliográfica previa agrupó provisionalmente los estudios en función del nivel de partición y de los controles de independencia descritos; esta clasificación se utiliza como herramienta de organización de la revisión y no como una estadística definitiva de toda la literatura, ya que no se ha verificado individualmente cada uno de los 39 estudios contra su artículo original. Dentro del segundo bloque, además, no todos los estudios comparten el mismo tipo de control: como muestran los tres casos verificados en los párrafos siguientes, partición a nivel de paciente explícita, deduplicación sin partición a nivel de paciente confirmada y validación externa entre datasets son controles distintos entre sí, y ninguno de los tres implica automáticamente los otros dos.

El trabajo de Nalawade et al. (2019), publicado en *Journal of Medical Imaging*, ilustra esta distinción con particular claridad, aunque sobre una tarea distinta de la abordada en este TFG: los autores predicen el estado mutacional IDH en gliomas (no el tipo de tumor) a partir de RM, sobre una cohorte de 260 sujetos, mediante varias arquitecturas convolucionales, entre ellas DenseNet-161, evaluadas con validación cruzada de 5 particiones y con una separación explícita de sujetos entre entrenamiento y evaluación antes de extraer los cortes. Los propios autores reportan dos cifras para el mismo modelo: una exactitud del 90,5 % cuando la evaluación se realiza por corte (*slice-level*), y una exactitud del 83,8 % cuando la evaluación se agrega por sujeto (*subject-level*). La diferencia entre ambas cifras, obtenidas sobre el mismo modelo y el mismo conjunto de datos, ejemplifica de forma directa por qué la unidad de evaluación —y no solo la arquitectura o el dataset— condiciona la

interpretación de un resultado. Dado que la tarea de Nalawade et al. es distinta (mutación IDH, no tipo tumoral), este estudio se emplea aquí exclusivamente como ejemplo metodológico sobre la unidad de evaluación, y no como comparador de rendimiento con este TFG.

El trabajo de Hassan et al. (2026), publicado en *Scientific Reports*, aporta una dimensión adicional: además de una validación cruzada estratificada de 5 particiones descrita por los propios autores como *patient-wise*, ejecutada sobre el 90 % de su conjunto de datos (con el 10 % restante reservado como test independiente), incorpora una validación externa sobre un segundo conjunto de datos distinto (aproximadamente 3.000 imágenes), obteniendo una exactitud interna del 98,3 % y una exactitud del 96 % en la validación externa. Este caso permite ilustrar que la partición a nivel de paciente dentro de un dataset y la validación externa sobre una cohorte distinta son dos controles metodológicos diferentes: un estudio puede disponer de partición *patient-wise* sin validación externa, y viceversa, por lo que ambos conceptos no deben agruparse bajo una misma categoría. La tarea de Hassan et al. es una clasificación binaria (tumor/no-tumor) sobre datasets distintos de Cheng, por lo que tampoco constituye un comparador directo de rendimiento con este TFG.

Un tercer matiz relevante lo aporta el trabajo de Arghistani, Erbay y Ozdil (2026), publicado en *Healthcare Analytics*, que emplea un conjunto de 7.023 imágenes (Kaggle, cuatro clases) y aplica un procedimiento de deduplicación mediante hashes SHA-256 y hash perceptual antes de repartir los datos en una proporción 70/15/15, alcanzando una exactitud del 97,10 % en el conjunto de test. Los propios autores señalan explícitamente que no pudo confirmarse una partición real a nivel de paciente, por no disponer de metadatos de identificación de paciente en las imágenes, y limitan en consecuencia el alcance clínico de sus conclusiones. Este caso es metodológicamente interesante porque muestra que deduplicar imágenes —eliminar copias exactas o casi idénticas— no equivale a garantizar una partición a nivel de paciente: son dos controles distintos, que pueden coexistir o darse por separado, y que abordan problemas diferentes (duplicados frente a dependencia entre muestras del mismo paciente).

En conjunto, estos tres estudios muestran que la literatura sobre clasificación de tumores cerebrales emplea, de forma heterogénea, controles de independencia de naturaleza distinta —partición a nivel de paciente, deduplicación de imágenes, validación externa entre datasets—, y que estos controles no son intercambiables entre sí ni implican automáticamente los otros dos.

3.5 Explicabilidad mediante Grad-CAM

Grad-CAM (*Gradient-weighted Class Activation Mapping*), descrito originalmente por Selvaraju et al. (2017), es un método de explicabilidad post-hoc para redes convolucionales, aplicable sin modificar la arquitectura de la red ni volver a entrenarla. A partir de los gradientes de una clase concreta respecto a las activaciones de una capa convolucional (habitualmente la última), Grad-CAM genera un mapa de importancia espacial de baja resolución que indica qué regiones de la imagen de entrada contribuyeron más a la predicción de esa clase. Este mapa puede superponerse visualmente sobre la imagen original para facilitar su interpretación.

En tareas de clasificación de imágenes, Grad-CAM puede emplearse como complemento visual de la predicción, pero su mapa de activación no constituye por sí mismo una validación cuantitativa de la localización espacial. Es importante mantener esta distinción con claridad: un mapa de Grad-CAM no es una segmentación, no delimita de forma precisa los bordes de una región, y no proporciona por sí mismo una validación espacial cuantitativa; para poder cuantificar en qué medida las regiones destacadas por Grad-CAM coinciden con la localización real del tumor sería necesario contrastar el mapa contra una referencia de segmentación (una máscara), lo cual constituye un análisis distinto del que ofrece Grad-CAM por sí solo.

Esta caracterización de Grad-CAM como herramienta cualitativa y post-hoc es coherente con el tratamiento que recibe en el resto de esta memoria: el capítulo 8 describe su implementación en este TFG, el capítulo 10 presenta los resultados visuales obtenidos, el capítulo 11 los interpreta, y el capítulo 13 discute explícitamente que el análisis Grad-CAM realizado en este trabajo no incorporó las anotaciones de máscara tumoral disponibles en el dataset Cheng como referencia cuantitativa. Este capítulo no repite ese desarrollo; se limita a situar Grad-CAM como método dentro de la literatura de explicabilidad aplicada a esta tarea.

3.6 Limitaciones de la comparación con la literatura

Los apartados anteriores muestran que dos cifras de exactitud publicadas en la literatura no son necesariamente comparables entre sí, ni con los resultados de este TFG, sin examinar antes un conjunto de factores metodológicos que pueden diferir de un estudio a otro: el dataset y la cohorte concreta empleados; el número de pacientes y de imágenes disponibles; la unidad a la que se realiza la partición (imagen, paciente o sujeto); el tipo de validación empleado (holdout simple, validación cruzada, y en su caso el número de particiones); la existencia o ausencia de un procedimiento de deduplicación; el preprocesado aplicado a las imágenes y el uso de técnicas de aumento de datos; la arquitectura y el grado de ajuste fino aplicado sobre ella; la

métrica concreta reportada (exactitud, *balanced accuracy*, F1, u otras) y la unidad sobre la que se calcula (por imagen, por paciente); y la existencia o ausencia de una validación externa sobre una cohorte distinta de la usada para entrenar y seleccionar el modelo.

No es necesario asumir que todos estos factores varíen en todos los estudios revisados para que la comparación resulte problemática: basta con que alguno de ellos no esté descrito con precisión suficiente, o difiera respecto al protocolo de este TFG, para que una diferencia numérica entre dos cifras deje de ser interpretable sin más contexto. Por esta razón, a lo largo de este capítulo se ha evitado deliberadamente presentar los estudios revisados como una clasificación ordenada de mejor a peor o emplear expresiones como “mejor resultado de la literatura”, y se ha limitado la comparación directa de cifras a los casos en los que el propio estudio proporciona el contraste necesario para interpretarlas (como ocurre con las cifras *slice-level* y *subject-level* de Nalawade et al., o con los distintos esquemas de evaluación comparados por Bulla et al.).

La comparación específica entre los resultados obtenidos en este TFG y los estudios de la literatura aquí presentados, en particular con Bulla et al. por compartir la misma cohorte de referencia, se retoma en el capítulo 11, una vez presentados los resultados propios del trabajo; este capítulo se limita a establecer el marco de comparabilidad bajo el que esa discusión posterior debe leerse.

La literatura revisada muestra que la elección de arquitectura es solo una de las variables que condicionan la interpretación de un resultado de clasificación de tumores cerebrales: la unidad de partición, la independencia entre pacientes, los controles de duplicados y el protocolo de validación empleado resultan igualmente determinantes. Esta observación motiva la atención particular que este TFG presta, en los capítulos siguientes, a la trazabilidad de los datos, a la partición a nivel de paciente, a la reproducibilidad del protocolo y a la validación interna de los resultados obtenidos. Por eso, este capítulo no busca anticipar si este TFG se sitúa por encima o por debajo de la literatura revisada, sino dejar establecido el marco de comparabilidad —qué factores deben comprobarse antes de leer una diferencia numérica como significativa— bajo el cual deben interpretarse tanto los resultados propios, presentados en los capítulos 10 y 11, como cualquier comparación futura con trabajos adicionales que pudieran incorporarse a esta revisión.

Tabla 3.1 — Estudios representativos verificados

Estudio	Dataset/tarea	Modelo	Unidad de partición	Evaluación	Métrica reportada	Observación de comparabilidad
Bulla, Anantha y Peram (2020)	Cheng/figshare (3.064 img./233 pac.), glioma/meningioma/pituitario	InceptionV3 (transfer learning)	Comparación explícita: image-level y patient-level (10-fold agrupado por paciente)	Holdout y tres esquemas de 10-fold (imagen, imagen estratificado, agrupado por paciente)	Accuracy: 99,82 % (esquema agrupado por paciente)	Mismo dataset que este TFG; accuracy, no balanced accuracy; esquema de recorte de imagen no completamente descrito
Nalawade et al. (2019)	TCIA, mutación IDH en gliomas (260 sujetos) — tarea distinta a la de este TFG	CNN 2D (DenseNet-161, mejor resultado; también ResNet-50 e Inception-v4)	Subject-level explícito (5-fold, separación de sujetos antes de extraer cortes)	Validación cruzada de 5 particiones	Accuracy: 90,5 % (slice-level) / 83,8 % (subject-level)	Tarea distinta (mutación IDH, no tipo tumoral); útil solo como ejemplo metodológico slice-vs. subject-level
Hassan et al. (2026)	Dos datasets Kaggle: A (253 img., tumor/no-tumor) y B (Br35H, ≈3.000 img., validación externa)	InceptionV3 + Nadam	Patient-wise (según los autores), estratificado, 5-fold sobre el 90 % de los datos	CV interna patient-wise + validación externa sobre dataset B	Accuracy: 98,3 % (interna) / 96 % (validación externa)	Tarea binaria tumor/no-tumor, distinta de la clasificación en 3 clases de este TFG; ilustra patient-wise + validación externa como controles distintos
Arghistani, Erbay y Ozdil (2026)	Kaggle (7.023 img., 4 clases: glioma/meningioma/pituitario/no-tumor)	PMS-Net (CNN multi-rama con atención SE)	Deduplicación (SHA-256 + hash perceptual) sin patient-level	Split 70/15/15	Accuracy: 97,10 % (test)	Deduplicación ≠ patient-level; los propios autores limitan el alcance clínico por ausencia de

Estudio	Dataset/tarea	Modelo	Unidad de partición	Evaluación	Métrica reportada	Observación de comparabilidad
			confirmado (autores lo señalan explícitamente)			metadatos de paciente

Nota: esta tabla recoge únicamente los estudios verificados individualmente contra su fuente original. El resto de estudios de la revisión bibliográfica propia se recoge en el Anexo I del documento de anexos técnicos con su estado de verificación indicado en cada caso.

4. Planificación y gestión del proyecto

4.1 Enfoque de planificación

Este capítulo documenta cómo se planificó, gestionó y ejecutó este TFG a lo largo de su desarrollo, y lo hace distinguiendo con cuidado dos cosas que conviene no confundir: la planificación prospectiva (lo que, con la información disponible en cada momento, se esperaba hacer) y el seguimiento retrospectivo (lo que finalmente se hizo, reconstruido a partir de los artefactos y del registro temporal disponible).

El proyecto no se gestionó desde el inicio con una metodología formal de las habitualmente empleadas en desarrollo de software — no se ha documentado el uso de Scrum, Kanban, sprints, un backlog formal ni un diagrama de Gantt previo y completo. La descripción que se ajusta a la evidencia disponible es la de un desarrollo iterativo guiado por evidencia experimental y por revisión metodológica: el alcance, las herramientas y el propio problema a resolver se fueron ajustando a medida que aparecían resultados, se detectaban problemas en los datos o en el pipeline, y se recibía retroalimentación externa del tutor y del cotutor. Esto no equivale a improvisación: varias de las decisiones más importantes del proyecto —la reconstrucción del dataset, el abandono de la tarea binaria como núcleo, la incorporación de una validación cruzada adicional— fueron respuestas documentadas a problemas concretos, no ajustes de un plan que no llegó a existir por escrito.

Por ese motivo, la Estructura de Descomposición del Trabajo (EDT) y el diagrama de Gantt que se presentan en este capítulo no son una reconstrucción de un plan que existiera en su momento, sino una organización a posteriori del trabajo realmente ejecutado en diez paquetes de trabajo coherentes, acompañada de una estimación de referencia de las horas que, con criterio conservador, cabría esperar que se hubieran previsto al inicio del proyecto para cada paquete. Ambas cosas —lo realmente dedicado y la estimación de referencia— se presentan siempre por separado y nunca se mezclan en una única cifra.

4.2 Evolución temporal del proyecto

El proyecto arrancó en septiembre de 2025, antes de que su autor tuviera claro que acabaría siendo este TFG, con un repaso de los fundamentos de redes convolucionales (convolución, pooling, entrenamiento y validación) y una primera adaptación de esos conceptos a imágenes de resonancia magnética cerebral. A principios de octubre de 2025 existía ya un primer clasificador binario tumor/no-tumor funcional, aunque de carácter muy experimental, construido sobre datasets agregados de Kaggle sin identificador de paciente real. En esa misma etapa temprana se desarrolló también una línea de trabajo sobre interfaz e integración conocida como ScanIA (visor HTML, Dockerfile orientado a Cloud Run, inferencia 2D sobre volúmenes NIfTI vinculada a la tarea binaria de entonces), documentada con precisión en el capítulo 5 de esta memoria.

Entre diciembre de 2025 y abril de 2026 el proyecto quedó prácticamente en pausa, con actividad puntual de revisión. Se retomó de forma más seria en mayo y junio de 2026, coincidiendo con la formalización del TFG y el primer contacto con el tutor. En julio de 2026 la actividad aumentó de forma notable: se revisaron el prototipo original, los datasets disponibles y el desarrollo previo de ScanIA, y se preparó el planteamiento metodológico que seguiría el proyecto a partir de entonces —en particular, la detección de que el dataset inicial no permitía establecer con fiabilidad la correspondencia entre imagen y paciente real, lo que motivó la reconstrucción patient-level que se convertiría en el eje del trabajo.

El grueso del proyecto se concentra en agosto y septiembre de 2026: en estas semanas se completó la reconstrucción patient-level del dataset (fuentes Cheng e IXI con identificador real), se resolvieron los controles de duplicados y el sesgo de procedencia detectado en la tarea binaria reconstruida, se produjo la reorientación hacia la clasificación multiclase, se compararon arquitecturas bajo un protocolo común, se ejecutaron los trabajos en el clúster HPC de la Universidad de Sevilla y se obtuvieron los resultados de holdout, 5CV y Grad-CAM presentados en los capítulos de resultados y discusión de esta memoria. A esto se suma la redacción de la memoria, la preparación de tablas y figuras,

el montaje de los anexos y la maqueta final, que por sí solas representan varias decenas de horas dentro del total.

La Tabla 4.3 y la Figura 4.2 detallan esta evolución por paquete de trabajo y mes; la Tabla 4.1 la resume por mes en términos globales.

Tabla 4.1 — Dedicación real por mes (resumen)

Periodo	Horas	Actividad predominante
Septiembre 2025	18 h 34 min	Aprendizaje inicial y primeras pruebas
Octubre 2025	27 h 59 min	Desarrollo del clasificador binario y del prototipo ScanIA
Noviembre 2025	9 h 42 min	Revisión puntual antes de la pausa
Diciembre 2025 – abril 2026	0 h 00 min	Proyecto en pausa
Mayo – junio 2026	28 h 29 min	Retoma y formalización del TFG
Julio 2026	39 h 52 min	Prototipado, revisión y preparación metodológica
Agosto 2026	88 h 25 min	Reconstrucción patient-level y experimentación
Septiembre 2026	93 h 15 min	Evaluación final, redacción, maqueta y despliegue operativo
Total	306 h 19 min	—

(Tabla ya verificada en fases anteriores del proyecto; se traslada aquí desde el antiguo capítulo de gestión sin modificar sus cifras.)

Conforme el proyecto avanzó hacia la tarea multiclase sobre Cheng, la secuencia de trabajo —descrita con detalle metodológico en el capítulo 9 y cuyos resultados se presentan en el capítulo 10 de esta memoria— fue consolidando progresivamente el protocolo experimental: varias decisiones metodológicas quedaron fijadas por escrito antes de la evaluación posterior, en particular el protocolo de evaluación final se fijó antes de abrir el conjunto de holdout, y la validación cruzada de cinco particiones se ejecutó después, con el

modelo y el protocolo ya seleccionados, sin reabrir la selección de arquitectura. Esta fijación previa del protocolo, junto con la verificación de los hashes registrados al inicio y al final de cada evaluación, constituye un mecanismo de control de configuración que no formaba parte de las primeras iteraciones del proyecto.

La Tabla 4.2 resume, a modo de síntesis de gestión, las principales etapas del proyecto junto con el problema u objetivo que las motivó, la decisión adoptada y su consecuencia práctica.

Tabla 4.2 — Evolución del proyecto: problema/objetivo, decisión y consecuencia

Etapas	Problema/objetivo	Decisión	Consecuencia
Prototipo inicial	Clasificación binaria exploración interfaz	Desarrollo preliminar y sobre datasets de Kaggle disponibles	Primeros prototipos experimentales (clasificación y visualización)
Auditoría de datos	Falta de trazabilidad real de paciente (group_id no identifica pacientes reales)	Reconstrucción desde fuentes con identificador de paciente real	Base patient-level (Cheng + IXI)
Tarea binaria reconstruida	Sesgo de procedencia (la etiqueta coincide con el origen de los datos)	No usar la tarea binaria como núcleo final	Reorientación a la tarea multiclase sobre Cheng
Multiclase	Comparar modelos bajo un protocolo común	Benchmark de arquitecturas + fine-tuning	Selección de un candidato según criterio fijado por escrito
Evaluación final	Estimar el rendimiento	Holdout reservado, protocolo fijado por	Resultado del protocolo pre-5CV

modelo ya fijado escrito y verificado
 por hash

Evaluación Analizar la 5CV sobre los 233 Caracterización
 de sensibilidad del pacientes de la de la variabilidad
 variabilidad resultado al cohorte interna observada
 interna particionado

4.3 EDT — Estructura de Descomposición del Trabajo

La Figura 4.1 muestra la EDT resultante: diez paquetes de trabajo (PT1–PT10), contruidos agrupando las 101 sesiones de trabajo registradas y clasificadas por su descripción y categoría en el registro Clockify, coherentes con las etapas reales del proyecto ya narradas en los apartados anteriores.

Figura 4.1 — Estructura de Descomposición del Trabajo (EDT)
 Diez paquetes de trabajo (PT1-PT10) derivados de la actividad real documentada en el registro Clockify (tabla 4.1).

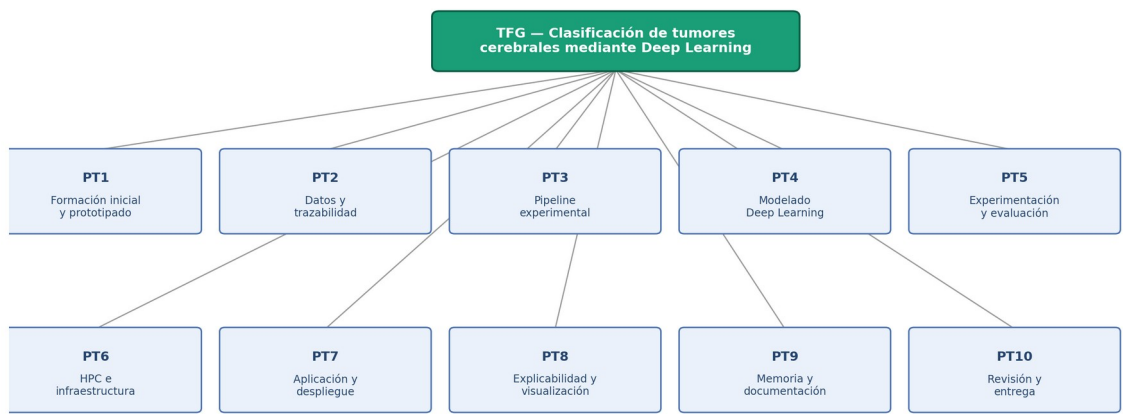


Figura 4.1 — Estructura de Descomposición del Trabajo (EDT)

Tabla 4.3 — EDT con horas previstas (estimación de referencia reconstruida a posteriori), horas reales y desviación

Paquete	Descripción	Horas previstas*	Horas reales**	Desviación
PT1	Formación inicial y prototipado	15,0 h	23,4 h	+8,4 h (+56 %)
PT2	Datos y trazabilidad	25,0 h	63,1 h	+38,1 h

					(+152 %)
PT3	Pipeline experimental		18,0 h	11,0 h	-7,0 h (-39 %)
PT4	Modelado Deep Learning		12,0 h	5,9 h	-6,1 h (-51 %)
PT5	Experimentación y evaluación	y	70,0 h	83,1 h	+13,1 h (+19 %)
PT6	HPC e infraestructura		10,0 h	13,3 h	+3,3 h (+33 %)
PT7	Aplicación despliegue	y	8,0 h	14,4 h	+6,4 h (+80 %)
PT8	Explicabilidad visualización	y	4,0 h	3,3 h	-0,7 h (-17 %)
PT9	Memoria documentación	y	45,0 h	66,1 h	+21,1 h (+47 %)
PT10	Revisión y entrega		15,0 h	22,7 h	+7,7 h (+51 %)
Total			222,0 h	306,3 h	+84,3 h (+38 %)

* Estimación de referencia reconstruida a posteriori: estas cifras no proceden de una planificación formal registrada al inicio del proyecto, que no existió, sino de una reconstrucción razonada de lo que cabría esperar de un planteamiento inicial conservador para un TFG de estas características. Se presentan únicamente como término de comparación para el análisis de desviaciones del apartado 4.7, nunca como una medición.

** Horas reales: derivadas directamente de las 101 sesiones de trabajo registradas en el informe resumido de Clockify (registro completo del proyecto, sin escalados ni estimaciones intermedias), reclasificadas una a una en los diez paquetes de trabajo. La suma de las horas reales de PT1 a PT10 coincide de forma exacta con el total registrado de 306 h 19 min en 101 sesiones.

4.4 Estimación de esfuerzo por paquetes de trabajo

La estimación de referencia de la Tabla 4.3 se construyó bajo un criterio explícito y deliberadamente simple: asumir, para cada paquete, la dedicación que un planteamiento inicial razonable —sin conocer todavía los problemas de trazabilidad del dataset agregado ni el volumen real de redacción de una memoria de estas características— habría podido prever. En particular:

Se asumió una fase de formación y prototipado breve (PT1), sin anticipar que el prototipo ScanIA y las primeras pruebas ocuparían varias sesiones adicionales.

Se infravaloró de forma notable el paquete de datos y trazabilidad (PT2), precisamente porque el problema de fondo —que el dataset inicial no permitía identificar pacientes reales— no era conocido al principio del proyecto; su detección fue, como se explica en el capítulo de estado del arte y en el de datos, uno de los puntos de inflexión metodológicos del trabajo.

Se sobrevaloró el paquete de pipeline experimental (PT3) y el de modelado (PT4) bajo la hipótesis, habitual en un planteamiento inicial optimista, de que construir y configurar el pipeline y el transfer learning llevaría más tiempo que ejecutarlo y evaluarlo; en la práctica ocurrió lo contrario, y la mayor parte del esfuerzo de modelado se concentró en la ejecución y el análisis de resultados (PT5).

Se infravaloraron de forma consistente los paquetes de memoria y documentación (PT9) y de revisión y entrega (PT10), un patrón habitual en la planificación de trabajos académicos.

Esta estimación no pretende ser una medición retrospectiva alternativa ni sustituir a las horas reales: su único propósito es servir de referencia para el análisis de desviaciones del apartado 4.7.

4.5 Planificación temporal y diagrama de Gantt

Tal y como señaló el tutor durante la revisión de la preentrega, este capítulo debe incluir una planificación temporal explícita en forma de diagrama de Gantt. Dado que no existió un diagrama de Gantt formal definido al inicio del proyecto, la Figura 4.2 presenta una reconstrucción retrospectiva de su ejecución temporal a partir de la cronología y del registro de dedicación disponible: cada barra muestra, para un paquete de trabajo, los periodos

mensuales en los que se registró actividad real, con las horas dedicadas indicadas sobre cada segmento. El hueco entre diciembre de 2025 y abril de 2026 no es una ausencia de datos, sino la pausa real del proyecto ya documentada en el apartado 4.2.

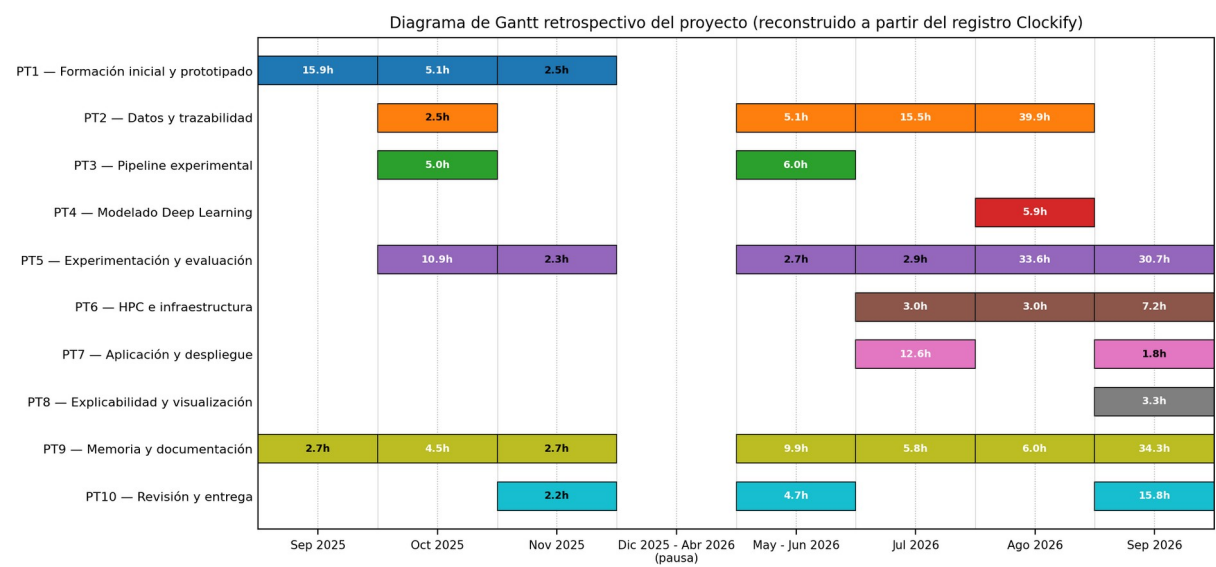


Figura 4.2 — Diagrama de Gantt retrospectivo del proyecto

A modo de complemento del Gantt anterior, la Figura 4.3 presenta la misma información de dedicación real en forma de mapa de calor mensual por paquete de trabajo, construida directamente a partir del registro Clockify (sin estimaciones ni relleno de huecos): cada celda es la suma de horas dedicadas ese mes a ese paquete, y resulta útil para apreciar la intensidad relativa de la actividad en cada periodo.

Figura 4.3 — Dedicación real por paquete de trabajo y mes (mapa de calor)
 Cada celda es la suma de horas registradas ese mes para ese paquete de trabajo; el hueco dic. 2025 - abr. 2026 refleja la pausa real del proyecto (capítulo 4.2).

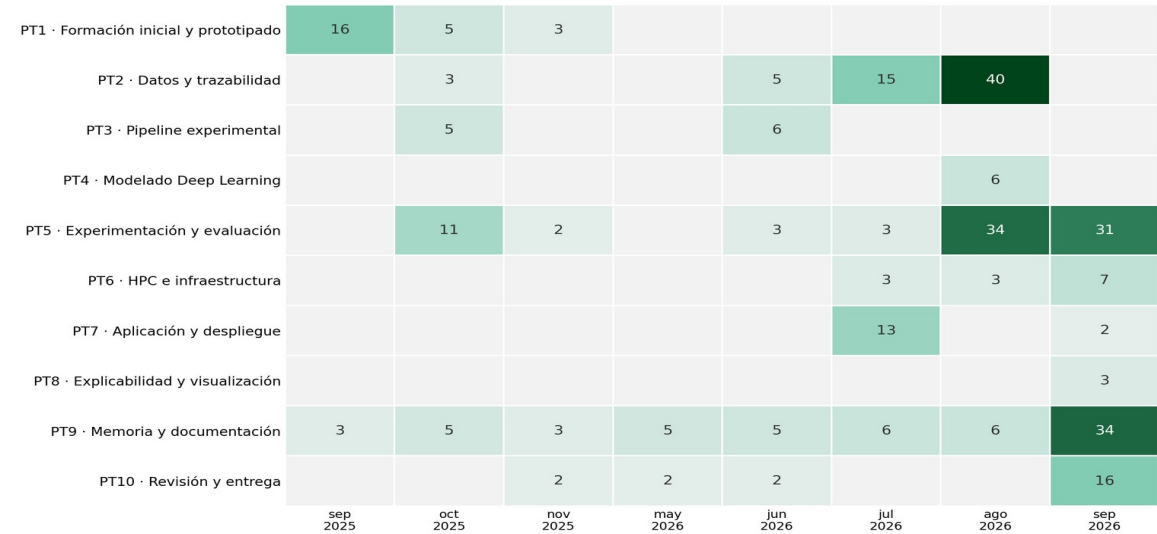


Figura 4.3 — Dedicación real por paquete de trabajo y mes (mapa de calor)

Este mapa de calor complementa el Gantt de la Figura 4.2 mostrando, además de los periodos activos, la intensidad relativa de la dedicación en cada uno. Se leen en él tres patrones relevantes para la gestión del proyecto: (1) los paquetes PT1, PT3 y PT9 tienen actividad temprana, coherente con el prototipado inicial y la documentación continua a lo largo de todo el proyecto; (2) PT2 (datos y trazabilidad) se concentra de forma muy marcada en julio-agosto de 2026, el periodo en el que se ejecutó la reconstrucción patient-level; y (3) PT5, PT6, PT8 y PT10 se concentran en las últimas semanas (agosto-septiembre de 2026), coherente con la ejecución de los experimentos finales en HPC, el análisis de resultados, el Grad-CAM y el cierre de la memoria.

4.6 Seguimiento real del esfuerzo

El seguimiento real del esfuerzo se apoya íntegramente en el registro de Clockify ya introducido: 306 h 19 min repartidas en 101 sesiones de trabajo entre el 8 de septiembre de 2025 y el 24 de septiembre de 2026. Como ya se advertía en versiones anteriores de esta memoria, las fechas y duraciones más antiguas (correspondientes a las primeras semanas del proyecto) no se registraron en el momento en que ocurrieron, sino que se reconstruyeron después a partir de los artefactos disponibles —código, notebooks, capturas y anotaciones—; solo las últimas semanas de redacción y maquetación se registraron ya en tiempo real. Por tanto, estas cifras deben leerse como una

estimación razonada de la dedicación real, y no como una medición cronométrica exacta desde el primer día. Esta distinción es importante porque no debe interpretarse que el registro Clockify estuvo operativo desde septiembre de 2025: la herramienta se empezó a usar en tiempo real mucho después, y el resto del historial es una reconstrucción posterior, honesta pero no instrumentada.

4.7 Análisis de desviaciones

La Figura 4.4 compara, paquete a paquete, la estimación de referencia (apartado 4.4) con la dedicación real. El proyecto, en conjunto, consumió un 38 % más de horas de las previstas en esa estimación de referencia (306,3 h frente a 222,0 h).

Figura 4.4 — Horas previstas frente a horas reales por paquete de trabajo

Total previsto: 222 h · Total real: 306 h 19 min (+38 %).
La mayor desviación se concentra en PT2 (+152 %), coherente con la reconstrucción patient-level no anticipada (capítulo 4.7).

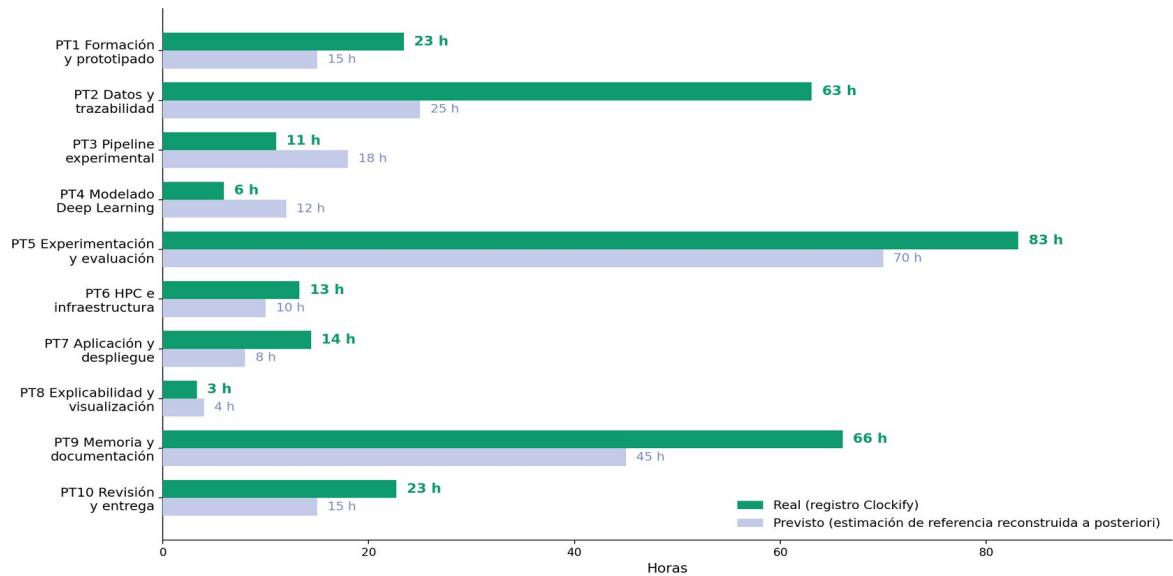


Figura 4.4 — Horas previstas frente a horas reales por paquete de trabajo

La desviación no se reparte de forma homogénea, y su patrón es coherente con la narrativa ya presentada en el capítulo de estado del arte y en el de datos: la mayor desviación en términos absolutos y relativos se concentra en PT2 (Datos y trazabilidad), con un +152 % sobre la estimación de referencia. Esto no es un fallo de gestión, sino la consecuencia directa de un hallazgo metodológico que no podía anticiparse al inicio: la imposibilidad de identificar pacientes reales en el dataset agregado inicial, que obligó a

reconstruir el conjunto de datos desde fuentes con identificador real (Cheng, IXI) y a incorporar controles de duplicados y de sesgo de procedencia no previstos originalmente.

Las siguientes desviaciones más relevantes son PT9 (Memoria y documentación, +47 %) y PT10 (Revisión y entrega, +51 %), ambas coherentes con un patrón habitual en trabajos académicos de esta envergadura: la redacción, la preparación de tablas y figuras, el montaje de anexos y la revisión editorial suelen consumir más tiempo del previsto, especialmente cuando — como en este caso— se incorpora feedback externo del tutor y del cotutor que exige revisiones estructurales adicionales (esta misma fase de reestructuración post-feedback es, de hecho, una instancia más de esa desviación). PT7 (Aplicación y despliegue) es el que más se desvió al alza en términos relativos después de PT2 (+80 %, aunque en términos absolutos sigue siendo un paquete pequeño, 14,4 h), coherente con el trabajo adicional de diagnóstico y corrección de la incidencia de Cloud Run documentada en el capítulo de viabilidad, la integración final del modelo multiclase en el prototipo funcional y la grabación del vídeo demostrativo del software.

En sentido contrario, PT3 (Pipeline experimental, -39 %) y PT4 (Modelado Deep Learning, -51 %) se completaron con menos horas de las previstas, porque buena parte del esfuerzo que un planteamiento inicial habría asignado a "construir y configurar" el pipeline y los modelos se desplazó, en la práctica, hacia "ejecutar y analizar" (PT5), que sí se desvió al alza (+19 %).

4.8 Estimación de costes

Esta estimación de costes es una simulación con fines de planificación de proyecto, no una liquidación económica real: el alumno no ha cobrado los importes de personal que se calculan a continuación, y la mayor parte de los recursos técnicos empleados son institucionales o ya sufragados por terceros (clúster HPC de la Universidad de Sevilla, cuenta de Google Cloud del proyecto). Se distingue en todo momento entre coste real (lo efectivamente pagado o facturado, cuando se conoce) y coste teórico equivalente (lo que costaría reproducir el mismo trabajo en condiciones de mercado, bajo supuestos explícitos y conservadores).

4.8.1 Coste de personal (simulado)

Tarifa de referencia. Se toma como base un salario bruto anual de 30.000–35.000 € para un perfil junior de ingeniería informática en España en 2026 (fuente: Universidad Francisco de Vitoria, "Cuánto cobra un ingeniero informático en España en 2026"), con punto medio de 32.500 €/año. Aplicando un factor de coste de empresa del 33 % (cotizaciones sociales a cargo del empleador, valor de referencia habitual en España) sobre una jornada de referencia de 1.750 horas productivas al año, resulta una tarifa de coste-empresa de aproximadamente 25 €/hora, que se redondea a la baja por prudencia. No se ha localizado una fuente pública específica y suficientemente fiable que diferencie tarifas de mercado entre un ingeniero ML/Data Scientist, un perfil DevOps/Cloud y un perfil de documentación técnica junior en España; en lugar de introducir cifras distintas sin respaldo para cada rol, se aplica esta misma tarifa única y conservadora a los cuatro roles simulados, explicitando esta simplificación.

Tabla 4.4 — Coste de personal simulado, por rol

Rol simulado	Paquetes de trabajo	Horas reales	Tarifa (coste empresa)	Coste simulado
Ingeniero ML / Data Scientist	PT1, PT2, PT4, PT5, PT8	178,9 h	25 €/h	4.471 €
Ingeniero de Software	PT3, PT7	25,4 h	25 €/h	635 €
DevOps / Cloud	PT6	13,3 h	25 €/h	332 €
Documentación / gestión técnica	PT9, PT10	88,8 h	25 €/h	2.219 €
Total		306,3 h		≈ 7.658 €

Esta cifra es un coste teórico equivalente: representa lo que costaría, en condiciones de mercado y con una única tarifa de referencia, remunerar el número de horas reales dedicadas a este proyecto. No es, en ningún caso, una afirmación de que el alumno haya percibido este importe.

4.8.2 Coste de recursos técnicos (hardware y cloud)

Tabla 4.5 — Coste real frente a coste teórico equivalente de los recursos técnicos

Recurso	Uso real documentado	Coste real	Coste teórico equivalente
Clúster HPC / GPU NVIDIA A30 (Universidad de Sevilla)	16 jobs SLURM documentados (capítulo 12, Tabla 12.1); solo 4 de ellos tienen wall-clock GPU confirmado con exactitud, sumando $\approx 1,15$ GPU-hora	0 € (recurso institucional compartido, sin facturación al alumno)	≈ 4 € (cota inferior: $1,15 \text{ h} \times 3,67 \text{ \$/h}$, tarifa pública on-demand de Google Cloud para NVIDIA A100 40GB a fecha de esta memoria — no existe tarifa pública on-demand para A30 en las nubes principales, por lo que se usa la referencia más cercana disponible; el coste real total del uso de GPU sigue sin poder calcularse de forma completa, como ya concluye el capítulo de viabilidad, al no estar documentado el wall-clock de la mayoría de jobs)

Equipo de desarrollo personal (portátil/sobremesa)	Usado durante los ≈13 meses de calendario del proyecto (con la pausa de diciembre 2025–abril 2026 sin actividad)	0 € (equipo ya en propiedad del alumno)	≈ 300 € (supuesto en conservador: equipo valorado en 1.200 €, amortizado a 3 años, prorrateado a los ≈9 meses de actividad efectiva)
Google Cloud Storage (buckets de imágenes, modelos y heatmaps)	Almacenamiento de unos pocos GB durante el desarrollo y la fase de despliegue operativo	No facturado de forma identificable al alumno / cubierto por el nivel gratuito o un importe residual	Del orden de 1–2 €/mes durante el periodo de uso activo (tarifa pública de referencia de GCS, orden de magnitud, no cifra facturada)
Cloud Run (servicio run-model)	Servicio de inferencia con volumen de peticiones bajo (pruebas + vídeo demostrativo)	No facturado de forma identificable al alumno / cubierto por el nivel gratuito o un importe residual	Del orden de unos pocos euros en el periodo de uso activo (tarifa pública de referencia de Cloud Run, orden de magnitud)
Total aproximado		0 €	≈ 305–310 €

Como ya concluye el capítulo de viabilidad y recursos de esta memoria, no existe una base económica suficientemente completa para calcular un coste monetario total defendible de la infraestructura técnica empleada; las cifras de esta tabla son, de forma explícita, órdenes de magnitud y cotas inferiores sobre

lo confirmado, no una facturación real reconstruida. Además, por prudencia respecto a la mezcla de divisas: la tarifa de Google Cloud usada como referencia para el GPU cotiza en dólares estadounidenses (3,67 \$/h); la conversión aproximada a euros de esta tabla aplica un tipo de cambio de referencia de 1 USD $\approx$ 0,92 EUR (orden de magnitud a fecha de esta memoria, sin pretensión de exactitud de tipo de cambio en tiempo real), lo que da el $\approx$ 4 € indicado arriba. Se insiste también en que la tarifa de NVIDIA A100 se usa exclusivamente como referencia de orden de magnitud ante la ausencia de tarifa pública on-demand para NVIDIA A30: ambas GPU no son equivalentes en prestaciones ni en precio, y esta cifra no debe leerse como el coste real de una A30.

4.8.3 Coste teórico equivalente total del proyecto

Sumando el coste de personal simulado ($\approx$ 7.658 €) y el coste teórico equivalente de recursos técnicos ($\approx$ 305–310 €), el coste teórico equivalente total de reproducir este proyecto en condiciones de mercado sería del orden de $\approx$ 7.963–7.968 €. Se insiste en que esta cifra es una simulación de planificación con fines exclusivamente académicos, construida sobre tarifas de referencia públicas y supuestos explícitos y conservadores, y no una liquidación económica real ni una afirmación sobre lo que el alumno ha cobrado o gastado.

4.9 Hitos y feedback durante el desarrollo

La cronología documentada de los hitos principales, usando las fechas tal como constan en los artefactos originales, es la siguiente: antes del 24/08/2026 se detectó el problema de trazabilidad del dataset inicial; el 24/08/2026 quedó documentada la reconstrucción patient-level; el 27/08/2026 se ejecutó el run que reconstruyó la tarea binaria y generó el control multiclase de bajo nivel; entre el 2 y el 3/09/2026 se realizaron la selección del modelo y la evaluación de holdout final; el 4/09/2026 se completó la validación cruzada de cinco particiones; y el 24/09/2026, ya en una sesión de trabajo posterior al cierre de la redacción de los capítulos experimentales, se diagnosticó y corrigió la incidencia de disponibilidad del servicio Cloud Run run-model (documentada en el Anexo J del documento de anexos técnicos) y se produjo el material audiovisual de demostración del software.

Varias de las decisiones más importantes del proyecto estuvieron precedidas por revisiones y cuestiones planteadas durante el desarrollo con los tutores (necesidad de trazabilidad del origen de cada imagen, necesidad de un split a nivel de paciente o de volumen, comprobaciones de fuga de información, y revisión del uso de canales de entrada en la CNN propia del proyecto). No se dispone del correo original de esas comunicaciones, por lo que esta evidencia es documental secundaria y no se presenta como cita textual. Estas cuestiones no diseñaron por sí mismas la solución adoptada: actuaron como un requisito o una preocupación metodológica que el proyecto respondió después con una implementación concreta, en particular con la reconstrucción patient-level (PT2) y los controles de disjunción por paciente y de unicidad por hash.

La Tabla 4.6 recoge, de forma resumida, las cuestiones metodológicas planteadas durante el desarrollo del proyecto y la respuesta concreta que este incorporó frente a cada una de ellas.

Tabla 4.6 — Cuestión metodológica planteada y respuesta incorporada al proyecto

Cuestión metodológica	Respuesta incorporada
Trazabilidad del origen de cada imagen	Reconstrucción del dataset desde fuentes con identificador de paciente real (Cheng/IXI)
Necesidad de separación por paciente o volumen	Partición por patient_id en todas las etapas (selección, holdout, 5CV)
Controles frente al riesgo de fuga de información	Verificación de unicidad por SHA-256 y disjunción de patient_id entre conjuntos
Revisión del uso de canales de entrada en la CNN propia	Distinción documentada entre la CNN propia (1 canal) y los modelos preentrenados (3 canales)
Sensibilidad del resultado final al particionado	Incorporación de una validación cruzada de cinco particiones (5CV) como evolución metodológica propia del proyecto

4.10 Síntesis

Este capítulo ha reorganizado, con una perspectiva explícita de planificación y gestión de proyecto software, el mismo desarrollo ya narrado como evolución metodológica en el resto de esta memoria: una EDT de diez paquetes de trabajo derivada de la actividad real, un Gantt real basado íntegramente en el registro Clockify, un análisis de desviaciones que conecta cada paquete con los hallazgos metodológicos que lo explican, y una estimación de costes simulada, construida de forma explícita y conservadora, que distingue en todo momento entre lo real (306 h 19 min, 0 € facturados directamente al alumno) y lo teórico equivalente (≈ 8.000 € si este trabajo se hubiera contratado en condiciones de mercado).

El proyecto evolucionó desde una prueba inicial centrada en un clasificador binario y en la exploración de una interfaz de visualización (ScanIA), hacia un proceso en el que la trazabilidad y la independencia por paciente pasaron a ser requisitos centrales del diseño experimental, y, en una fase posterior, hacia la integración de los modelos entrenados en un demostrador software actualizado que se documenta en el capítulo siguiente. Ese cambio de prioridad no fue una decisión aislada de gestión, sino la respuesta directa a la constatación de que el dataset inicial no permitía demostrar adecuadamente la independencia entre pacientes —el mismo hallazgo que explica, en este capítulo, por qué el paquete de datos y trazabilidad es el que más se desvió de cualquier estimación inicial razonable.

5. Análisis y diseño de la aplicación

5.1 Introducción y objetivo

Este capítulo documenta, con la perspectiva de Ingeniería del Software que enmarca este TFG, el demostrador software que permite ejecutar los modelos entrenados en este trabajo e interactuar con sus resultados. Conviene fijar desde el principio qué papel ocupa este capítulo en el conjunto de la memoria: el núcleo científico del TFG sigue siendo el pipeline experimental y el modelo de clasificación multiclase, descritos y evaluados en los capítulos 7 a 11 de esta memoria; este capítulo no evalúa de nuevo ese modelo ni modifica

ninguna de sus métricas, sino que documenta la aplicación cliente-servidor que lo pone a disposición de un usuario, integrada en una fase posterior del proyecto como aportación complementaria desde la Ingeniería del Software (véase también el apartado 2.1 de esta memoria, donde esta aportación se recoge como objetivo secundario incorporado durante la evolución del proyecto, y no como parte de la especificación formal inicial).

Todo el contenido de este capítulo está respaldado por evidencia verificable: el código fuente completo del cliente de escritorio (`main.py`, 1.637 líneas) y del backend de inferencia (`main-cloud.py`, 1.880 líneas), su fichero de configuración real (`config.json`), sus dependencias (`requirements.txt`), el Dockerfile histórico asociado a la línea de prototipado ScanIA, y un vídeo demostrativo de aproximadamente dos minutos del que se han extraído y verificado cuatro fotogramas contra el propio código. No se ha inventado ninguna funcionalidad no verificada en estas fuentes, no se han generado mockups ficticios, y no se ha construido ningún diagrama de clases artificial: donde el código no está organizado en torno a clases de dominio, este capítulo lo dice explícitamente y utiliza un diagrama de componentes en su lugar (apartado 5.3).

5.2 Evolución desde ScanIA al demostrador actual

Figura 5.1 — Línea temporal: de ScanIA al demostrador actual

El nombre "ScanIA" se conserva en el cliente actual; se documenta como evolución posterior de esa línea de prototipado, no como identidad de código verificada (apartado 5.2).



Figura 5.1 — Línea temporal: de ScanIA al demostrador actual

Como ya se apunta brevemente en el capítulo de planificación y gestión del proyecto, en una fase temprana y cronológicamente independiente de la reorientación hacia la clasificación multiclase, el proyecto incluyó una línea de trabajo sobre interfaz e integración conocida como ScanIA: una plantilla HTML

(`brain_viewer.html`) con su hoja de estilos, un `Dockerfile` orientado a un despliegue compatible con Cloud Run, una interfaz orientada a visualización volumétrica 3D mediante la librería `vtk.js` cargada por CDN (sin lógica JavaScript conectada verificable), y una lógica de inferencia 2D sobre volúmenes NIfTI en un notebook histórico, vinculada exclusivamente a la tarea binaria de esa época. Sobre esos artefactos concretos no está verificada la existencia de un backend completo y accesible, ni un despliegue real, ni una integración con el modelo multiclasa final; este capítulo es el que documenta con detalle, por primera vez en esta memoria, qué de ScanIA está verificado y qué no lo está.

El demostrador que se documenta en este capítulo es un artefacto distinto y posterior: un cliente de escritorio (`main.py`) y un backend de inferencia (`main-cloud.py`) cuyo código fuente se ha inspeccionado directamente, que sí están conectados entre sí y con el modelo multiclasa final, y cuya conexión con el servicio Cloud Run `run-model` (ya documentado por la incidencia de despliegue del Anexo J del documento de anexos técnicos) se ha confirmado mediante el propio fichero de configuración del cliente (`config.json`, campo `api_url`).

El nombre "ScanIA" se conserva en el cliente actual —aparece en el título de la ventana, en la pantalla de carga (`splash.py`, no utilizada realmente, y la función `show_splash` propia de `main.py`) y en el texto de aviso experimental —, lo que permite documentar el demostrador actual como una evolución posterior de aquella línea de prototipado, ampliada para integrar las funcionalidades verificadas en este capítulo. Esto no implica una identidad de código completa y verificada entre el prototipo histórico (la plantilla `brain_viewer.html` con `vtk.js`) y la aplicación actual: no se ha localizado evidencia directa de esa continuidad de código, y este capítulo no la afirma. Si en el futuro aparece esa evidencia, se documentará aparte.

5.3 Arquitectura cliente-servidor

Figura 5.2 — Diagrama de componentes/módulos

Sustituye a un diagrama de clases: `main.py` y `main-cloud.py` están organizados por funciones, no por clases de dominio (apartado 5.3).

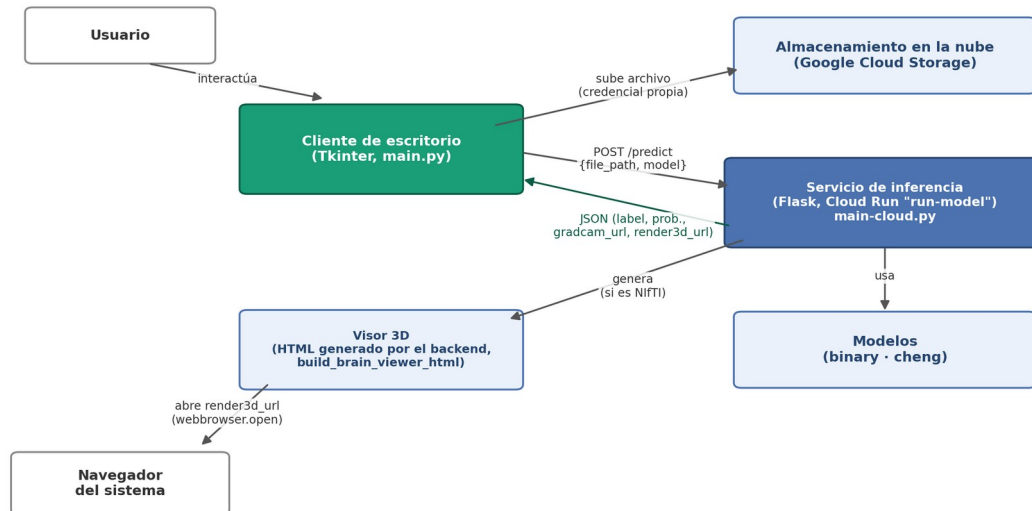


Figura 5.2 — Diagrama de componentes/módulos

La arquitectura verificada por código es la siguiente: el cliente de escritorio se autentica localmente contra `credentials.json` (sin intervención del backend), permite seleccionar un archivo de imagen o un volumen NIfTI y un modelo (binary o cheng), sube ese archivo directamente a Google Cloud Storage utilizando una credencial de servicio propia embebida en el cliente (buckets `mri-bucket-heatmap` y `mri-bucket-3d`), y a continuación envía una petición HTTP POST al endpoint `/predict` del servicio Cloud Run `run-model` (`main-cloud.py`) con el cuerpo `{file_path|image_path, model}`. El backend ejecuta el preprocesado y la inferencia con el modelo seleccionado, genera el mapa Grad-CAM cuando corresponde (apartado 5.9, RB-03) o, si la entrada es un volumen NIfTI, ejecuta el pipeline de cortes 2D y genera el HTML del visor volumétrico 3D (`build_brain_viewer_html`), y devuelve un JSON con la etiqueta, las probabilidades y, según el caso, `gradcam_url` o `render3d_url`. El cliente descarga y muestra el resultado en un modal, o abre el visor 3D en el navegador del sistema mediante `webbrowser.open()`.

Todas las llamadas de red se ejecutan en un hilo en segundo plano (`threading.Thread(daemon=True)`, 7 usos) con actualización segura de la interfaz mediante `root.after(0, ...)`, con timeouts configurables (600 s para imágenes, 900 s para volúmenes NIfTI) y captura de errores mediante `messagebox.showerror` sin interrumpir la aplicación.

5.4 Cliente de escritorio

El cliente (`main.py`, 1.637 líneas) está construido íntegramente sobre Tkinter, la biblioteca estándar de interfaz gráfica de Python, sin frameworks adicionales (no usa PyQt, `customtkinter` ni Electron). La estética oscura de la interfaz se consigue mediante tres widgets propios dibujados sobre Canvas — `RoundedButton`, `RoundedCard`, `RoundedEntry` (líneas 301-398)—, que son utilidades visuales genéricas, no clases de dominio (véase apartado 5.6 sobre por qué no se construye un diagrama de clases).

El fichero está organizado por funciones, no por clases: no existe una separación en módulos independientes ni un patrón MVC. Las pantallas identificadas son: login (`show_login`, L517), registro (`show_register`, L572), splash (`show_splash`, L651, tras un login correcto), ventana principal (`start_app`, L1419, con barra superior, aviso experimental, selector de modelo y tarjeta de subida de archivo con cinco estados), modal de resultado binario (`show_prediction_modal`, L867), modal de resultado Cheng/multiclase (`show_cheng_prediction_modal`, L952), modal de análisis NIfTI/3D (`show_nifti_modal`, L1054) y modal de ajustes (`show_settings`, L1188).

Un artefacto suelto confirmado durante la inspección: `splash.py` (61 líneas, una pantalla de carga alternativa) existe en el repositorio pero no está importado ni usado por `main.py`, que define su propia función de splash inline; se documenta aquí por transparencia, no como parte del flujo activo.

5.5 Backend de inferencia

El backend (`main-cloud.py`, 1.880 líneas), ya introducido en capítulos anteriores de esta memoria, es un servicio Flask desplegado en Cloud Run bajo el nombre `run-model`, con dos endpoints relevantes para el cliente: `/health` (comprobación de estado, consultada por el cliente cada 30 segundos) y `/predict` (inferencia). El campo `model` de la petición selecciona entre el modelo

binary (histórico, tarea tumor/no-tumor) y cheng (el modelo multiclase final de este TFG, ResNet50-FT). Para una entrada NIfTI, el pipeline volumétrico usa siempre el modelo binario para el escaneo corte a corte del volumen completo (RB-02) y, si el usuario tiene seleccionado cheng, lo aplica además —no en su lugar— para clasificar el tipo de tumor sobre los cortes ya detectados. El mapa Grad-CAM se genera siempre para el modelo cheng y únicamente para el modelo binary cuando la probabilidad estimada de tumor es igual o superior a 0,9 (RB-03).

Precisión terminológica obligatoria (aplicable a todo este capítulo): debe distinguirse en todo momento (1) el formato de entrada NIfTI (un volumen), (2) la inferencia, que es 2D, corte a corte, tanto para el modelo binario como para el modelo Cheng, y (3) la visualización volumétrica 3D en el navegador, que es una representación posterior generada por el backend a partir de los resultados 2D, no el modelo de clasificación. En ningún punto de esta memoria se describe el sistema como una "CNN 3D" ni como un "clasificador volumétrico 3D".

5.6 Funcionalidades

La Tabla 5.1 recoge, con su evidencia asociada, las funcionalidades confirmadas del demostrador tras la inspección directa del código y del vídeo.

Tabla 5.1 — Funcionalidades confirmadas del demostrador

Funcionalidad	Evidencia	Estado
Login con usuario/contraseña local	show_login, load_credentials (L260-267, 517-569)	Confirmada por código y vídeo
Registro de nuevo usuario	show_register (L572-628)	Confirmada por código
Selección de archivo (JPG/PNG/NIfTI)	select_image, filedialog.askopenfilename (L1336-1346)	Confirmada por código y vídeo
Selección de modelo binary/cheng, persistente	build_model_selector, config.json (L1263-1329)	Confirmada por código y vídeo
Subida directa a GCS desde el cliente (no vía backend)	upload_to_gcs (L140-150), credencial de servicio propia	Confirmada por código
Llamada HTTP a /predict con {file_path image_path, model}	predict_image, predict_file (L159-182)	Confirmada por código

Comprobación periódica del estado del servicio (/health, cada 30 s)	check_cloud_health, schedule_poll (L185-193, 1521-1528)	Confirmada por código y vídeo
Resultado binario + Grad-CAM condicional (score $\geq 0,9$)	backend + show_prediction_modal	Confirmada por código
Resultado multiclase + distribución de probabilidad + Grad-CAM (siempre)	show_cheng_prediction_modal (L952-1052)	Confirmada por código y vídeo
Procesamiento de volumen NIfTI y visor 3D en el navegador	predict_file, show_nifti_modal, webbrowser.open (L1054-1181)	Confirmada por código y vídeo
Ejecución en segundo plano sin bloquear la interfaz	threading.Thread(daemon=True), 7 usos	Confirmada por código
Configuración editable de URL, timeouts y umbral	show_settings (L1188-1256), persistida en config.json	Confirmada por código y vídeo
Aviso de uso experimental / no diagnóstico médico	DISCLAIMER_TEXT (L39-43), visible en pantalla principal y modales	Confirmada por código y vídeo
Cierre de sesión con confirmación	logout, on_close (L1449-1515)	Confirmada por código
Persistencia de credenciales sin hash (limitación)	save_credentials (L270-272, 542)	Confirmada por código — limitación de seguridad, apartado 5.14
Carpeta local gradcam_images para heatmaps	declarada, nunca escrita	No confirmada (código muerto)
Historial de análisis previos	sin función ni pantalla asociada	No confirmada

5.7 Requisitos funcionales

Tabla 5.2 — Requisitos funcionales (RF)

ID	Descripción	Evidencia	Estado
RF-01	El sistema debe permitir iniciar sesión con usuario y contraseña locales	show_login, L517-569	CUMPLIDO
RF-02	El sistema debe permitir crear una cuenta nueva	show_register, L572-628	CUMPLIDO
RF-03	El sistema debe permitir seleccionar un archivo de imagen (JPG/PNG) o de volumen (NIfTI) desde el sistema de ficheros	select_image, L1336-1346	CUMPLIDO
RF-04	El sistema debe permitir elegir entre el modelo binario y el modelo Cheng	build_model_selector, L1263-1329	CUMPLIDO

	multiclase antes del análisis		
RF-05	El sistema debe subir el archivo seleccionado a almacenamiento en la nube antes del análisis	upload_to_gcs, L140-150	CUMPLIDO
RF-06	El sistema debe enviar la petición de análisis al servicio de inferencia en la nube y mostrar el resultado	predict_image/predict_file, L159-182; modales de resultado	CUMPLIDO
RF-07	El sistema debe mostrar la probabilidad estimada y, para el modelo Cheng, la distribución de probabilidad por clase	show_prediction_modal, show_cheng_prediction_modal	CUMPLIDO
RF-08	El sistema debe mostrar el mapa de calor Grad-CAM cuando el backend lo proporcione	ambos modales de imagen, L920-1044	CUMPLIDO
RF-09	El sistema debe permitir seleccionar estudios NIfTI, procesar sus cortes mediante el pipeline implementado y abrir una representación volumétrica 3D del resultado	show_nifti_modal, open_3d, L1054-1181	CUMPLIDO
RF-10	El sistema debe comprobar y mostrar el estado de conexión del servicio en la nube de forma periódica	refresh_cloud_health, schedule_poll, L245-257, 1521-1528	CUMPLIDO
RF-11	El sistema debe permitir configurar la URL del servicio, los tiempos de espera y el umbral visual	show_settings, L1188-1256	CUMPLIDO
RF-12	El sistema debe mostrar un aviso de que el resultado no constituye un diagnóstico médico	DISCLAIMER_TEXT, L39-43	CUMPLIDO

(RF-09 se ha reformulado respecto al inventario original para no sugerir una inferencia volumétrica 3D: el pipeline procesa cortes 2D y el resultado se presenta después sobre una representación 3D, nunca al revés — apartado 5.5.)

5.8 Requisitos no funcionales y grado de cumplimiento

Los requisitos no funcionales no se confunden aquí con el estado actual del software: un RNF puede estar planteado como deseable y, tras auditar el código, confirmarse no satisfecho en el prototipo actual —eso no invalida el requisito, documenta una deuda técnica real.

Tabla 5.3 — Requisitos no funcionales (RNF) y grado de cumplimiento

ID	Descripción	Evidencia	Estado
RNF-01	La interfaz no debe bloquearse durante el análisis (operaciones de red en segundo plano)	threading.Thread(dae mon=True), 7 usos	CUMPLIDO
RNF-02	El tiempo de espera del análisis debe ser configurable, con valores por defecto amplios (600 s / 900 s)	config.json, show_settings	CUMPLIDO
RNF-03	Los fallos de red o del servicio deben notificarse al usuario sin cerrar la aplicación	except Exception + messagebox.showerr or en cada llamada	CUMPLIDO
RNF-04	La configuración de conexión debe persistir entre sesiones	save_config/ load_config, L67-84	CUMPLIDO
RNF-05	Las credenciales de usuario deberían almacenarse mediante un mecanismo seguro que evite conservar contraseñas en texto plano	save_credentials: creds[username] = password, comparación == directa (L270-272, 542)	NO SATISFECHO EN EL PROTOTIPO ACTUAL — limitación de seguridad real, documentada en el apartado 5.14, no una característica
RNF-06	La aplicación debería poder ejecutarse en distintos equipos sin depender de rutas locales específicas ni de credenciales embebidas	ruta local fija de la credencial de servicio de GCP, L24-26	PARCIALMENTE SATISFECHO — la URL del servicio y otros parámetros sí son configurables (show_settings), pero la credencial de GCP embebida en el cliente es específica del equipo del autor y no portable tal cual

5.9 Reglas de negocio

Tabla 5.4 — Reglas de negocio (RB)

ID	Descripción	Fuente
RB-01	Un usuario no puede registrarse con un nombre de usuario ya existente	create_action, L602-604
RB-02	El pipeline NIfTI usa siempre el modelo binario para el escaneo volumétrico completo; si el usuario tiene seleccionado "cheng", se usa además —no en su lugar— para clasificar el tipo de tumor sobre los cortes ya detectados	predict_file, L159-166, y main-cloud.py
RB-	El mapa de calor Grad-CAM solo se genera para el modelo binario	_predict_image_binary/

03	cuando la probabilidad de tumor es $\geq 0,9$; para el modelo Cheng se genera siempre	<code>_predict_image_cheng</code> en <code>main-cloud.py</code>
RB-	Si se pierde la conexión con el servicio en la nube, el modelo	<code>model_health</code> ,
04	seleccionado se marca como "Sin conexión" pero no se oculta; el análisis fallará si se intenta	<code>MODEL_STATUS_STY</code> <code>LE</code>

5.10 Casos de uso

Figura 5.3 — Diagrama de casos de uso
Actor principal: Usuario. Actor secundario/sistema externo: servicio de inferencia sobre Cloud Run (apartado 5.10, tabla de CU-01 a CU-07).

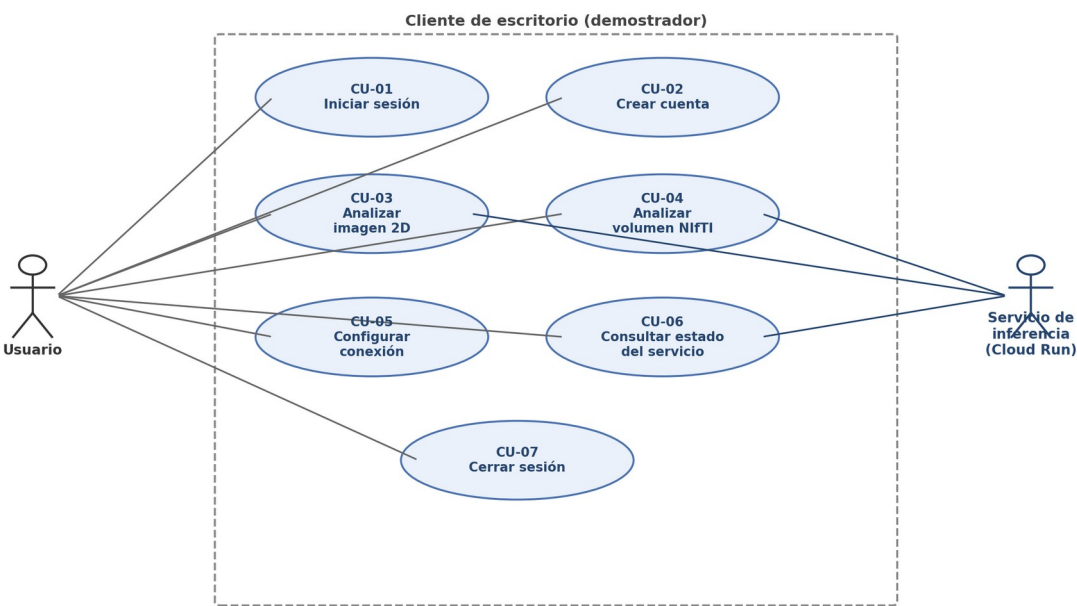


Figura 5.3 — Diagrama de casos de uso

Actores: Usuario de la aplicación (principal); servicio de inferencia sobre Cloud Run (secundario/sistema externo).

Tabla 5.5 — Casos de uso: identificación y contexto

ID	Nombre	Actor(es)	Precondiciones
CU-01	Iniciar sesión	Usuario	Tener cuenta creada
CU-02	Crear cuenta	Usuario	Ninguna
CU-03	Analizar imagen 2D	Usuario, Servicio de inferencia	Sesión iniciada, servicio disponible

ID	Nombre	Actor(es)	Precondiciones
CU-04	Analizar volumen NIfTI	Usuario, Servicio de inferencia	Sesión iniciada, servicio disponible
CU-05	Configurar conexión	Usuario	Sesión iniciada
CU-06	Consultar estado del servicio	Usuario, Servicio de inferencia	Sesión iniciada
CU-07	Cerrar sesión	Usuario	Sesión iniciada

La Tabla 5.6 continúa la misma numeración de casos de uso (CU-01 a CU-07) con el flujo principal, los flujos alternativos y la postcondición de cada uno.

Tabla 5.6 — Casos de uso: comportamiento

ID	Flujo principal	Flujos alternativos	Postcondiciones
CU-01	Introduce usuario/contraseña → validación local → acceso a la app	Credenciales incorrectas → mensaje de error; opción de registrarse	Sesión iniciada
CU-02	Introduce usuario/contraseña/confirmación → se guarda en credentials.json	Usuario ya existe / contraseñas no coinciden → error	Cuenta creada, vuelve a login
CU-03	Selecciona modelo → selecciona archivo → sube a GCS → backend predice → se muestra resultado	Error de red/servicio → mensaje de error	Resultado (label, probabilidad, Grad-CAM) mostrado
CU-04	Selecciona archivo .nii/.nii.gz → sube a GCS → backend procesa el volumen corte	Sin URL de render → aviso de no disponible	Resultado NIfTI mostrado; visor 3D abierto si se solicita

ID	Flujo principal	Flujos alternativos	Postcondiciones
	a corte → se muestra resumen + enlace al visor 3D		
CU-05	Abre Ajustes → edita URL/timeouts/umbral guarda	Valores numéricos → se ignoran	no Configuración persistida
CU-06	Automático cada 30 s; también manual al guardar Ajustes	Servicio caído → chip "Sin conexión"	Estado visible en la barra superior
CU-07	Pulsa "Salir" → confirmación → vuelve a login	Cancela → permanece en la app	Sesión cerrada

5.11 Historias de usuario

Las historias de usuario que siguen se han reconstruido a posteriori a partir de las funcionalidades finalmente implementadas. No formaron parte de una especificación formal previa al desarrollo.

HU-01 — Como usuario, quiero iniciar sesión con mis credenciales, para acceder de forma controlada a la herramienta de análisis. Criterios de aceptación: rechaza usuario/contraseña incorrectos; permite crear una cuenta nueva desde la misma pantalla.

HU-02 — Como usuario, quiero elegir entre el modelo binario y el modelo multiclase antes de subir una imagen, para adaptar el análisis al tipo de estudio que tengo. Criterio: la elección se recuerda entre sesiones.

HU-03 — Como usuario, quiero subir una imagen JPG/PNG y recibir una predicción con su probabilidad y, cuando proceda, un mapa Grad-CAM, para interpretar visualmente en qué se ha fijado el modelo. Criterio: el resultado se muestra dentro del tiempo de espera configurado o se informa claramente del error.

HU-04 — Como usuario, quiero subir un volumen NIfTI completo y explorarlo en 3D, para revisar el estudio volumétrico completo y no solo un corte. Criterio:

se muestra el corte de mayor puntuación, estadísticas del volumen y un enlace a un visor 3D interactivo.

HU-05 — Como usuario, quiero poder cambiar la URL del servicio y los tiempos de espera, para apuntar la aplicación a un despliegue distinto sin modificar el código. Criterio: los cambios persisten tras cerrar la aplicación.

HU-06 — Como usuario, quiero ver en todo momento si el servicio en la nube está disponible, para saber si puedo lanzar un análisis antes de intentarlo.

5.12 Diagramas de secuencia

Los tres diagramas siguientes describen únicamente llamadas verificadas en el código; no se ha inventado ninguna secuencia ni parámetro.

Figura 5.4 — Diagrama de secuencia: análisis binario 2D
Grad-CAM condicional (RB-03): solo si la probabilidad de tumor es ≥ 0.9 (apartado 5.12).

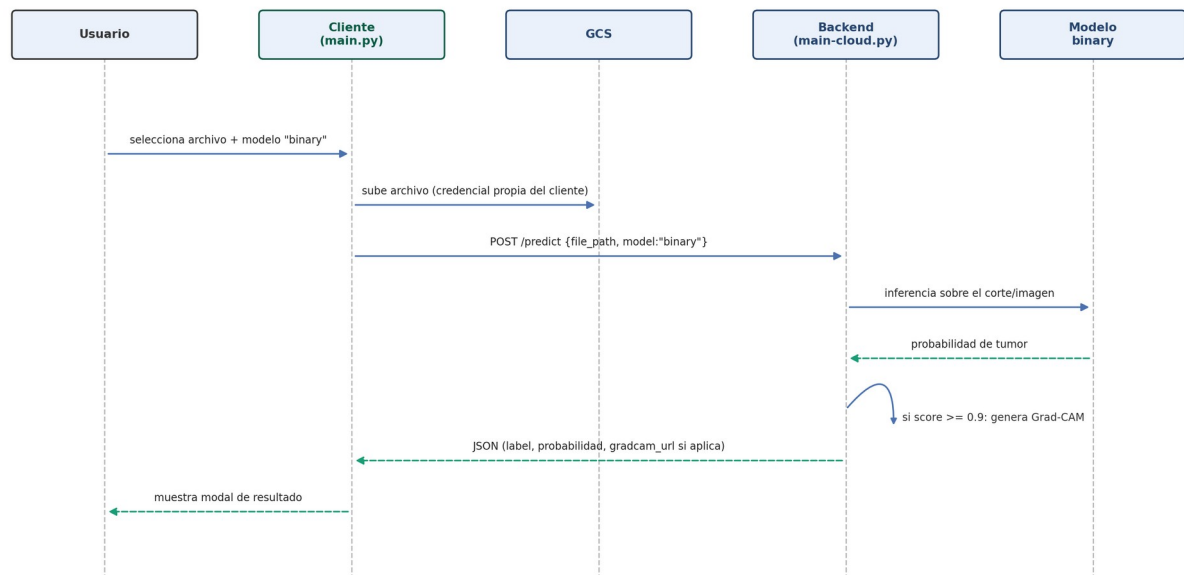


Figura 5.4 — Diagrama de secuencia: análisis binario 2D

Figura 5.5 — Diagrama de secuencia: análisis Cheng multiclase 2D

A diferencia del modelo binario, el Grad-CAM se genera siempre (RB-03) y se añade la distribución de probabilidad por clase (apartado 5.12).

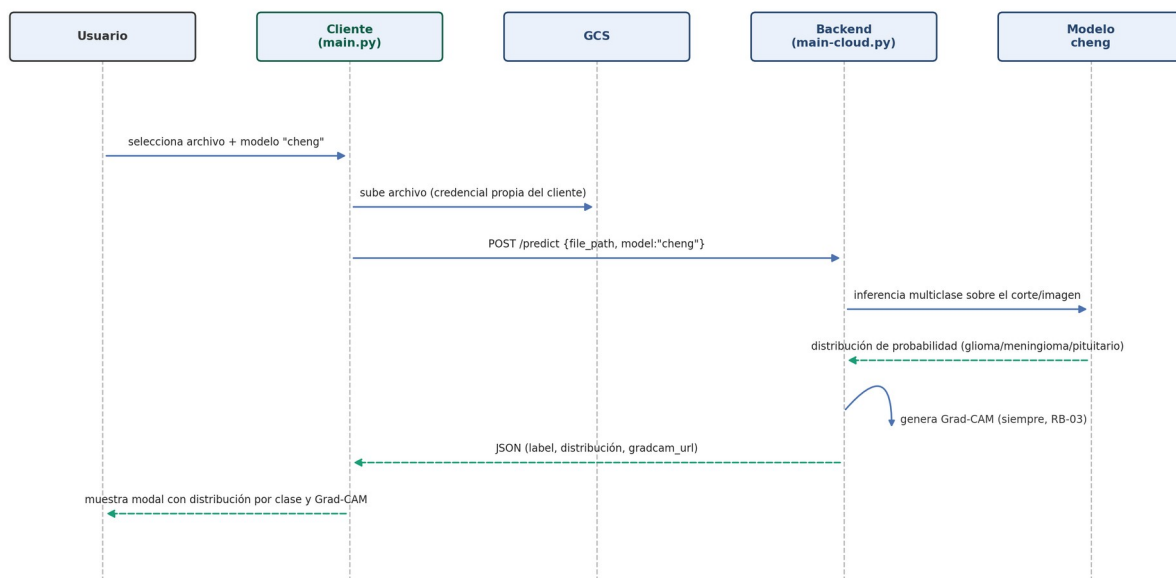


Figura 5.5 — Diagrama de secuencia: análisis Cheng multiclase 2D

Figura 5.6 — Diagrama de secuencia: análisis de volumen NIfTI y visor 3D

La inferencia es siempre 2D, corte a corte; el volumen 3D es una representación posterior generada por el backend, no el modelo de clasificación (apartado 5.12, precisión terminológica NIfTI/2D/3D).

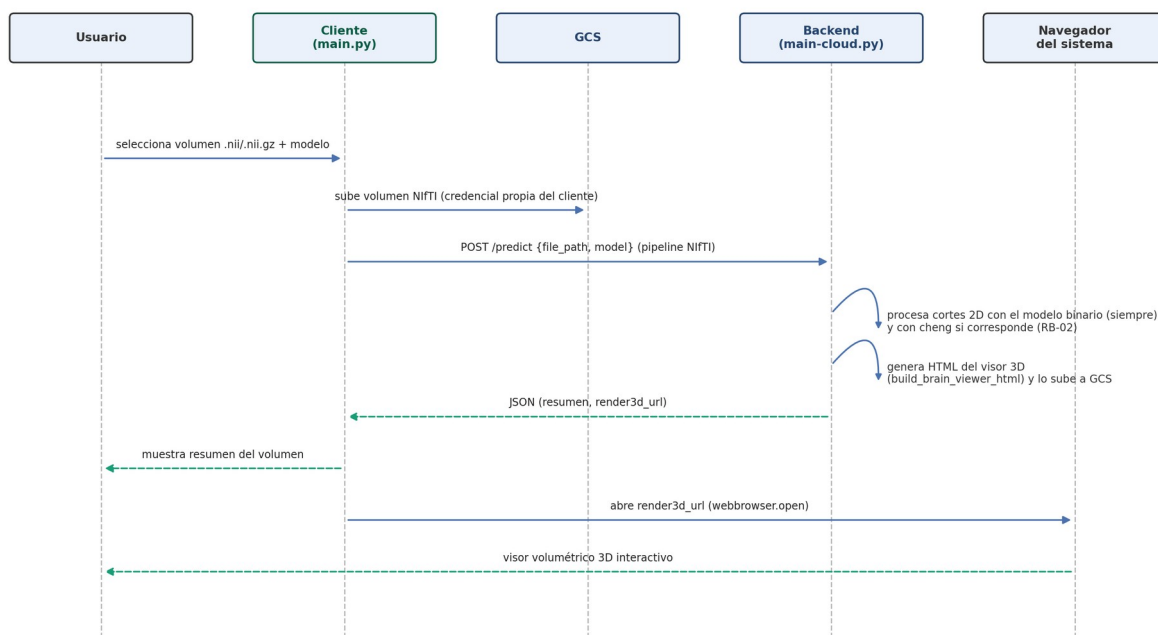


Figura 5.6 — Diagrama de secuencia: análisis de volumen NIfTI y visor 3D

En los tres casos se mantiene la distinción del apartado 5.5: la inferencia es siempre 2D, corte a corte; en el caso NIfTI (Figura 5.6), el volumen 3D que

finalmente ve el usuario es una representación generada por el backend a partir de esos resultados 2D, no una salida directa del modelo.

5.13 Interfaz y capturas

Las cuatro capturas siguientes son fotogramas reales extraídos del vídeo demostrativo y cotejados directamente contra el código fuente (no son mockups ni recreaciones). No se incluye la pantalla de login/registro por no aportar valor adicional a la explicación del funcionamiento y para no exponer, ni siquiera vacíos, campos de credenciales.

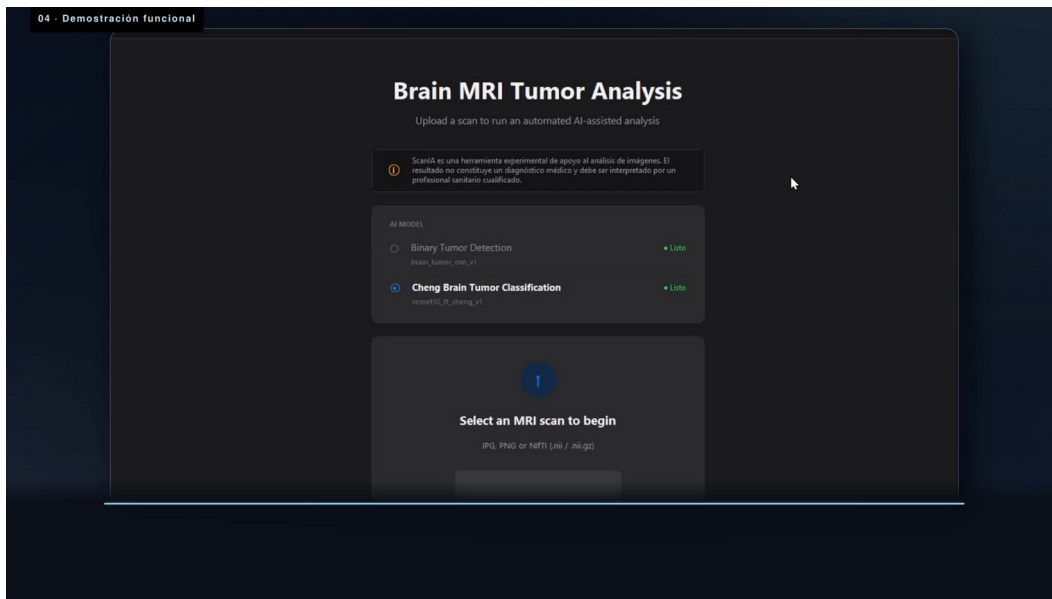


Figura 5.7 — Pantalla principal, con el aviso de uso experimental y el selector de modelo

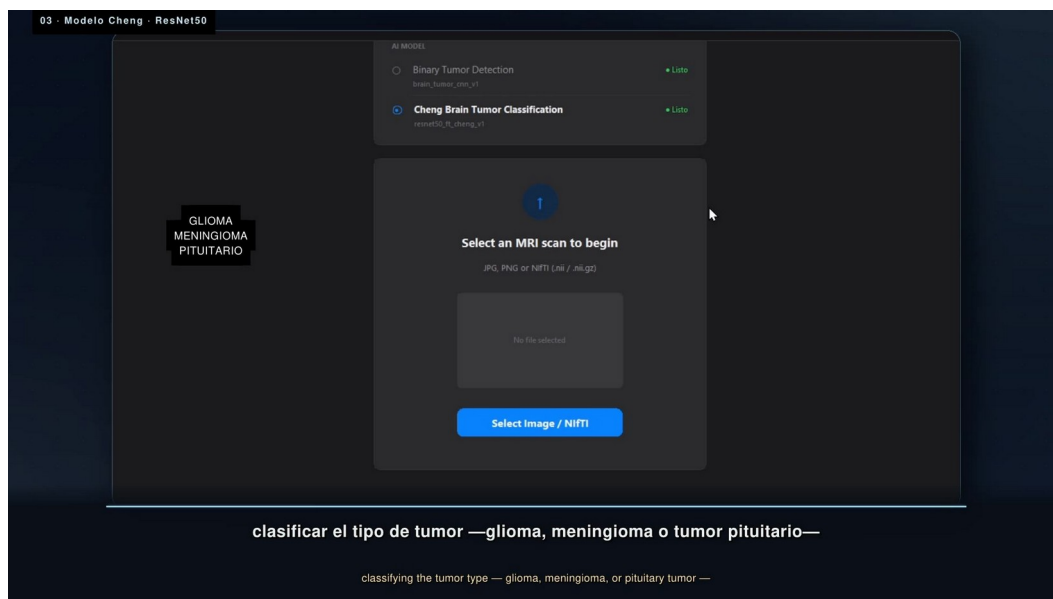


Figura 5.8 — Selector de modelo ("Cheng Brain Tumor Classification" activo) y botón de selección de archivo



Figura 5.9 — Modal de resultado del modelo Cheng: clase predicha, probabilidad, distribución por clase y mapa Grad-CAM



Figura 5.10 — Visor volumétrico 3D en el navegador, con el resultado Cheng superpuesto (meningioma, 58,7 %)

5.14 Limitaciones del demostrador

Se documentan aquí, sin disfrazarlas, las limitaciones reales confirmadas por código:

Credenciales locales sin hash ni cifrado (RNF-05, no satisfecho): credentials.json almacena usuario/contraseña en texto plano y la validación se realiza mediante comparación directa de cadenas (==). No hay evidencia de que estas credenciales se transmitan por red —el problema es de almacenamiento y validación local, no de transmisión.

Credencial de cuenta de servicio de Google Cloud embebida en el cliente (RNF-06, parcialmente satisfecho): el cliente incluye una credencial de servicio propia para subir archivos directamente a Google Cloud Storage, con una ruta de configuración específica del equipo de desarrollo del autor. Esto no es portable sin modificación y constituye una superficie de exposición de credenciales que debería resolverse intermediando la subida a través del backend.

Dependencia de configuración de un equipo concreto: más allá de la credencial de GCP, algunos valores de arranque están ligados al entorno de desarrollo original.

Ausencia de arquitectura modular: `main.py` concentra presentación, comunicación de red y configuración en un único fichero de 1.637 líneas, sin separación en módulos independientes.

Ausencia de pruebas automatizadas sobre el cliente de escritorio.

Ausencia de trazabilidad persistente de cada inferencia: no existe un registro (log) de qué imagen, modelo y resultado se generó y cuándo; queda como línea de trabajo futura (capítulo de conclusiones).

Estas limitaciones no cuestionan la validez del núcleo científico del TFG, que no depende de este demostrador; documentan la deuda técnica de un artefacto de Ingeniería del Software desarrollado y actualizado en una fase posterior del proyecto, con fines de demostración y no como servicio consolidado para uso continuado.

5.15 Relación con infraestructura y despliegue

El servicio Cloud Run que consume este demostrador (`run-model`) es el mismo cuya incidencia de disponibilidad se diagnosticó y corrigió en una sesión de trabajo posterior al cierre de la redacción de los capítulos experimentales, documentada en el Anexo J del documento de anexos técnicos: el servicio devolvía errores 503 por agotamiento de memoria al cargar varias copias del modelo en distintos procesos trabajadores, y se corrigió ampliando la memoria del contenedor y fijando la concurrencia a una petición por instancia. Este capítulo no repite ese detalle técnico de despliegue —se remite al Anexo J del documento de anexos técnicos—, y se limita a señalar que la inspección directa del código del cliente y del backend confirma, de forma independiente y más sólida que la sola observación operativa del servicio, que ambos artefactos están conectados entre sí y con este mismo servicio: `config.json` apunta exactamente a la URL de `run-model` corregida en esa incidencia.

Como ya se precisa en el capítulo de viabilidad y recursos de esta memoria, no existe relación confirmada entre este servicio operativo actual y los artefactos de despliegue históricos de la línea de prototipado ScanIA (el Dockerfile descrito en el apartado 5.2): son observaciones independientes, y este capítulo no las combina para sugerir una continuidad de código que no se ha verificado.

5.16 Vídeo demostrativo

El material audiovisual de difusión del proyecto (aproximadamente dos minutos, con subtítulos en español e inglés, disponible en <https://youtu.be/QgTOD3KOTpl>) muestra el funcionamiento del demostrador descrito en este capítulo, incluyendo la clasificación binaria histórica y la clasificación multiclase final, con un aviso explícito de que la herramienta es de carácter experimental y no constituye un dispositivo de diagnóstico médico. Este vídeo se referencia aquí como material demostrativo del software implementado, nunca como evidencia de rendimiento científico: los resultados cuantitativos de este TFG siguen respaldados exclusivamente por los artefactos, JSON, logs, holdout y 5CV descritos en los capítulos de resultados y discusión.

5.17 Síntesis

Este capítulo ha documentado, con la misma exigencia de trazabilidad aplicada al resto de esta memoria, un demostrador software real y verificado por código: un cliente de escritorio en Tkinter y un backend Flask desplegado en Cloud Run, conectados entre sí y con el modelo multiclase final de este TFG, descritos mediante un diagrama de componentes (no de clases, por no existir un modelo de dominio orientado a objetos suficientemente representativo), doce requisitos funcionales y seis no funcionales con su grado real de cumplimiento, cuatro reglas de negocio, siete casos de uso, seis historias de usuario documentadas a posteriori, tres diagramas de secuencia y cuatro capturas reales.

Sus limitaciones —credenciales sin cifrar, credencial de GCP embebida, ausencia de modularidad y de pruebas automatizadas— se documentan con la misma franqueza que el resto de limitaciones de este trabajo, y no comprometen en ningún caso los resultados científicos del núcleo experimental multiclase, que siguen siendo el resultado central de este TFG.

6. Datos y reconstrucción metodológica

6.1 Fuentes de datos iniciales

El proyecto no comenzó directamente con el conjunto de datos que finalmente se utiliza como núcleo experimental. Las primeras versiones del clasificador se

construyeron sobre datasets agregados de origen Kaggle, organizados como colecciones de imágenes de resonancia magnética clasificadas por tipo de tumor. Estos conjuntos fueron útiles durante el prototipado inicial: permitieron construir las primeras versiones del clasificador, probar la infraestructura de entrenamiento y familiarizarse con el problema antes de invertir esfuerzo en una reconstrucción más cuidadosa de los datos.

Sin embargo, esta primera organización de los datos presentaba una limitación que solo se hizo evidente al revisarla con más detenimiento: no ofrecía una forma fiable de saber a qué paciente pertenecía cada imagen. El origen exacto y el volumen de estas fuentes agregadas no quedaron documentados con la precisión suficiente como para citarlos aquí con cifras concretas, y por ese motivo no se desarrollan en detalle en esta sección. Lo relevante no es el tamaño de aquel conjunto inicial, sino la forma en que estaba organizado, que se explica en el apartado siguiente.

La limitación detectada no es, en realidad, un problema de la plataforma de origen ni de los datos en sí, sino de cómo se habían organizado y etiquetado las fuentes concretas empleadas en esta primera fase. Kaggle actúa aquí simplemente como el canal de distribución a través del cual se obtuvieron los datos agregados; el problema real, que se describe en la sección 6.2, es la ausencia de un identificador de paciente fiable en la organización que se empleó.

6.2 Problema de trazabilidad en la primera versión

La primera versión del pipeline organizaba las imágenes utilizando un campo denominado `group_id` como si fuera equivalente a un identificador de paciente. Este campo agrupaba imágenes que, en apariencia, pertenecían a un mismo conjunto, pero no existía evidencia suficiente para establecer una correspondencia uno a uno entre `group_id` y paciente real.

Al revisar las fuentes derivadas empleadas en esa primera etapa, ninguna de ellas publicaba un identificador de paciente utilizable de forma directa. El `group_id` era, en el mejor de los casos, un proxy sin verificación, no un identificador real. Esta situación tiene una consecuencia metodológica directa: si las imágenes se reparten entre los conjuntos de entrenamiento, validación y test sin una identificación real de paciente, la organización inicial no permite descartar de forma suficientemente fiable que existan dependencias entre imágenes pertenecientes a un mismo sujeto repartidas en más de un conjunto.

Es importante ser precisos en este punto. No se afirma que se detectara fuga de información demostrada en esa primera versión del pipeline; lo que se detectó es un riesgo estructural derivado de la forma en que estaban organizados los datos, no

una fuga confirmada mediante un análisis posterior sobre esos conjuntos concretos. Esta distinción entre riesgo y fuga demostrada es la que motivó la decisión de reconstruir el dataset desde fuentes que sí ofrecieran una identificación de paciente verificable, en lugar de intentar corregir a posteriori la organización existente.

6.3 Reconstrucción con identificación real de paciente

Ante esta limitación, se optó por abandonar las fuentes agregadas de la primera versión y reconstruir el conjunto de datos a partir de dos fuentes que sí permiten identificar al paciente de origen de cada imagen, aunque de formas distintas entre sí.

Para el dataset Cheng, empleado como fuente de la clase tumor en la tarea binaria y, más adelante, como base completa de la clasificación multiclase, el identificador de paciente se deriva directamente del campo PID presente en la estructura `cjdata` de los ficheros originales. Se trata de un identificador incluido por los propios autores del dataset como parte de sus metadatos. ([Cheng, 2024](#))

Para el dataset IXI, empleado como fuente de la clase no tumor en la tarea binaria, no existe un campo equivalente dentro de un fichero de metadatos estructurado accesible en este proyecto; el identificador de sujeto se deriva en su lugar del propio nombre de fichero de cada imagen, que codifica el identificador del sujeto correspondiente. ([IXI Dataset, s. f.](#))

Esta diferencia no es un simple detalle técnico: Cheng e IXI no son metodológicamente equivalentes en cuanto a la forma de identificación del sujeto, aunque en ambos casos el identificador resultante permite, dentro de las fuentes empleadas, agrupar correctamente las imágenes por paciente. Esta distinción vuelve a ser relevante en la sección 6.6, al analizar las diferencias de procedencia entre ambas fuentes.

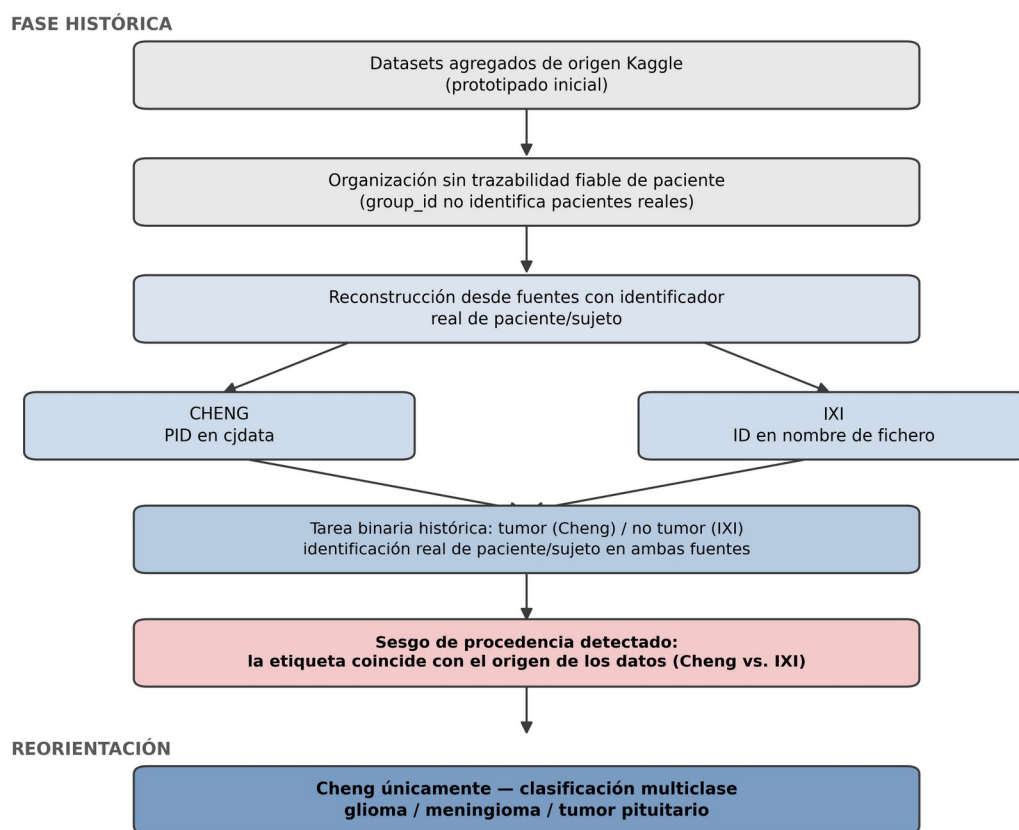
Con esta reconstrucción, el dataset binario quedó formado por 467 pacientes, de los cuales 233 proceden de Cheng y 234 de IXI, sumando un total de 5.872 imágenes. Esta cifra de pacientes reales, verificable a partir del identificador de cada fuente, es la mejora principal respecto a la organización anterior: no se trata simplemente de haber cambiado de fuente de datos, sino de haber sustituido un proxy sin garantías por un identificador de paciente verificable en cada una de las dos fuentes utilizadas.

La tabla 6.1 resume esta evolución, desde la organización inicial hasta el dataset multiclase que se describe en la sección 6.8.

Tabla 6.1 — Evolución de los datos utilizados en el proyecto

Etapa	Fuente	Tarea	Pacientes	Imágenes/ cortes	Identificación de paciente	Papel en el TFG
Prototipo inicial	Colecciones agregadas de origen Kaggle	Clasificación binaria (prototipo)	Precisión no consta	Precisión no consta	group_id (proxy sin verificación)	Prototipado inicial; descartado como base experimental
Reconstrucción patient-level (binaria)	Cheng (tumor) + IXI (no tumor)	Clasificación binaria (tumor / no tumor)	467 (233 Cheng + 234 IXI)	5.872 imágenes	Cheng: PID de cjdata; IXI: identificador de sujeto en el nombre de fichero	Tarea histórica; permitió detectar el sesgo de procedencia descrito en 4.6
Dataset Cheng multiclase	Cheng (3 clases)	Clasificación multiclase (glioma, meningioma, tumor pituitario)	233	3.064 cortes	PID de cjdata	Núcleo experimental del TFG a partir de la reorientación descrita en 4.7

Figura 6.1 — Evolución y reconstrucción de las fuentes de datos



El problema histórico fue la organización de los datos y la falta de trazabilidad fiable de paciente, no el origen Kaggle en sí mismo.

Figura 6.1 — Reconstrucción de los datos con identificación real de paciente

Esquema del proceso descrito en esta sección, desde las colecciones agregadas de origen Kaggle sin identificador de paciente verificable, pasando por la detección del problema de trazabilidad, hasta la reconstrucción sobre Cheng e IXI con identificador de paciente real (PID de cjdata e identificador de sujeto en el nombre de fichero, respectivamente).

6.4 Manifiestos y control de duplicados

Toda la información de trazabilidad de las imágenes reconstruidas se centraliza en un fichero de manifiesto, manifest_final.csv, en el que cada fila corresponde a una imagen e incluye, entre otros, los campos patient_id, source, sha256 y local_file. El campo sha256 corresponde a una huella de integridad calculada sobre el contenido de cada imagen, mientras que local_file registra la ruta en la que se almacena físicamente. A este manifiesto se asocia además un manifest_hash, descrito con prudencia como una huella asociada al manifiesto que el pipeline utiliza para trazabilidad e integridad, sin que el algoritmo exacto y la serialización empleados para

calcularla estén documentados con el detalle suficiente como para describirlos aquí más allá de esa caracterización general.

Sobre este manifiesto se aplicó un procedimiento de control de duplicados en tres pasos. El primero comprobó la unicidad del campo sha256 para descartar duplicados exactos a nivel de fichero; no se eliminó ningún duplicado exacto en esta comprobación. El segundo aplicó un procedimiento de hashing perceptual (pHash), con un umbral de distancia de Hamming ≤ 12 sobre una representación de 256 bits, para generar candidatos a casi-duplicado, es decir, imágenes distintas mediante sha256 pero visualmente muy similares entre sí. Por último, cada candidato generado por el pHash se sometió a una confirmación mediante el índice de similitud estructural (SSIM), exigiendo un valor $\geq 0,92$ para considerarlo confirmado; en ningún caso se realizó un borrado destructivo de los candidatos no confirmados.

El resultado de este procedimiento es que no se confirmó ningún caso de casi-duplicado mediante el procedimiento pHash+SSIM aplicado sobre el conjunto completo de 467 pacientes y 5.872 imágenes. Esta formulación es deliberada y distinta de decir que el pHash no produjo ningún candidato: el pHash sí genera candidatos como paso intermedio del procedimiento, pero el número bruto de candidatos producidos antes de aplicar el filtro SSIM no quedó exportado a ningún artefacto persistente, solo impreso durante la ejecución del notebook, por lo que esa cifra no se reconstruye de memoria y no se incluye en esta memoria. Lo que sí está verificado es el resultado final del procedimiento completo: cero candidatos confirmados, ni entre fuentes, ni entre clases, ni entre pacientes. No se afirma que el dataset no contenga casi-duplicados en un sentido absoluto, sino que el procedimiento descrito, aplicado sobre este universo concreto, no confirmó ninguno.

Un ejemplo concreto de esta trazabilidad de artefactos es la comparación realizada entre el manifiesto generado durante la primera ejecución completa del pipeline y el manifiesto reutilizado en la fase posterior de selección de arquitectura. Ambos ficheros contienen 5.872 filas y 20 columnas, de las cuales 19 son idénticas fila a fila entre ambas versiones; la única columna que difiere es `local_file`. Sus huellas SHA-256 de fichero completo, sin embargo, no son iguales entre sí, precisamente porque esa reorganización de rutas basta para cambiar el hash del fichero completo aunque el contenido tabular relevante no cambie. Por eso conviene formular esta comparación con precisión: ambos ficheros representan la misma colección de 5.872 registros respecto a las variables relevantes de los datos, aunque la reorganización de las rutas de `local_file` hace que los ficheros completos y sus huellas no sean idénticos.

Conviene no confundir, además, dos huellas que comparten el mismo nombre de campo pero corresponden a momentos distintos del proyecto. El `manifest_hash`

asociado al manifiesto binario completo de la Etapa A es un valor distinto del `manifest_hash` que aparece posteriormente reutilizado en los artefactos de resultados de la Etapa B; este segundo valor corresponde al mismo manifiesto completo de 467 pacientes, no a un subconjunto reducido a Cheng, aunque en ese momento del proyecto solo se estuviera trabajando ya con los pacientes de Cheng. No se trata de un mismo fichero citado dos veces, sino de dos huellas distintas que documentan la reutilización de un artefacto equivalente en su contenido relevante; mantener esta distinción evita atribuir por error un cambio de dataset a lo que en realidad es esa reutilización.

6.5 Controles de fuga de información implementados

Sobre el dataset reconstruido se establecieron varios controles de fuga de información implementados, orientados a reducir el riesgo descrito en la sección 6.2 en la nueva organización de los datos.

Un primer control comprueba la disjunción del campo `patient_id` entre los conjuntos de entrenamiento, validación y test: ningún paciente puede aparecer en más de uno de estos conjuntos a la vez. En la partición correspondiente a la tarea binaria, esta comprobación no encontró ningún solapamiento entre conjuntos. Un segundo control verifica la unicidad del campo `sha256` sobre el conjunto completo de imágenes, confirmando que las 5.872 imágenes del dataset binario tienen huellas de integridad distintas entre sí. Además, el procedimiento de `pHash` y `SSIM` descrito en la sección anterior no confirmó ningún candidato a casi-duplicado que pudiera introducir dependencias adicionales entre conjuntos. Por último, en toda validación cruzada realizada sobre estos datos —tanto en la fase de selección de arquitectura como en la validación de robustez que se describe en el capítulo 9— la partición se agrupa siempre por `patient_id`, de modo que ningún paciente puede quedar repartido entre distintos folds de una misma validación cruzada.

Estos controles de fuga de información implementados abordan de forma explícita el riesgo descrito en la sección 6.2, pero conviene formular la conclusión con la misma prudencia con la que se formuló el problema: no se ha cuantificado en qué medida disminuye ese riesgo, y los resultados anteriores describen los controles efectivamente implementados y su resultado sobre los conjuntos concretos verificados, no una demostración de ausencia total de fuga de información en un sentido absoluto. Dicho de otro modo, lo que se reporta es el resultado de aplicar estos controles, no una garantía universal más allá de lo que ellos mismos comprueban.

6.6 La tarea binaria tumor/no tumor y el sesgo de procedencia

La primera tarea completa que se evaluó sobre el dataset reconstruido fue una clasificación binaria: distinguir imágenes con tumor de imágenes sin tumor. Para esta tarea, la clase tumor se construyó íntegramente a partir del dataset Cheng, mientras que la clase no tumor se construyó íntegramente a partir del dataset IXI.

Esta forma de construir la tarea introduce una diferencia de procedencia que va más allá de la simple etiqueta de clase. Las imágenes de Cheng son resonancias magnéticas ponderadas en T1 con contraste, adquiridas en dos hospitales chinos. Las imágenes de IXI son resonancias ponderadas en T1 sin contraste, adquiridas en centros de Londres. Es decir, la clase a predecir coincide exactamente con el dataset de procedencia: toda imagen de la clase tumor procede de Cheng y toda imagen de la clase no tumor procede de IXI. Cualquier diferencia sistemática entre ambas fuentes —el uso de contraste, el equipo de adquisición, el protocolo de imagen, el propio origen geográfico de los datos— queda, por construcción, perfectamente asociada a la etiqueta que se pretende predecir.

Sobre esta tarea, el modelo evaluado obtuvo una balanced accuracy a nivel de paciente del 98,57 % sobre un conjunto de holdout. Aislada de su contexto, esta cifra podría leerse como una capacidad muy alta de detección de tumor. Sin embargo, no debe presentarse como evidencia central de esa capacidad, porque el propio diseño de la tarea no permite descartar que buena parte de ese rendimiento proceda de la diferencia de procedencia entre Cheng e IXI, y no de la presencia o ausencia real de tumor en la imagen.

Para investigar esta posibilidad, se construyó un clasificador de control basado únicamente en diez descriptores de bajo nivel de la imagen —de tipo estadístico y de textura, sin utilizar en ningún momento las representaciones aprendidas por una red convolucional entrenada—. Este clasificador de control, aplicado sobre el mismo conjunto, alcanzó una accuracy del 98,16 % y un área bajo la curva ROC del 99,55 %. El hecho de que un clasificador construido únicamente a partir de descriptores de bajo nivel alcance un rendimiento tan elevado muestra que existe una señal fácilmente explotable asociada a la procedencia de las imágenes, sin necesidad de utilizar las representaciones aprendidas por la CNN. Este control no permite determinar qué características concretas explican dicha separación: los diez descriptores se calculan sobre la imagen completa, y no puede afirmarse que estén libres de toda señal relacionada con la región tumoral.

De este resultado no se deriva que el modelo principal haya aprendido exclusivamente a distinguir el hospital de origen, ni que se haya demostrado que no distingue en absoluto la presencia de tumor; ninguna de las dos afirmaciones está

respaldada por el análisis realizado. La conclusión que sí puede sostenerse es más acotada: la tarea binaria presentaba una confusión estructural entre etiqueta y procedencia —tumor asociado siempre a Cheng, no tumor asociado siempre a IXI, por construcción—, de modo que su elevado rendimiento no permite separar, con la confianza suficiente, qué parte corresponde a una señal relacionada con el tumor y qué parte corresponde a una señal de dominio o de protocolo de adquisición.

Como evidencia adicional de que existe señal de dominio detectable dentro de estos datos, un clasificador entrenado para identificar el centro de adquisición dentro de las imágenes de IXI —es decir, dentro de una única clase de la tarea binaria— alcanzó una accuracy del 89,32 %. Esta cifra se aporta únicamente como indicio adicional de que la procedencia deja una señal reconocible en la imagen; no se atribuye esta señal a ningún elemento concreto del proceso de adquisición, como un modelo de escáner determinado, ya que esa atribución no ha sido verificada en este trabajo.

6.7 Reorientación hacia la clasificación multiclase

El resultado del control de sesgo descrito en la sección anterior llevó a una decisión metodológica central en la evolución de este proyecto: dejar de utilizar la tarea binaria tumor/no tumor como núcleo experimental del TFG.

La tarea binaria no se descarta ni se elimina de la memoria: se conserva como experimento histórico y, sobre todo, como caso de estudio metodológico sobre el sesgo de procedencia entre fuentes de datos, precisamente por lo que permitió aprender acerca del diseño de este tipo de experimentos. Lo que cambia es su papel dentro del trabajo: deja de presentarse como el resultado central y pasa a ser un antecedente que motiva la decisión descrita a continuación.

A partir de este punto, el núcleo experimental del proyecto se orienta hacia la clasificación multiclase dentro del dataset Cheng, distinguiendo entre glioma, meningioma y tumor pituitario. Esta reorientación tiene una ventaja metodológica concreta: al trabajar dentro de una única fuente de datos, las tres clases proceden de la misma cohorte de referencia, por lo que se elimina la asociación determinista entre clase y dataset que caracterizaba a la tarea binaria. Esto no implica que todos los pacientes de Cheng compartan las mismas condiciones de adquisición ni demuestra la ausencia de otras señales de dominio o posibles atajos: no está establecido que todos los pacientes de Cheng compartan protocolo, hospital, escáner o condiciones de adquisición idénticas.

Esto no debe interpretarse, por tanto, como una garantía de que todo sesgo desaparece por el simple hecho de trabajar con una única fuente. La reorientación

elimina específicamente el confusor determinista entre clase y dataset de procedencia que afectaba a la tarea binaria, pero no demuestra por sí sola la ausencia de otros posibles atajos dentro de la cohorte Cheng. Como control adicional, se aplicó una estrategia basada en descriptores de bajo nivel a la tarea multiclase sobre Cheng, análoga a la empleada en la tarea binaria. Este clasificador alcanzó una accuracy del 69,22 %. El resultado indica que dentro de la cohorte Cheng existen características de bajo nivel que permiten separar parcialmente las clases; no permite atribuir esta capacidad a un hospital, escáner o protocolo concreto, ni implica por sí mismo que el modelo multiclase final dependa de estos factores. Se interpreta como un control metodológico adicional, no como una invalidación de la tarea, y se retoma de forma explícita en el capítulo 10, al presentar el mismo control sobre los datos multiclase en el contexto de los resultados.

6.8 Dataset multiclase definitivo

El dataset que finalmente constituye el núcleo experimental de este TFG está formado por los 233 pacientes de Cheng, distribuidos en 89 pacientes con glioma, 82 con meningioma y 62 con tumor pituitario. En número de cortes, esta distribución se traduce en 1.426 cortes de glioma, 708 de meningioma y 930 de tumor pituitario, hasta un total de 3.064 cortes.

Sobre esta cohorte se estableció, ya en las primeras fases del proyecto, un split a nivel de paciente en tres subconjuntos: 163 pacientes de entrenamiento, 35 de validación y 35 de test, sumando los 233 pacientes totales. En número de cortes, esta misma partición corresponde a 2.091, 499 y 474 cortes respectivamente. La tabla 6.2 desglosa esta distribución por clase.

Tabla 6.2 — Dataset Cheng multiclase: distribución final por clase

Clase	Pacientes	Cortes	Train (pac.)	Validation (pac.)	Test (pac.)
Glioma	89	1.426	63	14	12
Meningioma	82	708	62	10	10
Pituitario	62	930	38	11	13
Total	233	3.064	163	35	35

Este split histórico de 163/35/35 pacientes es el punto de partida de todo el trabajo posterior sobre el dataset multiclase, pero no debe confundirse con dos usos distintos que se le dan más adelante en el proyecto. Por un lado, los 163 pacientes de entrenamiento y los 35 de validación se combinan, sumando 198 pacientes, como conjunto sobre el que se realiza la selección de arquitectura; los 35 de test

permanecen completamente al margen de esa selección y se reservan como conjunto de prueba final. Por otro lado, más adelante en el proyecto los 233 pacientes se reagrupan en un nuevo particionado en cinco partes para una validación cruzada de variabilidad interna, sin alterar por ello el split histórico aquí descrito. El protocolo completo de esta selección, de la evaluación final y de la validación cruzada se explica en detalle en el capítulo 9; en este capítulo únicamente interesa dejar constancia de cuál es la cohorte de partida y de que estos tres usos —split histórico, selección y validación cruzada— no son intercambiables entre sí.

7. Diseño y arquitectura software del pipeline

7.1 Visión global del sistema experimental

Los capítulos anteriores han descrito qué datos se utilizan y de dónde proceden. Este capítulo describe, en cambio, cómo está organizado el sistema software que convierte esos datos en experimentos configurables, trazables, verificables y reproducibles. El pipeline experimental de este TFG no se diseñó de antemano como una arquitectura formal, documentada con diagramas previos a la implementación; se trata de un sistema que fue creciendo a medida que el proyecto avanzaba, y la organización que se presenta en este capítulo es una reconstrucción a posteriori de esa organización a partir del sistema finalmente implementado, no la descripción de un diseño arquitectónico que existiera como tal antes de escribir el código.

Dicho esto, el sistema resultante sí puede describirse, con utilidad, como compuesto por varias responsabilidades diferenciadas que cooperan entre sí: una capa de datos (los datasets, sus manifiestos, los splits y el `patient_id` descritos en el capítulo 6), una capa de configuración (parámetros, rutas, semillas, modos de ejecución), una capa de procesamiento y modelado (preprocesamiento, entrenamiento, inferencia, agregación por paciente), una capa de validación (comprobaciones automáticas de separación, de cobertura y de regresión), una capa de ejecución (los notebooks ejecutados de forma no interactiva desde scripts SLURM mediante `jupyter nbconvert --execute` en el clúster HPC) y una capa de resultados (modelos, ficheros JSON y CSV, logs, protocolos, hashes y paquetes empaquetados). El resto del capítulo desarrolla cada una de estas responsabilidades y las conexiones entre ellas.

La pregunta que este capítulo pretende responder no es qué resultados se obtuvieron ni con qué arquitectura de red, sino cómo está diseñado el sistema para que un experimento concreto —una ejecución determinada, con una configuración determinada, sobre unos datos determinados— pueda identificarse, trazarse hasta su origen y, en la medida en que la evidencia lo permite, repetirse.

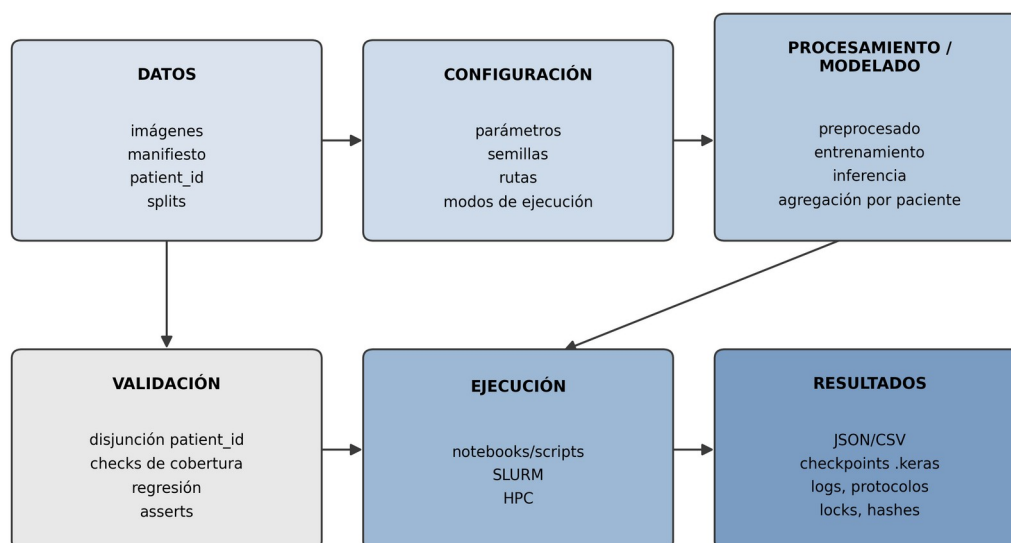
7.2 Arquitectura del pipeline

El pipeline no es una aplicación monolítica ni un servicio desplegado de forma continua: es un conjunto de notebooks y scripts que se ejecutan como experimentos discretos, cada uno de ellos delimitado en el tiempo, con una entrada y una salida bien definidas. La información fluye de una manera relativamente lineal, aunque con puntos de control explícitos en varios pasos intermedios. Los datos entran al sistema a través

de los manifiestos descritos en el capítulo 6, que actúan como la interfaz formal entre los ficheros de imagen físicos y el resto del pipeline. La configuración de cada ejecución determina el comportamiento del resto del sistema: qué datos se usan, con qué partición, bajo qué parámetros y en qué modo. Sobre esa combinación de datos y configuración actúan las funciones de procesamiento propiamente dichas, que entrenan modelos, generan predicciones y calculan métricas. Cada ejecución completa genera artefactos —ficheros de resultados, modelos entrenados, logs— que, a su vez, incorporan información de trazabilidad: los artefactos permiten relacionar el resultado con el manifiesto, la configuración, el particionado, el modelo y la ejecución que lo generaron, así como con el notebook o script conservado cuando este está disponible. Por último, las validaciones automáticas descritas más adelante en este capítulo pueden interrumpir el flujo de ejecución si detectan que una condición esperada no se cumple, en lugar de dejar que el experimento continúe silenciosamente sobre una base incorrecta.

Este diseño responde a la idea de pipeline experimental reproducible que se introdujo como criterio técnico en el capítulo 2: un pipeline en el que cada resultado queda acompañado de la información necesaria para identificar cómo se generó. Hay que aclarar, no obstante, qué no se pretende con esa expresión. No se garantiza que repetir una ejecución produzca un modelo binariamente idéntico al original, bit a bit. Lo que el diseño del pipeline persigue es una reproducibilidad del proceso: que el procedimiento, los datos y la configuración empleados en cada ejecución queden identificados con la precisión suficiente como para poder reconstruirlos y repetirlos, no que el resultado numérico vaya a coincidir cifra a cifra. Esta distinción, que se retoma con más detalle en la sección 7.4, es consecuencia directa de una decisión de configuración explícita del propio proyecto.

Figura 7.1 — Arquitectura lógica del pipeline experimental



Conjunto de notebooks y scripts ejecutados como experimentos discretos; no una arquitectura de microservicios, API o base de datos.

Figura 7.1 — Arquitectura del pipeline experimental

Las seis responsabilidades diferenciadas descritas en esta sección —datos, configuración, procesamiento/modelado, validación, ejecución y resultados— y el flujo general de información entre ellas.

7.3 Componentes y responsabilidades

Dentro de los notebooks del pipeline, buena parte de la lógica de procesamiento se organiza en un conjunto reducido de funciones reutilizables, en lugar de repetirse el mismo código en cada celda. Esta sección describe la responsabilidad arquitectónica de las principales funciones identificadas, entendida como el papel que cada una cumple dentro del flujo general del sistema, no como una documentación exhaustiva de su implementación interna ni de las fórmulas concretas que emplea, que corresponden al capítulo 8.

La tabla 7.1 resume estos componentes.

Tabla 7.1 — Componentes del pipeline

Componente	Responsabilidad	Artefactos principales
metricas	Calcula las métricas de evaluación a partir de las predicciones y las	Campos numéricos de los JSON de resultados

Componente	Responsabilidad	Artefactos principales
	etiquetas reales de un conjunto dado	
por_paciente	Agrega las predicciones de los distintos cortes pertenecientes a un mismo paciente antes de que se calculen las métricas finales	Predicciones agregadas por paciente, incorporadas a los JSON de resultados
predecir	Genera las predicciones del modelo sobre un conjunto de imágenes dado	Predicciones intermedias, consumidas por por_paciente y metricas
entrenar_o_cargar	Decide si un modelo debe entrenarse desde cero o cargarse a partir de un checkpoint ya existente, evitando reentrenamientos innecesarios	Checkpoints .keras
evaluar_y_guardar_modelo	Ejecuta la evaluación de un modelo ya entrenado y persiste sus resultados junto con los metadatos de trazabilidad asociados	JSON de resultados, hashes asociados
ultima_conv	Localiza la última capa convolucional del modelo, paso previo necesario para el análisis de explicabilidad	Referencia interna a una capa del modelo
gradcam	Genera los mapas de activación de clase empleados en el análisis de explicabilidad post-hoc	Mapas de activación

Esta descripción se apoya en el nombre y el papel de cada función dentro del pipeline, no en un contrato de interfaz formalmente documentado antes de escribir el

código; el objetivo de la tabla es dejar constancia de qué responsabilidad cumple cada pieza dentro del sistema, no ofrecer una especificación de API. La responsabilidad de `entrenar_o_cargar` conecta con el mecanismo de reutilización de checkpoints descrito posteriormente: antes de reutilizar un modelo ya guardado, el pipeline comprueba su compatibilidad con el experimento mediante las huellas asociadas al manifiesto, al particionado y a la configuración.

7.4 Gestión de configuración

Uno de los rasgos de ingeniería del software más claros del pipeline es la centralización de la configuración de cada ejecución en una sección inicial del propio notebook, en lugar de dispersar parámetros por distintas celdas del código. Esta sección concentra elementos como las semillas aleatorias, el tamaño de imagen empleado, las rutas de entrada y salida, los umbrales de decisión y los flags que determinan el modo de ejecución.

Desde una perspectiva de ingeniería del software, esta centralización cumple varias funciones. Reduce el riesgo de que un mismo parámetro tome valores distintos en diferentes puntos del código sin que ese cambio quede documentado. Facilita repetir una ejecución con la misma configuración, o con una variación deliberada y controlada de un único parámetro. Permite, además, identificar de un vistazo con qué configuración se generó un resultado concreto, en lugar de tener que reconstruirla inspeccionando el código completo del notebook. Como mecanismo de identificación de esa configuración, los artefactos de la Etapa B incorporan un campo `config_hash`, que permite comparar si dos artefactos están asociados a la misma configuración registrada por el pipeline, sin necesidad de comparar manualmente cada valor. Esta sección no desarrolla los hiperparámetros concretos del modelo final —tasa de aprendizaje, número de épocas, tamaño de lote y similares—, que se presentan en el capítulo 8; lo relevante aquí es el mecanismo de centralización e identificación de la configuración, no sus valores particulares.

Dentro de esta misma configuración se documenta una decisión consciente sobre el determinismo de la ejecución: el determinismo estricto de GPU no se activó explícitamente. En la práctica, esto significa que sí se fijaron semillas aleatorias y sí quedaron registrados los parámetros de configuración relevantes de las ejecuciones documentadas, pero no se activaron los mecanismos adicionales que fuerzan un comportamiento determinista a nivel de operaciones de GPU. La consecuencia directa de esta decisión es que la reproducibilidad del proceso no equivale a la identidad bit a bit del resultado: repetir una ejecución con la misma configuración y los mismos datos reconstruye el mismo procedimiento, pero no garantiza que el modelo resultante sea

numéricamente idéntico al original, cifra a cifra. Conviene ser explícitos en un punto: esta ausencia de determinismo estricto no se emplea en esta memoria para explicar automáticamente ninguna diferencia numérica observada entre una ejecución histórica y su reproducción posterior; la causa concreta de una diferencia de ese tipo no queda demostrada por el mero hecho de que el determinismo estricto no estuviera activado, y no se atribuye a esta decisión de configuración sin evidencia adicional que lo respalde.

7.5 Modelo de datos y manifiestos

El capítulo 6 ya explicó en detalle el contenido del manifiesto de datos —los campos `patient_id`, `source`, `sha256` y `local_file`— y el procedimiento de control de duplicados aplicado sobre él; esta sección no repite ese desarrollo, sino que sitúa el manifiesto dentro de la arquitectura del pipeline.

Desde el punto de vista del diseño software, el manifiesto funciona como el contrato de entrada del sistema: es el punto de unión entre los datos físicos, la partición en conjuntos de entrenamiento/validación/test, la configuración de cada ejecución y los resultados que esa ejecución produce. El manifiesto proporciona el punto de referencia común desde el que el pipeline relaciona las imágenes con sus pacientes, fuentes y particiones, en lugar de que cada componente acceda a los ficheros físicos de forma independiente y sin ese punto de referencia compartido. Los artefactos de resultados generados posteriormente conservan, mediante el campo `manifest_hash`, una referencia a qué estado del manifiesto se utilizó para producirlos, lo que permite reconstruir después de qué versión de los datos procede un resultado concreto sin necesidad de volver a comparar los ficheros físicos.

En este punto es necesario distinguir tres nociones que conviven en el pipeline pero que no son intercambiables: los datos físicos (las propias imágenes almacenadas en disco), los metadatos (la información descriptiva sobre esas imágenes recogida en el manifiesto, como el paciente o la fuente de procedencia) y los identificadores o huellas (los distintos hashes que permiten referirse a un estado concreto de esos datos o metadatos sin tener que volver a inspeccionarlos por completo). El procedimiento de control de duplicados mediante pHash y SSIM descrito en el capítulo 6 no se vuelve a desarrollar aquí; a efectos de este capítulo basta con situarlo como parte de la capa de datos que alimenta al resto del pipeline.

7.6 Trazabilidad de experimentos y artefactos

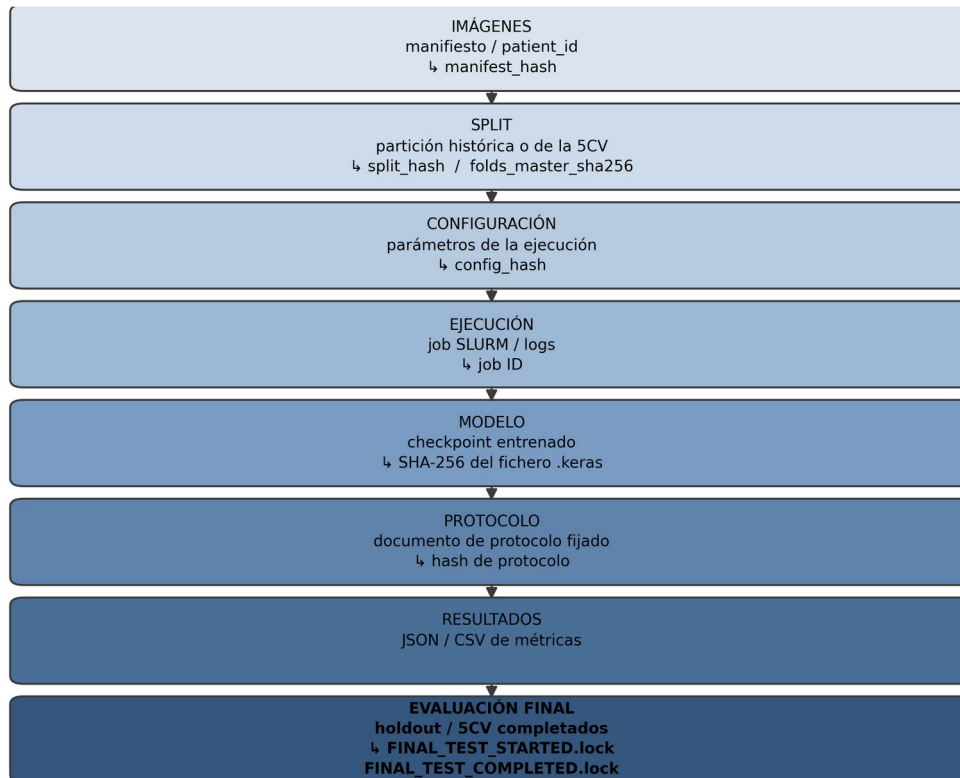
Esta es, posiblemente, la sección más central del capítulo, porque explica cómo una ejecución concreta del pipeline queda asociada, de forma verificable, al manifiesto de datos, al split empleado, a la configuración utilizada, al modelo entrenado, a las métricas obtenidas, a los logs generados y al job de ejecución que la

produjo. Esta cadena de trazabilidad es lo que permite, en principio, reconstruir el origen de cualquier artefacto de resultados sin depender únicamente de la memoria de quien lo generó.

El mecanismo se apoya en varios identificadores que cumplen funciones distintas y que conviene no confundir entre sí, aunque compartan la palabra “hash” en su nombre. El `manifest_hash` es un campo interno que el propio pipeline incorpora en cada artefacto de resultados para señalar de qué manifiesto de datos procede; no debe confundirse con el cálculo de SHA-256 del fichero `manifest_final.csv` completo que se empleó en el capítulo 6 para comparar dos copias del manifiesto entre sí —esa comparación fue un procedimiento de auditoría posterior sobre el fichero completo, mientras que el `manifest_hash` es un campo que el pipeline registra en el momento de ejecutar el experimento—. De manera análoga, el `config_hash` identifica la configuración concreta de una ejecución, descrita en la sección 7.4, y es un identificador distinto tanto del `manifest_hash` como del hash SHA-256 de cualquier fichero de imagen individual. Por su parte, el `split_hash` del split histórico 163/35/35 descrito en el capítulo 6 se conserva como referencia de integridad de ese particionado concreto, y es también un identificador distinto del que emplea el particionado en cinco partes utilizado más adelante en el proyecto para la validación de variabilidad interna —ese particionado dispone de su propio identificador (`folds_master_sha256`), diferente del `split_hash` histórico—; esta distinción se menciona aquí únicamente para mostrar que cada tipo de partición dispone de un identificador propio y no intercambiable, sin desarrollar en este capítulo el protocolo de esa validación cruzada, que corresponde al capítulo 9.

Como ejemplo concreto de esta cadena de trazabilidad puede citarse el modelo final del proyecto: su fichero de checkpoint tiene un hash SHA-256 registrado que se comprobó consistente entre el JSON de resultados de la evaluación final, el fichero `.lock` asociado y un listado independiente de huellas de los modelos finales. No es necesario reproducir aquí ese valor hexadecimal completo —puede consultarse en la tabla técnica de trazabilidad—; lo relevante en este capítulo es que la consistencia de ese hash entre los distintos artefactos es lo que permite afirmar que todos ellos se refieren al mismo fichero de modelo, sin que ello implique haber inspeccionado el contenido interno de ese binario más allá de su huella.

Figura 7.2 — Cadena de trazabilidad de datos, configuración, modelo y resultados



manifest_hash, config_hash, split_hash y el SHA-256 de un fichero son identificadores distintos, no intercambiables entre sí.

Figura 7.2 — Cadena de trazabilidad de artefactos

Los pasos que conectan el manifiesto de datos, el split, la configuración de la ejecución, el modelo entrenado, las métricas obtenidas, los logs generados y el job de ejecución que la produjo, descritos en esta sección.

7.7 Integridad, hashes y persistencia

Tres nociones tienden a mezclarse en el lenguaje habitual y conviene distinguirlas con precisión: la trazabilidad, entendida como la capacidad de saber de dónde procede un artefacto; la integridad, entendida como la capacidad de detectar si ese artefacto ha cambiado respecto al estado que se registró originalmente; y la reproducibilidad, entendida como la capacidad de reconstruir o repetir el procedimiento que lo generó a partir de la información registrada. El pipeline aborda las tres mediante mecanismos relacionados pero distintos, y este capítulo procura no emplearlas como sinónimos.

La integridad se apoya principalmente en hashes SHA-256, calculados de forma sistemática sobre los checkpoints de modelos y otros artefactos de resultados.

La Etapa B documenta, mediante un listado y sus huellas correspondientes, 24 checkpoints .keras; esto permite comprobar en cualquier momento posterior si un fichero de modelo concreto coincide con el que se registró en su momento, pero no equivale a haber examinado el contenido interno de esos 24 checkpoints —su nivel de evidencia respecto a su contenido interno real sigue siendo el de un artefacto no inspeccionado directamente, aunque su huella documentada sí esté verificada—. La persistencia de los resultados se apoya en varios tipos de fichero con funciones complementarias: los ficheros de resultados en JSON, que recogen las métricas y los metadatos de cada ejecución; los propios checkpoints de modelo; los ficheros de protocolo, que fijan por escrito las condiciones de una evaluación antes de realizarla; los ficheros .lock, que se describen con más detalle en la sección 7.10; y los logs de ejecución (.out/.err), que registran lo sucedido durante cada job.

7.8 Validaciones automáticas y gestión de errores

El pipeline incorpora varios tipos de validación automática que pueden interrumpir la ejecución si detectan que una condición esperada no se cumple, en lugar de dejar que el experimento continúe sobre una base incorrecta. Resulta útil distinguir varios tipos de estas validaciones según lo que protegen.

Existen invariantes de datos: comprobaciones que verifican la disjunción de patient_id entre conjuntos, la unicidad de las huellas SHA-256 de las imágenes y la cobertura completa de la cohorte esperada. También hay invariantes experimentales: comprobaciones sobre el número de pacientes esperado en cada partición o sobre el número de parámetros del modelo, verificado mediante un assert antes de entrenar cada fold de la validación cruzada de robustez, de modo que una discrepancia en la arquitectura del modelo se detecta antes de invertir tiempo de cómputo en un entrenamiento erróneo. Existe, además, un mecanismo de comparación de regresión frente a un resultado histórico de referencia, con una tolerancia configurada de $\pm 0,03$: si la reproducción de ese resultado se aparta del valor histórico en más de esa tolerancia, el propio código lanza una excepción (RuntimeError) que detiene la ejecución. Por último, hay manejo de errores explícito mediante bloques try/except, documentado concretamente en la carga opcional de datos desde Colab, con una alternativa de datos de prueba si esa carga falla; este manejo de errores no está generalizado a todo el pipeline, sino verificado en ese punto concreto.

El mecanismo de comparación de regresión frente al histórico se puede ilustrar con un caso real: en una de las ejecuciones del proyecto (job 70886), la reproducción del resultado se separó de la cifra histórica de referencia más allá de la tolerancia configurada, lo que provocó que el propio código lanzara la excepción correspondiente

y detuviera la ejecución antes de continuar. Esta memoria no desarrolla en este capítulo la magnitud exacta de esa diferencia ni las cifras históricas y reproducidas que la originan, porque son resultados experimentales que pertenecen al capítulo 10; lo que interesa aquí es el mecanismo en sí: la ejecución fue efectivamente interrumpida por una validación automática al superarse la tolerancia configurada.

No toda la evidencia disponible sobre este mecanismo es igual de completa, y conviene no ocultarlo. El mecanismo general de aborto por regresión —la comparación seguida de la excepción incondicional— está confirmado en el código fuente disponible. Sin embargo, existe una ejecución posterior en la que el registro de resultados indica explícitamente que la comparación de regresión no se aceptó (`regresion_aceptada: false`) y, pese a ello, el job continuó su ejecución sin detenerse. La causa exacta de esa discrepancia no puede reconstruirse con el código actualmente disponible: no existe una versión del notebook, entre las conservadas, que contenga el código exacto de fine-tuning ejecutado esa noche. Este caso se documenta aquí como una limitación de trazabilidad histórica del propio proyecto, no como un fallo caracterizado con una causa conocida.

Como ejemplos adicionales de invariantes verificadas mediante `assert`, cabe mencionar la comprobación de que la totalidad de los pacientes de la cohorte multiclase queda cubierta, sin excepción, al reagruparlos en el particionado de cinco partes de la validación de robustez, y la comprobación del número exacto de parámetros del modelo —tanto los parámetros totales como los entrenables durante el ajuste fino— antes de iniciar el entrenamiento de cada fold. Estas cifras concretas no se detallan aquí, ya que en esta sección únicamente se utilizan para ilustrar el mecanismo de validación automática.

La tabla 7.2 resume los principales mecanismos de trazabilidad y validación descritos en este capítulo.

Tabla 7.2 — Mecanismos de trazabilidad/validación

Mecanismo	Objeto protegido	Finalidad	Artefacto asociado
<code>manifest_hash</code>	Estado del manifiesto de datos empleado	Identificar de qué versión de los datos procede un artefacto de resultados	JSON de resultados
<code>config_hash</code>	Configuración de la ejecución	Comparar si dos artefactos están asociados a la	JSON de resultados de la

Mecanismo	Objeto protegido	Finalidad	Artefacto asociado
		misma configuración registrada por el pipeline	Etapas B
SHA-256	Integridad de ficheros de modelo y de resultados	Detectar si un fichero ha cambiado respecto al estado registrado originalmente	Checkpoints .keras / sha256_finales.txt
Ficheros .lock	Identidad de modelo, protocolo y split al inicio y al final de la evaluación final	Comprobar que los hashes registrados al inicio y al final de la evaluación son consistentes entre sí	FINAL_TEST_STARTED.lock / FINAL_TEST_COMPLETED.lock
Chequeo de regresión (tolerancia $\pm 0,03$)	Resultado reproducido frente al histórico de referencia	Interrumpir la ejecución si la diferencia supera la tolerancia configurada	Notebook multiclase / log del job 70886
Asserts de disjunción y cobertura	Separación entre conjuntos y cobertura completa de la cohorte	Interrumpir el flujo si se incumple una condición de partición esperada	Notebooks / preparar_5cv_robustez.py / agregar_5cv_robustez.py

7.9 Automatización de experimentos mediante HPC y SLURM

La ejecución de los experimentos se automatizó mediante un clúster de computación de altas prestaciones gestionado con SLURM, lo que permite lanzar cada experimento como un trabajo (job) no interactivo, gestionado como una ejecución independiente e identificado mediante su propio job ID. Cada notebook se ejecuta de forma no interactiva desde un script SLURM mediante `jupyter nbconvert --execute`, sin edición manual celda a celda durante la ejecución en el clúster; esto permite que la ejecución quede asociada al notebook, la configuración y los datos empleados,

evitando la edición manual durante el job. Cada job genera sus propios logs de salida y de error (.out/.err), que quedan asociados a su job ID concreto y sirven como registro de lo sucedido durante esa ejecución.

A lo largo del proyecto se documentan 16 IDs de job distintos en este HPC. No se listan aquí de forma individual —esa relación completa, junto con los recursos empleados por cada uno, pertenece al capítulo 12 y al Anexo B del documento de anexos técnicos—; en este capítulo interesa únicamente el concepto de que el job ID actúa como un elemento más de la cadena de trazabilidad, que permite asociar un resultado concreto a la ejecución exacta que lo produjo. En el script de benchmark multiclase se incorporó además un mecanismo `trap ... EXIT` para copiar logs y artefactos al finalizar la ejecución, como medida adicional para asegurar que esa información quedara disponible incluso si el job terminaba de forma inesperada; este mecanismo es específico de ese script y no una práctica generalizada al resto de los scripts SLURM del proyecto. Esta automatización se describe aquí en su dimensión de ingeniería del software —como mecanismo para ejecutar experimentos de forma reproducible y no interactiva—, no como un análisis de los recursos hardware empleados, que corresponde al capítulo 12; tampoco se trata de un sistema desplegado de forma continua para un uso operativo, sino de la ejecución automatizada de experimentos de investigación en el clúster HPC.

7.10 Reproducibilidad y empaquetado de resultados

Como ejemplo concreto de control del ciclo de evaluación, el protocolo de evaluación final del proyecto se fijó por escrito antes de abrir el conjunto de test reservado, y ese documento de protocolo quedó identificado mediante su propio hash, citado a su vez en el JSON de resultados de la evaluación final; en ese mismo momento quedaron también identificados, mediante sus respectivos hashes, el modelo evaluado y el split empleado. El propio ciclo de evaluación se delimita mediante un par de ficheros .lock: uno que se crea al iniciar la evaluación (FINAL_TEST_STARTED.lock) y otro que se crea al finalizarla (FINAL_TEST_COMPLETED.lock). La comparación permite comprobar que los hashes del modelo, del protocolo y del split registrados al inicio y al final de la evaluación son consistentes entre sí. Esta memoria no adelanta en este capítulo el resultado numérico de esa evaluación final, que se presenta en el capítulo 10; lo que interesa aquí es el mecanismo de control en sí.

Los resultados y artefactos generados a lo largo del proyecto se exportaron y empaquetaron mediante `tar`, generándose un paquete completo y un paquete ligero adicional. El log disponible del job de exportación correspondiente está truncado, sin una línea explícita de finalización correcta; por ese motivo, esta memoria no afirma

que ese job terminara correctamente, sino, con más precisión, que se conservan posteriormente los paquetes generados, aunque el log disponible del job de exportación esté truncado.

Para cerrar esta sección, merece la pena fijar con precisión qué se entiende por reproducibilidad a lo largo de todo este capítulo: la capacidad de reconstruir y repetir el procedimiento experimental a partir de los datos identificados, el split empleado, la configuración registrada, el código disponible, el protocolo escrito, los logs generados y los artefactos persistidos. No se entiende, en cambio, como la capacidad de obtener exactamente los mismos bits o el mismo modelo binario tras repetir una ejecución, algo que la propia decisión de no activar el determinismo estricto de GPU, descrita en la sección 7.4, hace explícitamente imposible de garantizar. Esta noción de reproducibilidad es la misma que se planteó como criterio técnico en el capítulo 2, y este capítulo ha descrito el conjunto de mecanismos —manifiestos, configuración centralizada, hashes, validaciones automáticas, protocolos y ficheros `.lock`— mediante los cuales el pipeline intenta satisfacerla.

7.11 Síntesis de las decisiones de diseño

El recorrido de este capítulo permite reconstruir la cadena completa que conecta los datos con la evidencia final del proyecto: los datos entran al sistema a través de un manifiesto que actúa como contrato de entrada; una configuración centralizada e identificable mediante hash determina cómo se procesan esos datos en cada ejecución; los componentes de procesamiento —entrenamiento, inferencia, agregación por paciente— actúan sobre esa combinación de datos y configuración; una serie de validaciones automáticas puede interrumpir el flujo si se incumple una condición esperada, ya sea de datos, de configuración o de regresión frente a un resultado histórico; la ejecución se automatiza y se gestiona como trabajos independientes de un clúster HPC gestionado con SLURM; y los principales artefactos resultantes conservan identificadores y huellas que permiten relacionarlos con los estados registrados de los datos, la configuración, el modelo y la ejecución que los produjo.

Ninguna de estas decisiones se presenta aquí como un diseño arquitectónico completo o cerrado desde el principio del proyecto: son, más bien, las piezas que fueron incorporándose para responder a necesidades concretas de trazabilidad, integridad y control de errores identificadas durante el propio desarrollo, en línea con lo ya señalado en el capítulo 2 sobre el origen progresivo de los criterios técnicos del trabajo. Lo que este capítulo aporta es la visión de conjunto de esas piezas como un sistema software coherente, y no como una colección de scripts inconexos.

8. Metodología de Deep Learning

8.1 Preprocesamiento de las imágenes

Antes de entrar en cualquier red, cada imagen se somete a un preprocesamiento común, verificado directamente en los artefactos del proyecto. El primer paso es un recorte a la caja del cerebro: el recorte centra el área cerebral y reduce la extensión de fondo y márgenes en la entrada de la red. A continuación se aplica una normalización por percentiles 1-99, que acota el rango de intensidades utilizado por el pipeline y reduce la influencia de valores extremos de intensidad. Por último, cada imagen se redimensiona a 224×224 píxeles, el tamaño de entrada fijado por el pipeline para las arquitecturas evaluadas.

Es importante ser precisos sobre lo que este preprocesamiento no incluye. No se ha verificado la aplicación de técnicas adicionales como corrección de campo de sesgo (bias field correction), registro espacial entre imágenes, segmentación previa del tumor, normalización z-score, ecualización adaptativa de contraste (CLAHE) ni aumentos de datos (data augmentation) sistemáticos; estas técnicas son habituales en otros trabajos de imagen médica, pero no forman parte de la evidencia disponible sobre este pipeline, por lo que no se describen aquí como parte del procedimiento.

El número de canales de entrada depende de qué modelo se utilice. La CNN propia empleada en el baseline histórico recibe la imagen en un único canal, en escala de grises, coherente con el origen de la imagen como una resonancia magnética. Los modelos preentrenados, en cambio, reciben tres canales: se replica el mismo canal de intensidad tres veces y, cuando corresponde, se aplica además la función `preprocess_input` específica de cada arquitectura. Esta réplica de canales responde a un motivo puramente técnico: las arquitecturas preentrenadas con ImageNet esperan una entrada RGB de tres canales, y replicar el canal original es la forma de satisfacer esa expectativa de forma. Este paso no debe malinterpretarse: la resonancia magnética no se convierte en una imagen a color ni adquiere información cromática real; se trata de una replicación del mismo canal de intensidad, sin que ello añada ninguna información adicional a la que ya contenía la imagen original en escala de grises.

8.2 Arquitecturas consideradas y aprendizaje por transferencia

La selección de arquitectura de este trabajo comparó cinco arquitecturas convolucionales bajo un mismo protocolo: VGG16, ResNet50, MobileNetV2, EfficientNetB0 e InceptionV3. Estas cinco arquitecturas no se eligieron al azar, sino que representan familias de diseño distintas dentro del Deep Learning aplicado a

visión por computador, lo que hace razonable compararlas entre sí como candidatas para la misma tarea. VGG16 representa el diseño clásico de bloques secuenciales de convolución y agrupamiento, con una profundidad moderada y una estructura sencilla de interpretar (Simonyan & Zisserman, 2014). ResNet50 introduce conexiones residuales que permiten entrenar redes más profundas sin la degradación de rendimiento asociada a arquitecturas puramente secuenciales muy profundas (He et al., 2016). MobileNetV2 pertenece a una familia orientada a la eficiencia computacional mediante convoluciones separables en profundidad (Sandler et al., 2018). EfficientNetB0, por su parte, aplica un escalado compuesto de profundidad, anchura y resolución (Tan & Le, 2019). InceptionV3, por su parte, se apoya en módulos que combinan convoluciones de distinto tamaño en paralelo dentro de un mismo bloque, capturando patrones a varias escalas simultáneamente (Szegedy et al., 2016). Esta diversidad de enfoques —profundidad, conexiones residuales, eficiencia paramétrica, escalado compuesto y diseño multiescala— es lo que justifica someterlas a una misma comparación, en lugar de partir de una única arquitectura elegida de antemano.

Esta sección no desarrolla el resultado de esa comparación: qué arquitectura obtuvo mejor rendimiento en cada fase es una cuestión que pertenece al capítulo 10. Aquí interesa únicamente qué se comparó y bajo qué estrategia metodológica.

Las cinco arquitecturas se emplean mediante aprendizaje por transferencia (transfer learning), partiendo en todos los casos de pesos preentrenados sobre ImageNet. La estrategia consiste en reutilizar las representaciones visuales ya aprendidas por cada red sobre ese conjunto de imágenes generalista, en lugar de entrenar cada arquitectura desde cero, y sustituir o añadir sobre esa base una cabeza de clasificación específica para las tres clases de este problema (glioma, meningioma, tumor pituitario), descrita en la sección 8.4. La estrategia se organiza en dos fases. En una primera fase, cada una de las cinco arquitecturas se evalúa con su base preentrenada congelada, entrenando únicamente la cabeza de clasificación añadida. En una segunda fase, solo los dos candidatos con mejor rendimiento en esa fase frozen pasan a una etapa adicional de fine-tuning, en la que se permite ajustar también una parte de las capas finales del backbone. Esta memoria no adelanta en este capítulo cuáles fueron esos dos candidatos ni su rendimiento relativo, porque esa selección es en sí misma un resultado experimental que se presenta en el capítulo 10; esta estrategia frozen → fine-tuning es, en todo caso, una decisión metodológica sobre cómo estructurar la comparación, y no una garantía de que el fine-tuning vaya a mejorar el resultado de cada arquitectura.

Tabla 8.1 — Arquitecturas consideradas

Arquitectura	Familia	Uso en este trabajo
VGG16	CNN profunda clásica, bloques secuenciales de convolución	Evaluada en la fase frozen
ResNet50	Red residual con conexiones de salto	Evaluada en la fase frozen
MobileNetV2	Arquitectura eficiente basada en convoluciones separables en profundidad	Evaluada en la fase frozen
EfficientNetB0	Familia con escalado compuesto de profundidad, anchura y resolución	Evaluada en la fase frozen
InceptionV3	Arquitectura con módulos de convoluciones paralelas multiescala	Evaluada en la fase frozen

8.3 Evolución desde el baseline histórico hasta el modelo final

A lo largo del proyecto conviven dos configuraciones que no deben confundirse. La primera es un baseline histórico, una CNN propia entrenada desde cero (sin transfer learning), que sirvió como punto de referencia en una fase anterior del trabajo. La segunda es la configuración final: una de las arquitecturas preentrenadas descritas en la sección anterior, sometida a la estrategia de transfer learning y, en su caso, a fine-tuning. Ambas configuraciones comparten la misma función de pérdida —`sparse_categorical_crossentropy`—, verificada de forma independiente en cada una de ellas; esta coincidencia no implica que ambos modelos sean equivalentes ni que compartan el resto de su configuración, que difiere en arquitectura, en el uso de pesos preentrenados y en varios de los hiperparámetros descritos en la sección 8.4.

El baseline histórico puede describirse brevemente como un punto de referencia útil durante la evolución metodológica del proyecto, no como la configuración final del trabajo. Su entrenamiento se detenía mediante un criterio de `EarlyStopping` sobre `val_accuracy`, con una paciencia de 8 épocas sin mejora, sin el uso de `ReduceLROnPlateau` ni de ponderación de clases (`class_weight`), y con una

semilla aleatoria fija de valor 42. Esta memoria no desarrolla aquí el resultado numérico de ese baseline, que se presenta en el capítulo 10; el objetivo de mencionarlo en este punto es mostrar la evolución metodológica del proyecto —de una CNN entrenada desde cero a una estrategia de transfer learning con selección comparada de arquitecturas—, no narrar de nuevo la historia experimental completa del trabajo.

8.4 Configuración de entrenamiento y fine-tuning

La cabeza de clasificación añadida sobre cada backbone preentrenado sigue una estructura común: una capa de Global Average Pooling, seguida de una capa de Dropout con una tasa de 0,4, una capa densa intermedia de 128 unidades con activación ReLU, una segunda capa de Dropout con una tasa de 0,3, y finalmente una capa densa de salida de 3 unidades con activación softmax. Cada uno de estos elementos cumple un papel conceptual concreto. La capa de Global Average Pooling reduce la representación espacial final del backbone —un mapa de activaciones con extensión espacial— a un único vector compacto, promediando cada canal sobre toda su extensión espacial. Las capas de Dropout introducen una regularización durante el entrenamiento, desactivando aleatoriamente una fracción de las unidades en cada paso para reducir el riesgo de sobreajuste. La capa densa intermedia de 128 unidades introduce una transformación aprendida específica para la tarea antes de la salida final. La capa final, con activación softmax, produce una distribución de probabilidad sobre las tres clases del problema, de modo que las tres salidas suman 1 y cada una puede interpretarse como la probabilidad estimada de que la imagen pertenezca a esa clase. No se atribuye a esta arquitectura de cabeza ninguna ventaja cuantificada frente a otras posibles configuraciones, ya que no se ha realizado en este trabajo una comparación experimental entre distintas cabezas de clasificación.

La función de pérdida empleada en el modelo final es `sparse_categorical_crossentropy`, verificada directamente en el script de entrenamiento. Esta función es coherente con la naturaleza del problema: se trata de una clasificación multiclase con tres categorías mutuamente excluyentes —cada corte pertenece a una única clase—, y las etiquetas se codifican como números enteros en lugar de vectores one-hot, que es precisamente el formato de etiqueta que esta variante de la entropía cruzada espera. De forma simplificada, para un ejemplo cuya clase verdadera es y y cuya probabilidad predicha para esa clase es p_y , la pérdida se calcula como $L = -\log(p_y)$: cuanto más próxima a 1 sea la probabilidad asignada a la clase correcta, menor es la pérdida, y cuanto más se aproxime a 0, mayor.

El entrenamiento del modelo final se organiza en dos fases claramente diferenciadas. En la primera fase, con el backbone preentrenado completamente congelado, se entrena únicamente la cabeza de clasificación añadida, utilizando el optimizador Adam con una tasa de aprendizaje de $1e-4$ durante 6 épocas. En la segunda fase, de fine-tuning, se consideran las últimas 20 capas del backbone para el ajuste fino, manteniendo congeladas las capas BatchNormalization, también con el optimizador Adam pero con una tasa de aprendizaje considerablemente menor, de $1e-5$, durante otras 6 épocas. El uso de una tasa de aprendizaje menor en esta segunda fase responde a una motivación metodológica general: introducir cambios más pequeños y conservadores sobre unos pesos que ya proceden de un preentrenamiento útil, en lugar de permitir que el ajuste fino altere de forma brusca esas representaciones. Esta memoria no presenta esa motivación como un mecanismo de protección frente al olvido catastrófico demostrado empíricamente en este proyecto, sino como el razonamiento metodológico general que orientó la elección de esa tasa de aprendizaje.

Un aspecto verificado con evidencia explícita, y que no debe darse por supuesto, es que las capas de BatchNormalization permanecieron congeladas durante todo el proceso de fine-tuning, incluso en las capas del backbone que sí se consideraron para el ajuste. Esto no se ha deducido únicamente a partir del número de parámetros entrenables del modelo: esta configuración está confirmada en los artefactos finales de evaluación consultados, mediante un campo explícito registrado por el propio pipeline, que indica que las capas BatchNormalization permanecieron en estado no entrenable durante el fine-tuning.

El tamaño de lote (batch size) utilizado en ambas fases del entrenamiento es de 32 imágenes. La tabla 8.2 resume de forma conjunta esta configuración de entrenamiento.

Tabla 8.2 — Configuración de entrenamiento

Elemento	Fase frozen/cabeza	Fase fine-tuning
Optimizador	Adam	Adam
Learning rate	$1e-4$	$1e-5$
Épocas	6	6
Batch size	32	32
Backbone	Congelado	Últimas 20 capas consideradas para ajuste
BatchNormalization	Congelada	Congelada

Elemento	Fase frozen/cabeza	Fase fine-tuning
Loss	<code>sparse_categorical_crossentropy</code>	<code>sparse_categorical_crossentropy</code>

8.5 Evaluación a nivel de paciente y métricas

El modelo produce sus predicciones inicialmente a nivel de corte: cada imagen individual recibe una probabilidad para cada una de las tres clases. Sin embargo, la unidad principal de evaluación definida en este trabajo es el paciente, por lo que estas predicciones se agregan antes de calcular las métricas finales. La regla de agregación implementada es la media aritmética: para cada paciente, la probabilidad asignada a cada clase es el promedio de las probabilidades de esa clase entre todos los cortes pertenecientes a ese paciente. Formalmente, si $P_i(c)$ es la probabilidad predicha de la clase c para el corte i de un paciente, y ese paciente tiene N_p cortes, la probabilidad agregada por paciente para la clase c es:

$$P_{\text{paciente}}(c) = (1 / N_p) \cdot \sum_i P_i(c)$$

Esta es la única regla de agregación implementada en el código para la tarea multiclase: cualquier valor del parámetro de agregación distinto de “media” provoca explícitamente un error (`NotImplementedError`) en tiempo de ejecución. Este detalle no es meramente anecdótico: demuestra que no existe una regla de agregación alternativa oculta ni configurable silenciosamente, sino una única regla, fija y verificable en el propio código.

Una vez calculada la probabilidad media por clase para un paciente, la clase final asignada a ese paciente es aquella con mayor probabilidad media, es decir, el argumento máximo (`argmax`) sobre las tres probabilidades agregadas:

$$\text{pred_paciente} = \text{argmax}_c P_{\text{paciente}}(c)$$

Esta predicción final por paciente —no la probabilidad de un corte individual— es la unidad sobre la que se calculan las métricas principales de evaluación de este trabajo. No deben confundirse ambos niveles: una probabilidad alta o baja en un corte concreto no determina por sí sola la predicción final de un paciente, que depende del promedio sobre todos sus cortes.

La métrica principal de evaluación de este trabajo es la *balanced accuracy*, calculada a nivel de paciente. Conceptualmente, la *balanced accuracy* es la media no ponderada del *recall* obtenido en cada una de las clases, lo que puede expresarse como:

$$BA = (1 / K) \cdot \sum_k \text{Recall}_k$$

donde K es el número de clases (en este trabajo, $K = 3$: glioma, meningioma y tumor pituitario) y Recall_k es la proporción de pacientes de la clase k correctamente

identificados como tales. Dado que las tres clases no contienen el mismo número de pacientes, una accuracy global pondera implícitamente el resultado según la frecuencia de cada clase. La balanced accuracy, al calcular la media no ponderada de los recalls por clase, asigna el mismo peso a cada una de ellas. Esto no significa que la balanced accuracy sea la única métrica válida para este problema, sino que se emplea aquí como la métrica principal del trabajo, complementada por otras métricas.

Junto a la balanced accuracy se registran también dos métricas secundarias: la accuracy simple, entendida como la fracción global de pacientes correctamente clasificados sin ponderar por clase, y el macro-F1, entendido como la media no ponderada del F1 obtenido en cada una de las tres clases. Esta memoria no presenta en este capítulo ningún valor numérico de estas métricas, ni desarrolla aquí el uso de intervalos de confianza, procedimientos de bootstrap o matrices de confusión, que se introducen con su protocolo completo en el capítulo 9 y con sus valores concretos en el capítulo 10; aquí basta con dejar constancia de qué métricas se calculan y con qué papel.

Tabla 8.3 — Métricas utilizadas

Métrica	Unidad de evaluación	Papel
Balanced accuracy	Paciente	Métrica principal
Accuracy	Paciente	Métrica secundaria
Macro-F1	Paciente	Métrica secundaria

8.6 Explicabilidad mediante Grad-CAM

Como complemento a la evaluación cuantitativa, el proyecto incorpora Grad-CAM como técnica de explicabilidad post-hoc ([Selvaraju et al., 2017](#)). El procedimiento se apoya en una función que localiza dinámicamente la última capa convolucional del modelo cargado, en lugar de fijar ese nombre de capa de forma manual y específica para cada arquitectura; en el modelo final utilizado para este análisis, la capa localizada por ese procedimiento es conv5_block3_out.

De forma conceptual, la generación de un mapa Grad-CAM sigue estos pasos: en primer lugar, se selecciona una clase objetivo sobre la que se quiere explicar la predicción del modelo; a continuación, se calcula el gradiente de la puntuación de esa clase respecto a las activaciones de la última capa convolucional localizada; ese gradiente se utiliza para ponderar la importancia de cada mapa de activación de esa capa; los mapas ponderados se combinan y se normalizan en un único mapa de calor; y, por último, ese mapa de calor se superpone sobre la imagen original, resaltando las regiones de la imagen que más contribuyeron a la puntuación de la clase

seleccionada. No es necesario reproducir aquí el código de esta implementación para que el procedimiento quede claro.

Es importante fijar con precisión el papel que Grad-CAM cumple en este trabajo, porque es una técnica fácil de sobreinterpretar. Aquí se emplea exclusivamente como un análisis cualitativo post-hoc: una herramienta que ayuda a observar en qué regiones de la imagen se apoya el modelo al producir una predicción concreta, no como una prueba de localización tumoral, ni como una evaluación de segmentación, ni como una forma de validación clínica, ni como una explicación causal completa del funcionamiento del modelo. En el análisis realizado en este trabajo no se utilizó una máscara espacial de segmentación como referencia (ground truth) para cuantificar el solapamiento entre el mapa de calor y la región tumoral; esta cuestión, incluyendo sus implicaciones para lo que puede y no puede concluirse de los mapas generados, se desarrolla con más detalle en capítulos posteriores. Esta ausencia de comparación cuantitativa contra una máscara en el análisis realizado no debe confundirse con una afirmación sobre la disponibilidad general de anotaciones de segmentación en el dataset empleado.

8.7 Síntesis de la metodología

Este capítulo ha descrito la metodología técnica de Deep Learning empleada en el núcleo experimental del proyecto: un preprocesamiento común de recorte, normalización por percentiles y redimensionado; una comparación de cinco arquitecturas preentrenadas bajo transfer learning, estructurada en una fase frozen y una fase de fine-tuning restringida a los mejores candidatos; una cabeza de clasificación común añadida sobre cada backbone; una configuración de entrenamiento centralizada, con una función de pérdida coherente con el problema y una estrategia deliberada de tasas de aprendizaje distintas entre fases; un mecanismo verificado de congelación de las capas de BatchNormalization durante el fine-tuning; una regla de agregación de predicciones de corte a paciente basada en la media, con una regla de decisión final basada en el argmax; un conjunto de métricas centrado en la balanced accuracy a nivel de paciente; y un análisis de explicabilidad post-hoc mediante Grad-CAM, delimitado explícitamente en su alcance interpretativo.

Ninguno de estos elementos se ha presentado en este capítulo junto con su resultado numérico ni con el desenlace de la comparación de arquitecturas: esta metodología se aplica posteriormente bajo distintos protocolos de evaluación a nivel de paciente, descritos en el capítulo 9, y sus resultados concretos se presentan en el capítulo 10. Lo que este capítulo aporta es la descripción técnica de cómo está

construido, entrenado y evaluado el modelo, como base necesaria para interpretar correctamente los resultados que se presentan más adelante.

9. Diseño experimental y reproducibilidad del protocolo

9.1 Visión general del diseño experimental

El capítulo 6 presentó el split histórico del subconjunto multiclase de Cheng: 163 pacientes de train, 35 de validation y 35 de test, sobre un total de 233 pacientes. Este capítulo no vuelve a explicar esa distribución por clases; se centra en una pregunta distinta, de naturaleza metodológica: cómo se separaron los pacientes entre las distintas etapas del trabajo, cómo se seleccionó el modelo final y cómo se evaluó posteriormente, sin confundir en ningún momento selección, holdout y validación cruzada.

El split histórico cumple dos funciones que conviene distinguir con precisión desde el principio. Por un lado, train y validation se agrupan para formar el conjunto sobre el que se realiza la selección de arquitectura: $163 + 35 = 198$ pacientes. Por otro lado, el conjunto de test permanece apartado como holdout final, reservado para una evaluación posterior. Es fundamental no confundir el validation histórico de 35 pacientes —que forma parte del conjunto de selección— con el test/holdout histórico, también de 35 pacientes, que es un grupo de pacientes distinto y que no interviene en ningún momento de la selección.

Sobre esta base, el diseño experimental completo puede resumirse en cuatro momentos sucesivos. Primero, la selección del modelo se realiza sobre los 198 pacientes de train+validation mediante una validación cruzada agrupada por paciente (sección 9.2). A continuación, una vez fijado el modelo y el protocolo, se evalúa una única vez el holdout de 35 pacientes que había permanecido intacto (sección 9.3). Más adelante, y solo después de esa evaluación, los 233 pacientes del split histórico se reagrupan en un nuevo particionado de cinco folds para estimar la variabilidad interna del modelo ya fijado (sección 9.4). Por último, las secciones 9.5 y 9.6 recogen las métricas y controles comunes al protocolo y las limitaciones que se derivan de este diseño.

Figura 9.1 — Secuencia temporal del protocolo de selección y evaluación

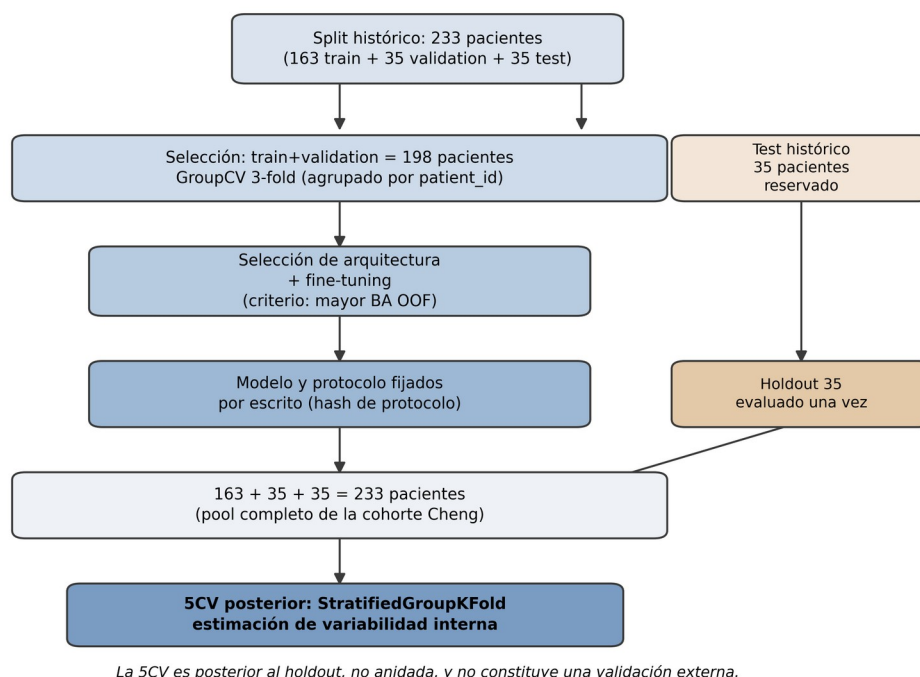


Figura 9.1 — Cronología del protocolo experimental

Split histórico (233 pacientes: 163 train + 35 validation + 35 test) → agrupación de train+validation en el conjunto de selección (198 pacientes, GroupCV 3-fold) mientras el test (35 pacientes) permanece intacto → modelo y protocolo fijados → evaluación del holdout final (35 pacientes) →, posteriormente, reagrupación del pool completo (233 pacientes) en un nuevo particionado de cinco folds (StratifiedGroupKFold) para la estimación de variabilidad interna.

9.2 Selección del modelo mediante validación cruzada a nivel de paciente

La selección de arquitectura se realiza sobre los 198 pacientes que resultan de unir el train histórico (163) y el validation histórico (35). Sobre ese conjunto se aplica una validación cruzada de tres folds agrupada por patient_id (GroupCV 3-fold). Aquí hay que ser precisos con la terminología: lo que está confirmado para esta etapa es un particionado agrupado por paciente en tres folds; no hay evidencia en el código disponible de que se emplease específicamente StratifiedGroupKFold en esta fase, a diferencia de la etapa de validación cruzada de cinco folds descrita en la sección 9.4, donde ese mecanismo sí está verificado de forma explícita. Por ello, esta memoria se refiere a esta etapa como “GroupCV 3-fold agrupado por patient_id” y no atribuye a esta fase un algoritmo de partición más específico del que la evidencia disponible permite afirmar.

La razón para agrupar por paciente, y no simplemente por corte, es metodológica: un mismo paciente puede aportar varios cortes al conjunto de imágenes, y si esos cortes se repartieran libremente entre entrenamiento y validación, un modelo podría evaluarse sobre cortes de un paciente que ya ha contribuido, a través de otros cortes suyos, al entrenamiento de ese mismo fold. La validación cruzada agrupada (grouped cross-validation) obliga a que todos los cortes de un paciente permanezcan en el mismo lado de cada partición ([scikit-learn developers, s. f.](#)). Este mecanismo es uno de los controles de fuga de información implementados en el proyecto; los controles adicionales de deduplicación por SHA-256 y pHash/SSIM ya se describieron en el capítulo 6 y no se repiten aquí.

Dentro de cada uno de los tres folds, cada paciente de los 198 actúa como validación exactamente en el fold en el que no ha participado del entrenamiento, lo que permite obtener una predicción out-of-fold (OOF) para cada uno de ellos. El conjunto de predicciones OOF de los 198 pacientes es lo que permite comparar entre sí a las cinco arquitecturas descritas en el capítulo 8 sin necesidad de tocar el holdout, que permanece completamente al margen de este proceso. Esta memoria no presenta en este capítulo los valores de balanced accuracy OOF obtenidos por cada arquitectura, ni identifica cuál de ellas resultó mejor en esta fase ni en la posterior fase de fine-tuning: esos resultados pertenecen al capítulo 10.

El criterio de selección fijado de antemano es la balanced accuracy OOF a nivel de paciente, calculada sobre los 198 pacientes del conjunto de selección. De acuerdo con ese criterio, el candidato con mayor balanced accuracy OOF pasó a la evaluación final sobre el holdout. Esta formulación es deliberadamente neutra respecto al resultado: el objetivo de este capítulo es dejar establecido el criterio y el procedimiento de selección, no anticipar cuál fue el candidato seleccionado ni su cifra concreta.

9.3 Evaluación sobre el holdout

Los 35 pacientes del test histórico no participaron en ningún momento de la comparación de arquitecturas descrita en la sección anterior, ni en el proceso de fine-tuning empleado para seleccionar el modelo final, ni en ninguna fusión de resultados considerada durante la selección. Permanecieron reservados durante toda esa etapa, y solo después de fijar el modelo, el criterio de selección y el protocolo de evaluación se procedió a evaluar ese conjunto, una única vez.

Antes de abrir el holdout, el protocolo de evaluación final se dejó fijado por escrito en un documento (`protocolo_evaluacion_final.md`) cuya huella quedó registrada. Además, un par de ficheros (`FINAL_TEST_STARTED.lock` y `FINAL_TEST_COMPLETED.lock`) permiten comprobar la consistencia entre los

identificadores de modelo, protocolo y split registrados al inicio y al final de la evaluación. La función de este mecanismo, en el contexto de este capítulo, es metodológica antes que técnica: su propósito es evitar que el procedimiento de evaluación pudiera modificarse retrospectivamente en función del resultado obtenido en el holdout. No conviene sobrestimar lo que estos ficheros demuestran: acreditan que los identificadores registrados al inicio y al final de la ejecución coinciden, pero no prueban que ningún fichero pudiera haber cambiado de forma transitoria durante el proceso. El funcionamiento interno de este mecanismo ya se describió con más detalle en el capítulo 7 y no se repite aquí.

La apertura documentada del holdout se produjo en una única ejecución (job 70930), dato que se menciona aquí únicamente como referencia de trazabilidad; los recursos y la cronología de ejecución en el clúster HPC se tratan en el capítulo 12 y no se desarrollan en este punto.

El protocolo de evaluación del holdout incorpora, además del cálculo directo de las métricas, un procedimiento de bootstrap de 2.000 réplicas para estimar un intervalo de confianza de la balanced accuracy (Efron, 1979). Su propósito es aportar una estimación de la incertidumbre alrededor del resultado puntual obtenido en el holdout, dado que este se calcula sobre un número relativamente reducido de pacientes. Esta memoria no presenta en este capítulo ni el valor de balanced accuracy obtenido ni el intervalo de confianza resultante: ambos se presentan en el capítulo 10, junto con el resto de resultados del holdout.

9.4 Validación cruzada de cinco folds para estimar variabilidad interna

Una vez evaluado el holdout, y con el modelo y el protocolo ya fijados, el proyecto incorpora una validación cruzada de cinco folds cuyo objetivo es observar la variabilidad interna del rendimiento de ese modelo ante distintos particionados de la misma cohorte. Es importante situar correctamente esta etapa en el tiempo: la validación cruzada de cinco folds se ejecuta después de la evaluación del holdout, no antes ni en paralelo a la selección de arquitectura.

Para esta etapa, los tres subconjuntos del split histórico —163 de train, 35 de validation y 35 de test— se reúnen en un pool único de 233 pacientes. El split histórico 163/35/35 no se modifica como artefacto: su huella (`split_hash`) se conserva exactamente igual que en capítulos anteriores. Pero ese split histórico no se reutiliza como particionado de evaluación en esta etapa. En su lugar, se genera un nuevo particionado de cinco folds sobre el pool completo de 233 pacientes, identificado mediante un campo propio (`folds_master_sha256`), distinto del `split_hash` histórico; cada uno de los cinco resultados de fold conserva además su propio campo de huella

(fold_json_sha256). Los valores hexadecimales completos de estos campos no se reproducen en el cuerpo del capítulo y pueden consultarse en el documento de anexos técnicos; lo relevante aquí es no confundir estos tres campos entre sí, ni confundir el particionado 198/35 de la selección con el particionado en cinco folds sobre 233 pacientes de esta etapa.

El particionado de esta etapa se construye explícitamente como:

```
StratifiedGroupKFold(  
    n_splits=5,  
    shuffle=True,  
    random_state=20260904  
)
```

agrupando por `patient_id`. Los tres componentes de este mecanismo cumplen funciones distintas: la agrupación (`group`) mantiene íntegros los cortes de cada paciente entre entrenamiento y validación, de la misma forma que en la selección de la sección 9.2; la estratificación (`stratified`) busca una distribución de clases razonablemente equilibrada entre los cinco folds, sin que ello implique que los cinco tengan una composición idéntica entre sí; y el barajado (`shuffle`) reordena los grupos de pacientes antes de construir el particionado, de forma reproducible gracias al valor fijado de `random_state`.

Cada uno de los cinco folds se entrena con una instancia fresca del modelo, inicializada de nuevo a partir de los pesos preentrenados sobre ImageNet, sin reutilizar secuencialmente los pesos ajustados en el fold anterior. Cada instancia utiliza además una semilla de entrenamiento propia: 1001, 1002, 1003, 1004 y 1005, derivadas mediante la fórmula $1000 + \text{número de fold}$. Esta semilla, que controla la inicialización aleatoria de cada ejecución de entrenamiento, no debe confundirse con el `random_state` de valor 20260904 empleado para construir el particionado de los cinco folds: son dos mecanismos de aleatoriedad distintos, con funciones distintas —uno decide cómo se reparten los pacientes entre folds, el otro controla la aleatoriedad interna de cada entrenamiento.

El protocolo de esta etapa declara explícitamente que no realiza selección de arquitectura ni optimización de hiperparámetros; el script que ejecuta cada fold solo recibe como argumento de línea de comandos relevante el índice del fold a ejecutar (`--fold-index`). El modelo y el protocolo llegan ya fijados desde las secciones anteriores, y esta etapa no reabre en ningún momento el proceso de selección.

Tampoco existe en esta etapa una noción de “mejor fold”: el script encargado de agregar los cinco resultados se limita a incorporarlos y a verificar su integridad, sin descartar ninguno de ellos ni ponderar unos folds más que otros en función de su

rendimiento individual. Esta decisión evita que, a posteriori, se seleccione informalmente la partición más favorable como si fuera representativa del comportamiento del modelo.

Dentro de cada fold, el procedimiento consiste en completar el entrenamiento definido, guardar el modelo resultante y, a continuación, evaluarlo sobre la partición de validación de ese fold; no se emplea EarlyStopping ni un mecanismo de selección del mejor checkpoint según una métrica de validación dentro de esta etapa. Esta memoria describe el protocolo tal como está implementado, sin presentarlo como la única alternativa metodológicamente válida.

9.5 Métricas y controles del protocolo

Las métricas empleadas en la evaluación del holdout y en la validación cruzada de cinco folds son las mismas tres descritas en el capítulo 8: balanced accuracy, accuracy y macro-F1, fijadas de antemano y calculadas de forma idéntica en ambas etapas. La balanced accuracy continúa siendo la métrica principal del trabajo; accuracy y macro-F1 se registran como métricas secundarias. Este capítulo no repite la formulación matemática de estas métricas, ya presentada en el capítulo 8, ni los valores obtenidos en ninguna de las dos evaluaciones, que se presentan en el capítulo 10.

Como ya se explicó en el capítulo 8, todas estas métricas se calculan a nivel de paciente: las predicciones de corte se agregan previamente mediante la media aritmética antes de aplicar cualquier métrica. Esa ecuación no se vuelve a desarrollar en este capítulo.

Tabla 9.1 — Etapas del protocolo

Etapa	Pacientes utilizados	Particionado	Objetivo
Split histórico	233	163/35/35	Organización inicial del subconjunto multiclase
Selección	198	GroupCV 3-fold agrupado por paciente	Comparación y selección de arquitectura
Holdout final	35	Conjunto reservado, evaluado una única vez	Evaluación posterior a la selección

Etapa	Pacientes utilizados	Particionado	Objetivo
Robustez interna	233	StratifiedGroupK Fold 5-fold	Estimación de la variabilidad interna entre particiones

Tabla 9.2 — Controles del diseño experimental

Control	Etapa	Finalidad
Agrupación por patient_id	Selección + 5CV	Mantener los cortes de un mismo paciente en un único lado de cada partición
Holdout reservado	Selección	Evitar que el conjunto de test intervenga en la selección del modelo
Protocolo fijado antes del holdout	Holdout	Registrar la decisión de evaluación antes de conocer el resultado
Nueva instancia por fold	5CV	Evitar el arrastre de pesos entre folds sucesivos
Modelo y protocolo fijos	5CV	No reabrir la selección de arquitectura dentro de esta etapa
Cobertura OOF del pool	5CV	Comprobar la cobertura de los 233 pacientes
Campos de huella del particionado	5CV	Trazabilidad e integridad del nuevo particionado

Estos controles abordan de forma explícita determinadas formas de riesgo de fuga de información y de selección post-hoc, pero no constituyen una demostración de ausencia total de fuga en todos los sentidos posibles del término.

El pipeline verifica mediante una comprobación automática (`assert`) que los cinco folds de la validación cruzada cubren los 233 pacientes del pool sin omisiones. Al tratarse de un esquema de validación cruzada de cinco folds, cada paciente pertenece a la partición de validación de uno de los folds y contribuye así a la

agregación OOF. Esta comprobación es un invariante de integridad del procedimiento y no un resultado predictivo del modelo.

9.6 Alcance y limitaciones del diseño experimental

Es necesario dejar establecidas con claridad las limitaciones que se derivan de este diseño, algunas de las cuales se retomarán con más profundidad en los capítulos 11 y 13.

La validación cruzada de cinco folds descrita en la sección 9.4 se realiza sobre la misma cohorte global de 233 pacientes de Cheng utilizada en las etapas de selección y evaluación multiclase. Por esta razón, esta etapa no constituye una validación externa: no introduce pacientes, hospitales ni fuentes de adquisición distintas de las ya empleadas en las fases anteriores.

Además, los 198 pacientes que se utilizaron inicialmente para seleccionar el modelo forman parte, posteriormente, del pool de 233 pacientes sobre el que se ejecuta la validación cruzada de cinco folds; los 35 pacientes restantes de ese pool son los que habían actuado como holdout histórico. En consecuencia, la validación cruzada de cinco folds no debe describirse como una evaluación completamente independiente del proceso de desarrollo y selección que la precedió.

Tampoco el procedimiento completo constituye una validación cruzada anidada (nested cross-validation). En una validación cruzada anidada (nested cross-validation), el proceso de selección de modelo o de hiperparámetros se repetiría dentro de cada partición externa de la evaluación ([Cawley & Talbot, 2010](#)). Eso no ocurre en este proyecto: la arquitectura se seleccionó primero, sobre los 198 pacientes descritos en la sección 9.2, y solo después se ejecutó la validación cruzada de cinco folds con el modelo y el protocolo ya fijados; durante cada uno de los cinco folds no se vuelve a ejecutar el proceso de selección de arquitectura.

Por último, el protocolo final documentado no incluye una cohorte externa independiente, distinta de Cheng, sobre la que evaluar el modelo. Esta ausencia de validación externa, junto con el hecho de que la 5CV posterior reutilice tanto los 198 pacientes empleados durante la selección como los 35 pacientes que habían constituido el holdout, condiciona el alcance de las conclusiones que pueden extraerse del rendimiento observado, y se retoma con más detalle en la discusión de limitaciones del trabajo.

9.7 Síntesis del protocolo

La secuencia completa del diseño experimental puede resumirse en cinco pasos. Primero, se realiza la selección de arquitectura mediante GroupCV 3-fold sobre los 198 pacientes de train y validation histórico. Segundo, una vez fijados el modelo, el

criterio de selección y el protocolo de evaluación, se abre y evalúa el holdout de 35 pacientes que había permanecido reservado. Tercero, ese resultado constituye la evaluación de holdout del protocolo pre-5CV, obtenida antes de introducir ninguna validación cruzada adicional. Cuarto, posteriormente se realiza la validación cruzada de cinco folds sobre el pool completo de 233 pacientes, con el modelo y el protocolo ya fijados. Quinto, esta validación cruzada de cinco folds complementa al resultado del holdout aportando una estimación de la variabilidad interna del modelo ante distintos particionados de la misma cohorte, sin sustituir ni invalidar retroactivamente el resultado obtenido en el holdout.

Este capítulo ha descrito exclusivamente el diseño de este protocolo: qué pacientes participan en cada etapa, bajo qué particionado y con qué controles. Ninguno de los valores de rendimiento asociados a estas etapas se ha presentado aquí; todos ellos, incluidos los resultados del holdout, de la validación cruzada de cinco folds y de la comparación de arquitecturas, se presentan en el capítulo 10.

10. Resultados

10.1 Resultados históricos y controles metodológicos

Antes de presentar los resultados del núcleo experimental multiclase, resulta útil recoger de forma ordenada los resultados de las fases históricas y de los controles metodológicos que se fueron generando a lo largo del proyecto, ya que varios de ellos condicionan cómo deben leerse los resultados posteriores.

El baseline histórico multiclase —una CNN propia entrenada desde cero, sin transfer learning, descrita en el capítulo 8— obtuvo una balanced accuracy a nivel de paciente del 79,66 %. Una reproducción posterior de ese mismo baseline, ejecutada sobre el mismo split y la misma configuración declarada, registró una balanced accuracy del 74,44 %, es decir, 5,21 puntos porcentuales por debajo del valor histórico. La causa exacta de esta diferencia no pudo establecerse a partir de los artefactos disponibles: los datos permiten constatar la magnitud de la diferencia, pero no permiten atribuirla de forma demostrada a un factor concreto.

En la tarea binaria histórica (tumor/no-tumor), abandonada como núcleo del trabajo en favor de la clasificación multiclase sobre Cheng, el modelo obtuvo una balanced accuracy a nivel de paciente del 98,57 %. Este resultado no puede leerse de forma aislada: el elevado rendimiento del modelo binario coexistió con un rendimiento también muy elevado del clasificador de procedencia basado en descriptores de bajo nivel —el mismo control de sesgo presentado en el capítulo 6—, que alcanzó una accuracy del 98,16 % y un AUC del 99,55 % sobre el mismo split de evaluación.

En el control equivalente aplicado a la tarea multiclase sobre la cohorte Cheng, el clasificador de bajo nivel alcanzó una accuracy del 69,22 %. Este resultado muestra que las características de bajo nivel utilizadas por el control contienen capacidad discriminativa parcial dentro de la cohorte. No se atribuye aquí esa capacidad discriminativa a ningún factor concreto (hospital, escáner, protocolo de adquisición u otro), cuestión que se retoma en el capítulo 11.

Tabla 10.1 — Resultados históricos y controles

Experimento	Métrica	Resultado	Papel
Baseline multiclase histórico	BA paciente	79,66 %	Referencia histórica
Reproducción baseline	BA paciente	74,44 %	Comprobación de regresión
Modelo binario	BA paciente	98,57 %	Resultado

Experimento	Métrica	Resultado	Papel
histórico			histórico
Shortcut binario	Accuracy	98,16 %	Control de procedencia
Shortcut binario	AUC	99,55 %	Control de procedencia
Control bajo nivel multiclase	Accuracy	69,22 %	Control metodológico

La tabla 10.1 reúne estos cinco resultados y controles en un mismo lugar, como punto de referencia antes de entrar en los resultados del núcleo experimental multiclase, que ocupan el resto de este capítulo.

10.2 Selección de arquitectura

Esta sección presenta los resultados obtenidos durante la selección de arquitectura descrita metodológicamente en el capítulo 9: qué balanced accuracy OOF obtuvo cada arquitectura en la fase frozen, qué ocurrió tras el fine-tuning de los dos mejores candidatos, y qué modelo resultó seleccionado de acuerdo con el criterio fijado de antemano. El protocolo en sí —GroupCV 3-fold sobre 198 pacientes, agrupación por paciente, cálculo de las predicciones OOF— no se vuelve a desarrollar aquí; se remite al capítulo 9 para esos detalles.

Sobre los 198 pacientes del conjunto de selección, las cinco arquitecturas evaluadas con su base preentrenada congelada obtuvieron los siguientes valores de balanced accuracy OOF a nivel de paciente: VGG16, 78,94 %; ResNet50, 83,13 %; MobileNetV2, 81,53 %; EfficientNetB0, 78,72 %; InceptionV3, 82,06 %. Estas cinco cifras se sitúan en un rango de 78,72 % a 83,13 %.

Figura 10.1 — Balanced accuracy OOF durante la selección de arquitectura

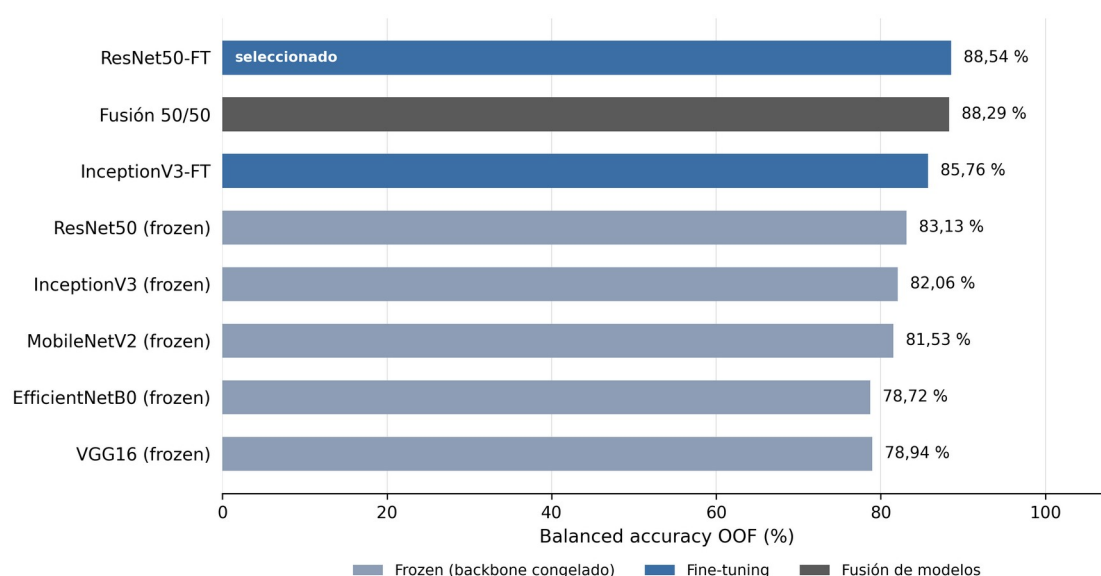


Figura 10.1 — Balanced accuracy OOF durante la selección de arquitectura

Los ocho puntos evaluados en esta sección (VGG16 frozen, ResNet50 frozen, MobileNetV2 frozen, EfficientNetB0 frozen, InceptionV3 frozen, ResNet50-FT, InceptionV3-FT, Fusión 50/50), en balanced accuracy OOF, ordenados de mayor a menor valor.

De acuerdo con el protocolo frozen → fine-tuning descrito en el capítulo 8, los dos candidatos con mejor balanced accuracy OOF en la fase frozen —ResNet50 y InceptionV3— pasaron a la fase de fine-tuning. El ajuste de las últimas 20 capas del backbone, con las capas BatchNormalization congeladas conforme a lo descrito en el capítulo 8, registró una balanced accuracy OOF más alta que la de la fase frozen correspondiente en ambos casos: ResNet50 pasó de 83,13 % (frozen) a 88,54 % (fine-tuning), e InceptionV3 pasó de 82,06 % (frozen) a 85,76 % (fine-tuning). Se evaluó además una fusión 50/50 de ambos candidatos ya ajustados, que obtuvo una balanced accuracy OOF del 88,29 %.

El orden observado entre estos tres resultados finales fue: ResNet50-FT (88,54 %) > Fusión (88,29 %) > InceptionV3-FT (85,76 %). El criterio de selección documentado fue la mayor balanced accuracy OOF a nivel de paciente entre los candidatos considerados. Aplicando ese criterio, ResNet50-FT fue el candidato seleccionado dentro de los experimentos realizados y pasó a la evaluación sobre el holdout final descrita en la sección 10.3.

Tabla 10.2 — Selección de arquitectura

Modelo	Etapas	BA OOF
VGG16	Frozen	78,94 %
ResNet50	Frozen	83,13 %
MobileNetV2	Frozen	81,53 %
EfficientNetB0	Frozen	78,72 %
InceptionV3	Frozen	82,06 %
ResNet50-FT	Fine-tuning — seleccionado	88,54 %
InceptionV3	Fine-tuning	85,76 %
Fusión ResNet50-FT + InceptionV3-FT	50/50	88,29 %

La tabla 10.2 recoge los ocho resultados de esta sección en un único lugar, marcando el modelo finalmente seleccionado sin presentarlo como una arquitectura universalmente superior, sino como el candidato con mayor balanced accuracy OOF dentro de los experimentos realizados en este trabajo.

10.3 Evaluación sobre el holdout final

Sobre los 35 pacientes del holdout —el conjunto reservado descrito en el capítulo 9, evaluado una única vez tras fijar el modelo y el protocolo—, ResNet50-FT obtuvo una balanced accuracy a nivel de paciente del 93,89 %, una accuracy del 94,29 % y un macro-F1 del 93,98 %. Las tres métricas, como el resto de las presentadas en este capítulo, se calculan sobre las predicciones agregadas por paciente, no sobre los cortes individuales.

Por clase, los recalls a nivel de paciente en el holdout fueron: glioma, 91,67 %; meningioma, 90,00 %; pituitario, 100,00 %. En el holdout, la clase pituitaria no presentó errores a nivel de paciente, mientras que glioma y meningioma registraron un error cada una.

		Predicción		
		Glioma	Meningioma	Pituitario
Clase real	Glioma	11	1	0
	Meningioma	0	9	1
	Pituitario	0	0	13

Figura 10.2 — Matriz de confusión del holdout final

Matriz 3×3 (orden glioma/meningioma/pituitario) con los valores $[[11,1,0],[0,9,1],[0,0,13]]$, 33 de 35 pacientes correctamente clasificados.

El intervalo bootstrap del 95 % para la balanced accuracy, calculado sobre 2.000 réplicas según el protocolo descrito en el capítulo 9, fue [84,26 %, 100 %]. El intervalo obtenido presenta una amplitud considerable. Su interpretación se desarrolla en el capítulo 11, y no debe leerse este intervalo en términos de una probabilidad de que el modelo acierte en un paciente futuro.

Se identificaron dos pacientes incorrectamente clasificados en el holdout, los únicos dos de los 35. El primero, CHENG-100572, tenía como clase real meningioma y fue predicho como pituitario, con una confianza de 0,820851 sobre 8 cortes disponibles para ese paciente. El segundo, CHENG-108479, tenía como clase real glioma y fue predicho como meningioma, con una confianza de 0,458572 sobre 2 cortes disponibles. Ambos casos se retoman de forma cualitativa en la sección 10.4, dentro del análisis Grad-CAM.

Tabla 10.3 — Holdout final

Métrica	Resultado
Balanced accuracy	93,89 %
Accuracy	94,29 %
Macro-F1	93,98 %

Métrica	Resultado
Recall glioma	91,67 %
Recall meningioma	90,00 %
Recall pituitario	100,00 %
Pacientes correctos	33/35
IC95 % bootstrap BA	[84,26 %, 100 %]

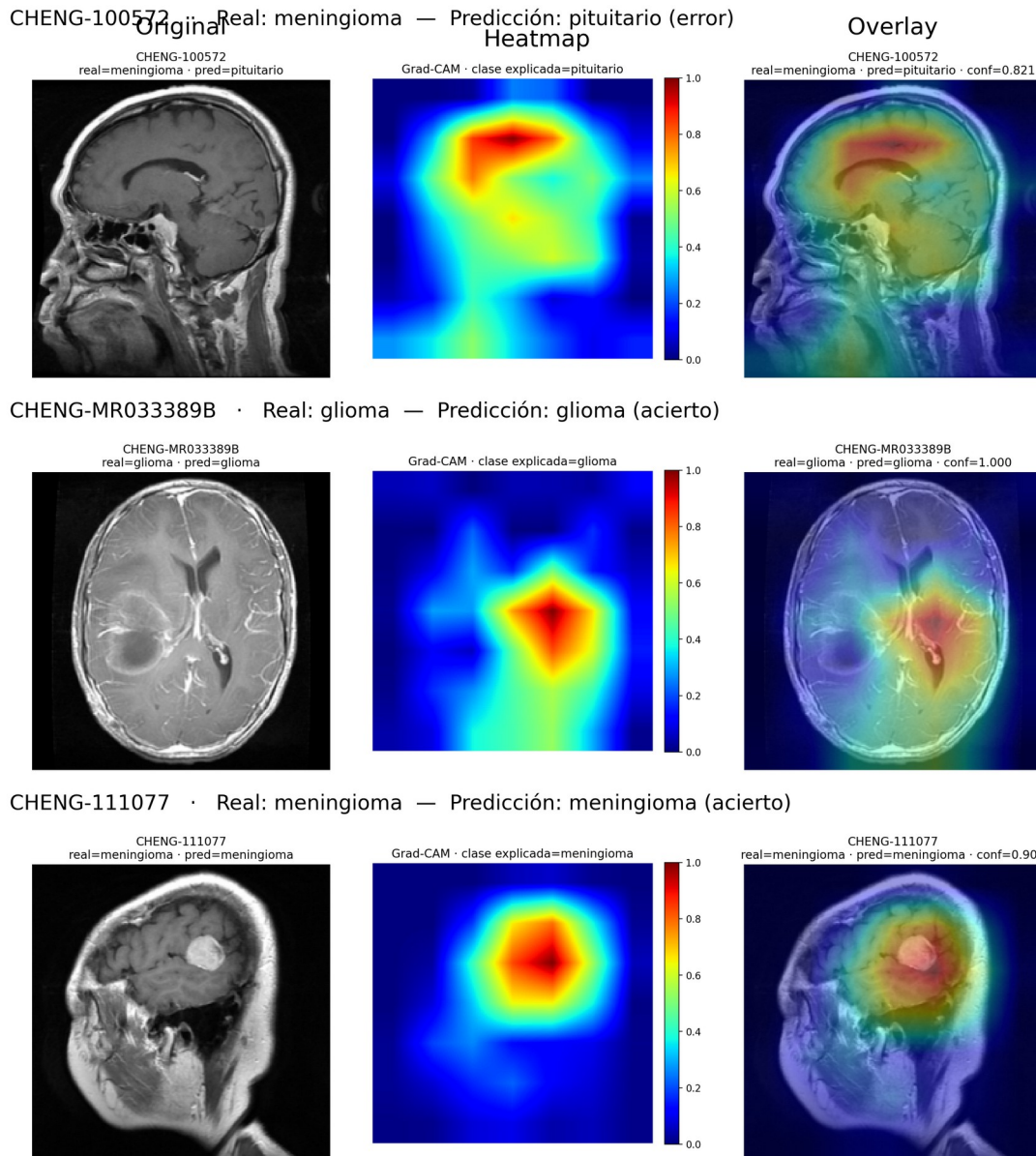
10.4 Análisis cualitativo mediante Grad-CAM

Como complemento cualitativo a las cifras anteriores, se generaron 15 imágenes Grad-CAM, organizadas en 5 casos de 3 imágenes cada uno (original, heatmap y overlay). De los cinco casos inspeccionados, tres corresponden a predicciones correctas y dos a los errores ya presentados en la sección 10.3. El procedimiento de generación de estos mapas ya se describió en el capítulo 8 y no se repite aquí.

Los cinco casos inspeccionados son: CHENG-100572 (error, meningioma → pituitario), CHENG-108479 (error, glioma → meningioma), CHENG-MR033389B (correcto, glioma), CHENG-111077 (correcto, meningioma) y CHENG-105936 (correcto, pituitario). Los dos primeros coinciden exactamente con los dos errores del holdout descritos en la sección 10.3.

La inspección visual de estos cinco casos, descrita en términos exclusivamente posicionales y sin atribuir a las regiones observadas ninguna correspondencia anatómica no verificada, arrojó las siguientes observaciones. En CHENG-100572, el mapa de calor se concentra en la parte superior de la imagen, ligeramente desplazado respecto al eje central, con un núcleo de mayor intensidad rodeado de un halo. En CHENG-108479, la activación aparece en la parte superior, próxima al borde de la imagen, con una intensidad relativamente menor que en el caso anterior. En CHENG-MR033389B, la imagen original muestra una región redondeada más oscura en la zona inferior izquierda, mientras que el mapa de calor se concentra en una región central-inferior próxima al eje, sin coincidencia visual clara con esa región más oscura. En CHENG-111077, la imagen original muestra una región redondeada más clara en la parte superior, y el mapa de calor se concentra también en la zona superior, con coincidencia posicional general. En CHENG-105936, no se distingue con claridad una estructura interna concreta en la imagen original —se aprecia una zona clara del borde inferior—, y el mapa de calor muestra activación bilateral en la región central-inferior.

Figura 10.3 — Grad-CAM de tres casos representativos del holdout



Selección cualitativa; los mapas no constituyen una validación de localización tumoral ni una segmentación.

Figura 10.3 — Grad-CAM de casos representativos

Selección editorial de tres de los cinco casos analizados —CHENG-100572 (error, con activación superior), CHENG-MR033389B (acierto, sin coincidencia visual evidente) y CHENG-111077 (acierto, con coincidencia posicional general)—, mostrando en cada uno la imagen original, el heatmap y el overlay. Esta selección de tres casos es una decisión editorial, no una selección estadística; la galería completa de los 15 PNG se reserva para el Anexo E del documento de anexos técnicos.

Como ya se estableció en el capítulo 8, este análisis se emplea exclusivamente como una herramienta cualitativa post-hoc: no cuantifica el solapamiento entre el

mapa de calor y ninguna región anatómica, no constituye una prueba de localización tumoral y no equivale a una evaluación de segmentación. En este análisis no se utilizó ninguna máscara espacial como referencia para cuantificar el solapamiento de los mapas Grad-CAM. Esta ausencia de comparación cuantitativa no implica una afirmación sobre la disponibilidad de anotaciones en la fuente original.

10.5 Validación cruzada de cinco folds

Esta sección presenta los resultados de la validación cruzada de cinco folds descrita metodológicamente en el capítulo 9, sobre el pool de 233 pacientes que reúne los tres subconjuntos del split histórico. Se presentan primero los cinco folds por separado, después la media y la desviación entre ellos, y por último el resultado calculado una única vez sobre las 233 predicciones OOF reunidas.

Los cinco folds obtuvieron los siguientes resultados a nivel de paciente. Fold 1 (47 pacientes): balanced accuracy 92,42 %, accuracy 89,36 %, macro-F1 90,11 %. Fold 2 (47 pacientes): balanced accuracy 82,30 %, accuracy 80,85 %, macro-F1 80,32 %. Fold 3 (46 pacientes): balanced accuracy 94,34 %, accuracy 93,48 %, macro-F1 94,29 %. Fold 4 (47 pacientes): balanced accuracy 90,07 %, accuracy 89,36 %, macro-F1 89,43 %. Fold 5 (46 pacientes): balanced accuracy 88,07 %, accuracy 86,96 %, macro-F1 87,07 %. Fold 3 presentó la mayor balanced accuracy observada entre estas cinco particiones evaluadas; Fold 2 presentó la menor.

Tabla 10.4 — Resultados por fold

Fold	Pacientes	BA	Accuracy	Macro-F1
1	47	92,42 %	89,36 %	90,11 %
2	47	82,30 %	80,85 %	80,32 %
3	46	94,34 %	93,48 %	94,29 %
4	47	90,07 %	89,36 %	89,43 %
5	46	88,07 %	86,96 %	87,07 %

La media y la desviación estándar muestral de estas cinco balanced accuracy son 89,44 % $\pm$ 4,64 pp; la media de accuracy es 88,00 % $\pm$ 4,63 pp; y la media de macro-F1 es 88,24 % $\pm$ 5,14 pp. Esta desviación estándar describe únicamente la dispersión entre los cinco valores obtenidos y no debe interpretarse como un error estándar ni como un intervalo de confianza.

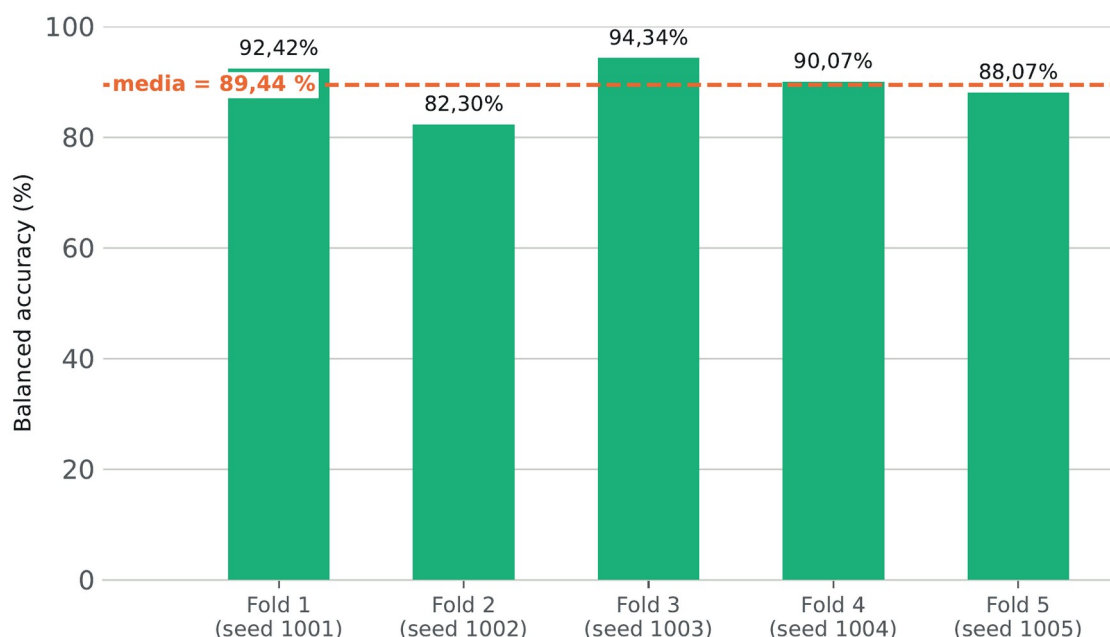


Figura 10.4 — Balanced accuracy por fold en la 5CV

Cinco puntos (92,42 %, 82,30 %, 94,34 %, 90,07 %, 88,07 %) con una línea horizontal en la media (89,44 %) y una banda de $\pm$ desviación estándar entre los cinco valores.

Junto a la media de las cinco métricas de fold, el pipeline calcula también un resultado agregado sobre las 233 predicciones OOF reunidas en un único conjunto: una balanced accuracy del 88,68 %, una accuracy del 87,98 % y un macro-F1 del 88,21 %. Esta cifra agregada de balanced accuracy (88,68 %) difiere de la media de las cinco balanced accuracy de fold (89,44 %) porque son dos formas distintas de agregar el mismo conjunto de resultados: la media de folds promedia cinco métricas ya calculadas por separado, mientras que la métrica OOF se calcula una única vez sobre las 233 predicciones reunidas, sin pasar primero por un promedio de métricas por fold. La consistencia de esta segunda vía de cálculo se ha comprobado adicionalmente verificando que la matriz de confusión agregada sobre los 233 pacientes coincide, celda a celda, con la suma de las cinco matrices de confusión de cada fold. La diferencia entre ambas cifras no se atribuye a ningún error de cálculo.

Los recalls por clase sobre las 233 predicciones OOF agregadas fueron: glioma, 88,76 %; meningioma, 80,49 %; pituitario, 96,77 %. La clase meningioma presenta el menor recall agregado de las tres.

Tabla 10.5 — Resultados OOF agregados (233 pacientes)

Métrica	Resultado
Balanced accuracy OOF	88,68 %

Métrica	Resultado
Accuracy OOF	87,98 %
Macro-F1 OOF	88,21 %
Recall OOF glioma	88,76 %
Recall OOF meningioma	80,49 %
Recall OOF pituitario	96,77 %
Diagonal de la matriz agregada	205/233

La matriz de confusión agregada sobre las 233 predicciones OOF es $[[79,9,1], [9,66,7],[1,1,60]]$ (orden glioma/meningioma/pituitario), con una diagonal de 205 de 233 pacientes.

		Predicción		
		Glioma	Meningioma	Pituitario
Clase real	Glioma	79	9	1
	Meningioma	9	66	7
	Pituitario	1	1	60

Figura 10.5 — Matriz de confusión agregada OOF de la 5CV

Matriz 3×3 con los valores agregados de los 233 pacientes OOF, mismo orden de clases que la Figura 10.2.

El fold con menor balanced accuracy observada, Fold 2, tuvo una composición de 17 pacientes de glioma, 18 de meningioma y 12 de pituitario en su partición de test, y una matriz de confusión de $[[16,1,0],[2,11,5],[0,1,11]]$, con una diagonal de 38 de 47 pacientes. Los recalls por clase en este fold fueron: glioma, 94,12 %; meningioma, 61,11 %; pituitario, 91,67 %. El patrón más visible en esta matriz es que 5 de los pacientes de meningioma de este fold fueron clasificados como pituitario.

Los historiales de entrenamiento exportados del Fold 2 no muestran una anomalía evidente durante el entrenamiento: las pérdidas disminuyen y las accuracies aumentan de forma monótona en ambas fases (cabeza y fine-tuning), sin valores no numéricos ni estancamientos visibles en los arrays disponibles. No se dispone de evidencia suficiente para atribuir el menor rendimiento observado en este fold a una causa concreta.

10.6 Síntesis de resultados

Este capítulo ha presentado, sección por sección, los resultados obtenidos en cada etapa del desarrollo experimental, sin desarrollar todavía su interpretación.

En los resultados históricos y de control (8.1), el baseline multiclase obtuvo una balanced accuracy del 79,66 %, con una reproducción posterior 5,21 puntos porcentuales por debajo, de causa no establecida; el modelo binario histórico obtuvo un 98,57 % de balanced accuracy, en coexistencia con un control de procedencia también muy elevado (98,16 % de accuracy); y el control de bajo nivel multiclase alcanzó un 69,22 % de accuracy.

En la selección de arquitectura (8.2), ResNet50-FT fue el candidato seleccionado de acuerdo con el criterio de mayor balanced accuracy OOF fijado de antemano, con un 88,54 % frente al 88,29 % de la fusión y al 85,76 % de InceptionV3-FT, tras un fine-tuning que mejoró el resultado frozen de ambos candidatos considerados. Sobre el holdout final (8.3), ResNet50-FT obtuvo una balanced accuracy del 93,89 %, con dos errores de 35 pacientes, cada uno descrito con su confianza y su número de cortes, y un intervalo bootstrap del 95 % de [84,26 %, 100 %]. El análisis Grad-CAM (8.4), realizado sobre esos mismos dos casos de error y tres casos de acierto, aportó una descripción cualitativa post-hoc de las regiones de activación del modelo, sin constituir en ningún caso una prueba de localización tumoral.

La validación cruzada de cinco folds (8.5) mostró una variabilidad observada de 82,30 % a 94,34 % de balanced accuracy entre las cinco particiones evaluadas, con una media de 89,44 % $\pm$ 4,64 pp y una cifra agregada OOF sobre los 233 pacientes de 88,68 %, ligeramente distinta de esa media por tratarse de dos formas de agregación distintas. Fold 2 registró la menor balanced accuracy de las cinco particiones evaluadas, con un recall de meningioma notablemente inferior al de los demás folds, sin que se haya podido atribuir esta observación a una causa concreta con la evidencia disponible.

Ninguno de estos resultados se ha interpretado aquí más allá de su descripción directa. La discusión de sus implicaciones, de sus posibles causas y de su comparación con la literatura revisada en el capítulo 3 se desarrolla en el capítulo 11.

11. Discusión

11.1 Lectura global

Los resultados presentados en el capítulo anterior describen un recorrido que parte de una tarea binaria abandonada como núcleo del trabajo, pasa por un baseline histórico multiclase y su reproducción, continúa con la selección de una arquitectura mediante comparación frozen y fine-tuning, y culmina en dos evaluaciones complementarias del modelo seleccionado: un holdout único y una validación cruzada de cinco particiones (5CV). Este capítulo no repite esas cifras como resultado, sino que discute qué puede y qué no puede inferirse de ellas.

Antes de entrar en el detalle, hay que fijar una distinción que se mantiene a lo largo de todo el capítulo: un mismo hallazgo experimental admite lecturas de distinto estatus epistémico. Algunas afirmaciones son hechos observados directamente en los artefactos (una cifra, una matriz de confusión, la existencia de un patrón de error concreto). Otras son interpretaciones razonables de esos hechos, consistentes con la evidencia disponible pero no las únicas posibles. Otras son hipótesis que no se han puesto a prueba dentro de este trabajo y que, por tanto, no deben presentarse como conclusiones. Y otras, finalmente, son limitaciones del propio diseño experimental que acotan el alcance de cualquier lectura. Mantener esta distinción explícita evita dos errores simétricos: sobreinterpretar un resultado favorable como si demostrara más de lo que demuestra, y descartar un resultado por una causa que no se ha comprobado.

El conjunto de resultados es, en su lectura más directa, consistente con un pipeline que separa correctamente por paciente, que documenta sus decisiones de diseño y que no oculta la variabilidad observada entre particiones. No es, sin embargo, una demostración de generalización a poblaciones o centros distintos de los representados en la cohorte utilizada, ni una validación clínica de ningún tipo. Ambas afirmaciones —lo que el trabajo sí respalda y lo que no— se desarrollan con más detalle en las secciones siguientes.

11.2 Del problema binario al núcleo multiclase

El resultado binario (98,57 % de balanced accuracy a nivel de paciente) fue, en su momento, el primer resultado cuantitativo del proyecto. Sin embargo, no constituye evidencia central de detección de tumor. La razón no es que la cifra sea incorrecta: es que el mismo split sobre el que se obtuvo permite a un clasificador basado únicamente en descriptores de bajo nivel de la imagen, sin utilizar las representaciones aprendidas por la CNN, alcanzar una accuracy del 98,16 %. Cuando un control basado únicamente en características de bajo nivel se aproxima tanto al resultado del modelo

entrenado, la cifra del modelo deja de poder interpretarse, por sí sola, como prueba de que la red aprendió a distinguir tumor de ausencia de tumor.

Esta observación exige una distinción que conviene dejar explícita, porque es fácil confundirla: el sesgo de procedencia detectado en la tarea binaria no es, por sí mismo, una fuga de información en el sentido de contaminación entre conjuntos de entrenamiento y evaluación. Los controles de separación por paciente implementados en este trabajo son independientes de este hallazgo y no quedan invalidados por él: puede coexistir una separación por paciente correcta con un atajo de bajo nivel explotable dentro del propio conjunto, si ese atajo está correlacionado con la etiqueta de forma sistemática en el origen de los datos. Tampoco es correcto extrapolar el hallazgo a la afirmación de que “la red solo aprendió el origen de las imágenes”: el control demuestra que existe una señal de bajo nivel fuertemente asociada con la etiqueta y suficiente para separar las dos clases con un rendimiento muy elevado, no que la totalidad de lo aprendido por la red se reduzca a esa señal. Lo que sí puede afirmarse, con el nivel de certeza que da la evidencia disponible, es que la cifra de 98,57 % no es interpretable como una medida aislada de capacidad diagnóstica, y que este hallazgo fue la motivación directa para reorientar el núcleo del trabajo hacia la tarea multiclase sobre la cohorte Cheng, donde no se ha detectado un atajo de magnitud comparable.

Esa reorientación no es meramente prudencial: cambia la pregunta de investigación. La tarea binaria, tal como estaba planteada, mezclaba una fuente de datos para los casos positivos y otra distinta para los negativos, lo que hacía estructuralmente difícil separar la señal tumoral del origen de la imagen. La tarea multiclase, en cambio, compara tres tipos de tumor dentro de una única cohorte (Cheng), lo que reduce —sin eliminar necesariamente— el riesgo de que la procedencia de la imagen actúe como variable de confusión entre clases.

11.3 Selección del modelo y efecto observado del fine-tuning

Antes de la selección de arquitectura propiamente dicha, el proyecto cuenta con un baseline histórico multiclase (79,66 % de balanced accuracy) y una reproducción posterior del mismo experimento (74,44 %). La diferencia entre ambos (−5,21 puntos porcentuales) queda documentada, pero su causa no está confirmada. El proyecto no activó de forma explícita el determinismo estricto de GPU en esta fase, lo cual se plantea como una posible fuente de variabilidad entre ejecuciones nominalmente idénticas, nunca como la causa demostrada de la diferencia observada: no se ha aislado el determinismo de GPU de otros factores posibles (versión de

librerías, orden de lectura de datos, u otros), por lo que atribuirle la diferencia completa sería una inferencia no respaldada por lo verificado.

La selección de arquitectura se realizó comparando cinco modelos preentrenados en su fase frozen (VGG16, ResNet50, MobileNetV2, EfficientNetB0 e InceptionV3) y, sobre los dos candidatos con mejor balanced accuracy OOF en esa fase, aplicando una segunda fase de fine-tuning de las últimas 20 capas del backbone. El criterio de selección, fijado por escrito antes de aplicarse, era la mayor balanced accuracy OOF a nivel de paciente entre los candidatos evaluados en frozen y fine-tuning. El resultado observado de aplicar ese criterio fue que ResNet50-FT superó tanto a InceptionV3-FT como a una fusión por promedio simple (50/50) de ambos candidatos fine-tuned, y fue en consecuencia el modelo seleccionado para la evaluación final. El criterio (qué métrica decide) no debe confundirse con el resultado (qué modelo ganó con esa métrica en este conjunto de datos): el segundo es contingente a esta cohorte y a este particionado, y no debe presentarse como una propiedad general de ResNet50 frente a otras arquitecturas.

El fine-tuning, en los dos candidatos en los que se aplicó, registró una balanced accuracy OOF superior a la obtenida en su fase frozen respectiva. Este dato es consistente con la idea de que ajustar las capas más profundas del backbone aporta una ganancia medible en esta tarea y con este protocolo, pero no permite generalizar cuánto de esa ganancia es atribuible al número concreto de capas ajustadas, a la tasa de aprendizaje empleada o a la propia arquitectura, porque estos factores no se variaron de forma independiente en el diseño experimental. Que la fusión de los dos modelos fine-tuned no superara al mejor de ellos por separado tampoco permite concluir que las estrategias de ensamblado sean poco útiles en general para esta tarea: solo se probó una fusión por promedio simple con pesos iguales, no un espacio más amplio de combinaciones.

11.4 Holdout y validación cruzada de cinco folds

El holdout final, evaluado una única vez sobre 35 pacientes tras fijar el protocolo por escrito, obtuvo una balanced accuracy de 93,89 %. La validación cruzada de cinco particiones (5CV), ejecutada posteriormente sobre el conjunto de 233 pacientes reagrupado en un nuevo particionado, arrojó un rango de balanced accuracy entre 82,30 % (fold 2) y 94,34 % (fold 3), con una media de 89,44 % (desviación estándar 4,64 puntos porcentuales) y una balanced accuracy OOF agregada de 88,68 %.

El resultado del holdout se sitúa cerca del extremo superior de ese rango, no en su centro ni en su extremo inferior. Esta observación es un hecho aritmético, no

una validación adicional del holdout ni una prueba de que el holdout sea representativo del comportamiento típico del modelo: el holdout constituye el resultado válido del protocolo pre-5CV en el que se obtuvo, y la 5CV es una evaluación posterior y complementaria sobre un particionado distinto de los mismos 233 pacientes, no una repetición del holdout ni una prueba que lo invalide retroactivamente. Dicho de otro modo: ambas cifras conviven porque responden a preguntas distintas —qué balanced accuracy se obtiene en una partición fija (holdout) frente a qué variabilidad se observó entre las cinco particiones evaluadas (5CV)— y ninguna de las dos es “la cifra correcta” que sustituye a la otra. Lo que sí puede afirmarse es que el propio rango de la 5CV (82,30 %–94,34 %) es la evidencia más directa disponible en este trabajo de que el resultado del modelo no es uniforme entre particiones, y que cualquier cifra puntual —incluida la del holdout— debe leerse con esa variabilidad en mente. Tampoco puede afirmarse, a partir de este único holdout y esta única 5CV, que el modelo generalice de forma uniforme “a cualquier partición posible”: solo se han observado cinco particiones adicionales al holdout, un número reducido de observaciones sobre el que no cabe extrapolar más allá de lo efectivamente registrado en ellas.

Es necesario además distinguir dos formas legítimas pero distintas de resumir la 5CV, que en ocasiones podrían leerse erróneamente como contradictorias: la media aritmética de las cinco balanced accuracy por fold (89,44 %) y la balanced accuracy calculada sobre la matriz de confusión agregada de los 233 pacientes OOF (88,68 %). Ambas cifras son correctas; difieren porque corresponden a dos métodos de agregación distintos —promediar cinco métricas ya calculadas por partición, frente a calcular una única métrica sobre el conjunto agregado de predicciones— y no porque una de ellas esté equivocada o porque exista algún error de cómputo entre ambas.

Figura 11.1 — Holdout frente al rango de balanced accuracy de la 5CV

El holdout se sitúa dentro del rango de variabilidad observado entre folds, cerca de su extremo superior; no se presenta como el resultado de referencia frente a la 5CV.

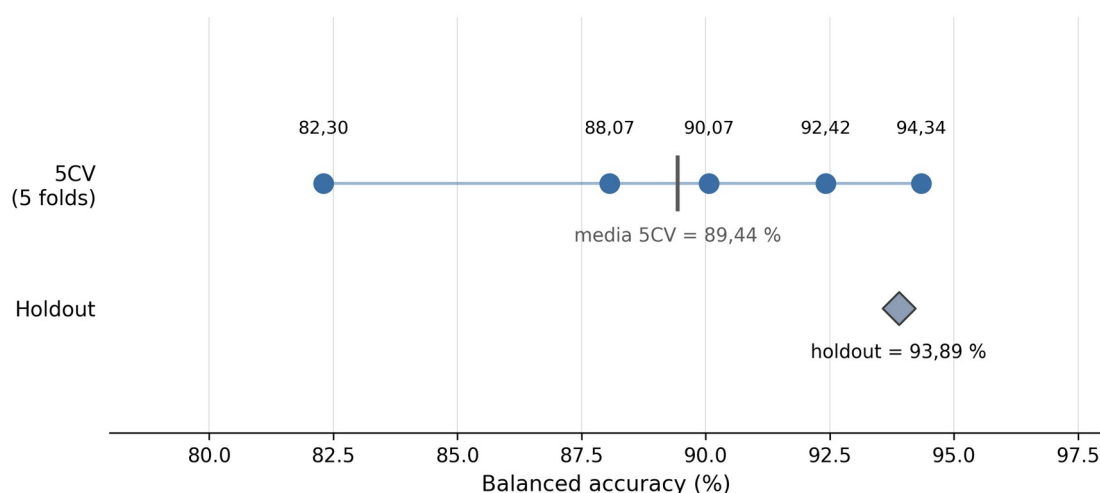


Figura 11.1 — Holdout frente al rango de variabilidad de la 5CV

El resultado del holdout (93,89 %) situado junto a los cinco valores de balanced accuracy por fold de la 5CV (82,30 %–94,34 %, media 89,44 %), mostrando la posición del holdout cerca del extremo superior de ese rango.

11.5 Comportamiento por clases y análisis del Fold 2

Los recalls por clase en la evaluación OOF de la 5CV (88,76 % para glioma, 80,49 % para meningioma y 96,77 % para pituitario) muestran que meningioma presenta el menor recall OOF agregado de las tres clases. Esta observación es un hecho verificable en la matriz de confusión agregada. No lo es, en cambio, la afirmación de que el meningioma sea una clase intrínsecamente más difícil de diagnosticar: esa sería una generalización sobre la naturaleza del problema clínico que excede lo que este trabajo puede sostener, dado que solo se ha evaluado un modelo, un pipeline de preprocesado y una única cohorte. En la matriz OOF agregada, los errores de meningioma se reparten principalmente entre glioma y pituitario.

El fold 2 de la 5CV es la partición con mayor tasa de error del conjunto, con una balanced accuracy de 82,30 % frente a un rango que llega hasta 94,34 % en el fold 3. Su matriz de confusión muestra un patrón concentrado: cinco casos de meningioma fueron clasificados como pituitario. Los historiales exportados del Fold 2 muestran una pérdida descendente y una accuracy ascendente en ambas fases, sin NaN ni una anomalía evidente en los arrays disponibles. Por tanto, esos historiales no muestran un fallo de entrenamiento evidente que explique por sí solo el menor rendimiento del

Fold 2. Con la evidencia disponible no puede atribuirse este comportamiento a una causa concreta.

11.6 Interpretabilidad mediante Grad-CAM

El análisis Grad-CAM se realizó sobre cinco casos seleccionados del conjunto de evaluación, generando quince imágenes (cinco casos por tres imágenes: original, mapa de calor y superposición), de los cuales tres corresponden a aciertos del modelo y dos a errores ya documentados en la evaluación cuantitativa. Este análisis se presenta en este trabajo exclusivamente como una herramienta cualitativa y exploratoria, aplicada a posteriori sobre un modelo ya entrenado y evaluado, y no como una prueba de localización tumoral, una cuantificación tipo IoU o Dice, ni como una forma de evaluación de segmentación.

Esta limitación no es una elección arbitraria de alcance, sino una consecuencia directa de cómo se ejecutó el análisis: el pipeline de este trabajo no incorporó, en ningún punto de su código de extracción o de generación de mapas de calor, una referencia espacial de segmentación (máscara tumoral) contra la que comparar cuantitativamente la superposición Grad-CAM/tumor. Esto es distinto de afirmar que esa referencia no exista en el origen: el dataset Cheng sí incluye anotaciones de máscara tumoral para cada imagen, pero este trabajo no las incorporó ni las utilizó como referencia espacial en el pipeline analizado. La observación visual disponible de los quince mapas se limita, por tanto, a una descripción posicional neutra —dónde se concentra la activación dentro de la imagen— sin que pueda traducirse en una afirmación sobre precisión de localización, sensibilidad o especificidad espacial. Incorporar esa referencia de máscara para una evaluación cuantitativa futura es una posibilidad razonable de continuación del trabajo, no una tarea ya realizada ni una limitación que reste validez a los resultados cuantitativos de clasificación, que no dependen de este análisis.

11.7 Comparabilidad con la literatura

La comparación entre trabajos requiere comprobar, además del dataset y la métrica, la unidad utilizada para construir las particiones. Un protocolo patient-level mantiene los cortes de un mismo paciente dentro de una única partición, mientras que un protocolo definido a nivel de imagen no garantiza por sí mismo esa independencia. Por ello, los porcentajes obtenidos bajo protocolos diferentes no deben compararse directamente sin verificar primero cómo se construyeron los conjuntos de entrenamiento y evaluación.

El estudio de Bulla, Anantha y Peram (2020), publicado en la revista *Traitement du Signal* (DOI 10.18280/ts.370407), es uno de los cuatro estudios aplicados

verificados contra su fuente original presentados en el capítulo 3, y constituye el comparador más directamente relacionado con este TFG por emplear la misma cohorte Cheng y la misma tarea de clasificación en tres clases. Confirma el uso del mismo dataset Cheng (3.064 imágenes, 233 pacientes, tres clases) y una arquitectura InceptionV3 con transfer learning, y reporta explícitamente un esquema de evaluación por paciente (agrupado, 10-fold). Los autores reportan una accuracy del 99,82 % en ese esquema patient-level. Esta cifra no es directamente equiparable a las de este trabajo en varios sentidos que conviene explicar: la métrica reportada por los autores es accuracy, no balanced accuracy —la métrica principal de este trabajo—, por lo que ambas cifras no miden exactamente lo mismo cuando la distribución de clases no es perfectamente equilibrada; el esquema de partición es un 10-fold agrupado por paciente, distinto del holdout único y de la 5CV empleados aquí; y no se ha podido confirmar en esta revisión si el preprocesado de imagen (recorte, normalización) aplicado por los autores es equiparable al de este trabajo. Por estas razones, esta comparación se presenta como una nota de comparabilidad —qué cifra ha reportado un trabajo que usa el mismo origen de datos— y no como una conclusión de que un enfoque sea superior o inferior al otro.

Nalawade et al., Hassan et al. y Arghistani et al., también verificados contra sus fuentes originales, se utilizan en esta memoria como referencias metodológicas sobre la unidad de evaluación, la validación externa y la deduplicación, respectivamente. No constituyen comparadores directos de rendimiento con este TFG porque difieren en la tarea, el dataset o el protocolo de evaluación.

11.8 Limitaciones metodológicas de la interpretación

Varias limitaciones acotan lo que puede inferirse de los resultados discutidos en este capítulo, sin que ninguna de ellas invalide por sí sola los resultados cuantitativos obtenidos: una limitación señala el alcance de una conclusión, no la corrección de la medición que la sustenta.

Por un lado, no se realizó una validación externa sobre una cohorte independiente en el protocolo final documentado: las tres etapas del proceso —selección de arquitectura, holdout final y 5CV— se apoyan en pacientes pertenecientes a la misma cohorte global Cheng de 233 pacientes, aunque cada etapa utiliza un subconjunto o particionado diferente. Esto significa que ninguna de las cifras presentadas mide el comportamiento del modelo ante datos de un origen distinto (otro hospital, otro escáner, otra población), y que la variabilidad observada entre folds de la 5CV es variabilidad interna dentro de la misma cohorte Cheng, no una estimación de la variabilidad esperable ante una fuente nueva.

Tampoco el procedimiento seguido constituye una validación cruzada anidada (nested CV): la arquitectura se seleccionó primero, sobre un particionado de 198 pacientes, y solo después se ejecutó la 5CV con el modelo y el protocolo ya fijados, sin que la selección de arquitectura se repita dentro de cada fold externo. Esto es una decisión de diseño documentada, no un error, pero implica que la 5CV evalúa la estabilidad del modelo ya seleccionado, no la estabilidad del propio proceso de selección de arquitectura ante distintas particiones.

Además, la interpretabilidad mediante Grad-CAM se limita a un análisis cualitativo sobre cinco casos, sin referencia de segmentación para una evaluación cuantitativa, como se ha detallado en la sección anterior.

Asimismo, la diferencia entre el baseline histórico multiclase y su reproducción (−5,21 puntos porcentuales) permanece sin causa confirmada, lo que introduce una fuente de incertidumbre sobre cuánta variabilidad experimental cabe esperar entre ejecuciones nominalmente idénticas de este pipeline, más allá de la ya cuantificada explícitamente por la propia 5CV.

Por último, el número de particiones adicionales al holdout (cinco folds) es reducido para caracterizar con precisión estadística la distribución completa de resultados esperable; el rango y la desviación estándar observados en la 5CV se basan únicamente en los cinco valores de balanced accuracy obtenidos, y no deben confundirse con el intervalo de confianza bootstrap del holdout, que procede de un cálculo distinto.

Frente a estas limitaciones, es justo señalar también, de forma equilibrada, las fortalezas metodológicas del diseño experimental que sí están verificadas: la separación estricta por paciente en todas las etapas, la existencia de un protocolo de evaluación final fijado por escrito y verificado por hash antes de abrir el conjunto de test, la ausencia de reapertura de la selección de arquitectura durante la 5CV, y la disponibilidad de un intervalo de confianza bootstrap para el resultado del holdout. Ninguna limitación aquí señalada resta validez a estos controles: acota el alcance de las conclusiones que pueden extraerse de los resultados, no la fiabilidad de cómo se obtuvieron esos resultados.

11.9 Síntesis de la discusión

Este trabajo parte de un resultado binario que, examinado con un control de atajo, no puede sostenerse como evidencia de detección de tumor, y que motivó la reorientación hacia una tarea multiclase dentro de una única cohorte. Dentro de esa tarea, la selección de arquitectura siguió un criterio fijado por escrito, y el resultado de aplicar ese criterio —ResNet50-FT como candidato seleccionado— es específico de

esta cohorte y este particionado, no una conclusión general sobre arquitecturas. Los dos candidatos sometidos a fine-tuning registraron una balanced accuracy OOF superior a la obtenida en su respectiva fase frozen, sin que pueda aislarse qué componente del ajuste fino explica esa diferencia.

La evaluación final combina un holdout único (93,89 %) y una validación cruzada de cinco particiones que revela un rango de variabilidad interna considerable (82,30 %–94,34 %), del que el holdout ocupa una posición cercana al extremo superior. Ambas cifras son válidas dentro de su propio protocolo y ninguna sustituye a la otra; juntas ofrecen una imagen más completa que cualquiera por separado, precisamente porque muestran que el resultado del modelo no es una cifra única sino un rango.

En la evaluación OOF agregada, meningioma presenta el menor recall de las tres clases. En el Fold 2 se observa además una concentración específica de errores meningioma → pituitario, sin que exista una causa demostrada para esa concentración. El análisis Grad-CAM aporta una exploración cualitativa complementaria, sin capacidad de cuantificar la localización de la señal.

La comparación directa de rendimiento con un estudio que comparte la misma cohorte y tarea se centra en Bulla et al. (2020), que reporta una accuracy del 99,82 % bajo un esquema group 10-fold patient-level sobre el mismo dataset. Dado que la métrica principal de este trabajo es balanced accuracy y que el protocolo de partición también difiere, ambas cifras no deben interpretarse como una comparación directa de rendimiento ni permiten establecer superioridad o inferioridad entre enfoques.

En conjunto, el trabajo aporta un pipeline con controles de separación por paciente verificados, un protocolo de evaluación final documentado y verificado por hash, y una caracterización explícita de su propia variabilidad interna mediante 5CV. No aporta, y no pretende aportar, una validación externa, una prueba de generalización a otras poblaciones, ni una validación clínica. Las limitaciones señaladas en este capítulo no son defectos ocultos: son, en su mayoría, características del alcance elegido para este trabajo, declaradas de forma explícita para que cualquier lectura posterior de sus resultados —incluida la de los propios autores en el capítulo de conclusiones— parta de una base de qué se ha demostrado y qué queda como pregunta abierta.

Tabla 11.1 — Hallazgos, interpretación permitida y límite de esa interpretación

Hallazgo	Interpretación permitida	Límite de la interpretación
Binario 98,57 % BA con shortcut de atajo al 98,16 %	La cifra binaria no es evidencia aislada de detección de tumor; motivó la reorientación a la tarea multiclase	No implica que la red no haya aprendido ninguna señal tumoral, ni que exista fuga de información entre entrenamiento y test
Baseline histórico 79,66 % vs. reproducción 74,44 % (–5,21 pp)	La diferencia está documentada y es real	La causa no está confirmada; el determinismo de GPU no estricto es solo una fuente posible, no la causa demostrada
ResNet50-FT seleccionado (88,54 % BA OOF) sobre fusión (88,29 %) e InceptionV3-FT (85,76 %)	Es el mejor candidato observado en este conjunto de datos bajo el criterio fijado (mayor BA OOF)	No es una conclusión general sobre la superioridad de ResNet50 frente a otras arquitecturas
Holdout 93,89 % cerca del extremo superior del rango 5CV (82,30 %–94,34 %)	Ambas cifras son válidas dentro de su protocolo; juntas muestran la variabilidad interna del modelo	El holdout no queda invalidado por la 5CV, ni la 5CV demuestra que el holdout sea “el resultado típico”
Meningioma con menor recall OOF (80,49 %) de las tres clases; en el fold 2, específicamente, 5 casos de meningioma clasificados como pituitario	Son patrones observados, específicos de este pipeline y esta cohorte	No implica que el meningioma sea intrínsecamente más difícil de diagnosticar en general; en la matriz OOF agregada los errores de meningioma se reparten entre glioma y pituitario, no solo hacia pituitario
Grad-CAM sobre 5 casos (15 imágenes), sin máscara	Aporta una exploración cualitativa de dónde se concentra la activación	No cuantifica localización tumoral ni sustituye una evaluación de segmentación

Hallazgo	Interpretación permitida	Límite de la interpretación
empleada como referencia cuantitativa		
Bulla et al. (2020): 99,82 % accuracy, 10-fold por paciente, mismo dataset	Es el comparador más directamente relacionado entre los estudios aplicados verificados, al compartir la cohorte Cheng y la tarea de tres clases	Métrica (accuracy, no BA), esquema de partición y preprocesado no son directamente equiparables a los de este trabajo

12. Viabilidad y recursos

12.1 Recursos técnicos utilizados

La ejecución de este proyecto se apoyó en un conjunto de recursos técnicos de tipo experimental, no en un entorno de producción software desplegado. A alto nivel, los experimentos se ejecutaron mediante notebooks Jupyter y scripts auxiliares en Python sobre TensorFlow/Keras, lanzados de forma automatizada en un clúster de computación de altas prestaciones (HPC) mediante el gestor de colas SLURM. Las distintas ejecuciones dejaron evidencias acordes con su función: logs de salida y error (.out/.err) y, cuando correspondía, ficheros JSON y CSV de métricas, checkpoints de modelo en formato .keras y otros artefactos de resultado. Este capítulo no repite el diseño del pipeline software, ya descrito en el capítulo 7: aquí interesa qué recursos fueron necesarios para ejecutarlo y sostenerlo, no cómo está construido internamente.

No se incluyen aquí versiones concretas de Python, TensorFlow, CUDA, cuDNN o del driver de la GPU, porque esa información no está confirmada de forma directa entre los artefactos consultados en esta auditoría; omitirla es preferible a completarla con valores genéricos no verificados.

12.2 Infraestructura software y entorno de ejecución

Hay tres tipos de componentes que a veces se mezclan al hablar de “la infraestructura del proyecto”. El primero es el software efectivamente ejecutado dentro del pipeline final: los notebooks ejecutados de forma automatizada en HPC mediante `jupyter nbconvert --execute`, junto con los scripts Python auxiliares o específicos de determinadas etapas lanzados desde los correspondientes jobs SLURM, cuyos resultados quedan persistidos, cuando corresponde, en los artefactos JSON/CSV y en los checkpoints de modelo ya mencionados. Durante la etapa de selección de arquitectura se documentaron 24 archivos de checkpoint .keras, mediante listado y hash; esta auditoría no ha abierto la arquitectura interna de cada uno de ellos, por lo que la cifra debe leerse como recuento de artefactos documentados, no como confirmación de 24 modelos cualitativamente distintos entre sí.

El segundo componente es la infraestructura institucional: el clúster HPC de la Universidad de Sevilla y su gestor de colas SLURM, sobre el que se ejecutaron los jobs documentados en este capítulo. Esta infraestructura es un recurso compartido de la institución, no un recurso desplegado específicamente para este proyecto.

El tercer componente es de naturaleza distinta y no debe mezclarse con los dos anteriores: un componente histórico de diseño y prototipado, ScanIA, ya descrito con detalle en el capítulo 5. Sobre este componente se conserva un `Dockerfile`

diseñado para un despliegue compatible con Cloud Run, pero no existe evidencia de que ese despliegue llegara a producirse, ni de un uso real de almacenamiento en la nube más allá de una dependencia declarada. Su mención en este capítulo tiene un único propósito: dejar claro que el diseño de una infraestructura histórica no equivale a infraestructura final verificada, y que ningún recurso de computación en la nube ha sido contabilizado como parte de los recursos realmente utilizados por el núcleo experimental de este trabajo. (Un despliegue distinto y posterior, no derivado confirmadamente de este Dockerfile, sí se verificó como operativo; véanse el apartado 12.8 y el capítulo 5.)

12.3 Ejecución en HPC mediante SLURM

Las ejecuciones documentadas del proyecto se organizaron como una serie de jobs SLURM con funciones diferenciadas. Se han identificado 16 IDs de job distintos y se han inspeccionado 12 scripts `.slurm`; algunos de estos scripts fueron reutilizados en varias ejecuciones. Los jobs quedan además documentados mediante sus logs y los artefactos generados por cada etapa. Entre esas funciones se encuentran: diagnóstico del entorno de ejecución, el run que reconstruyó la etapa binaria histórica junto con el control multiclase, benchmark y fine-tuning de arquitecturas, fusión por promedio de candidatos, evaluación de holdout final, generación de Grad-CAM, exportación de resultados, y las tres fases de la validación cruzada de cinco particiones (preparación, ejecución en array y agregación). La Tabla 12.1 resume esta clasificación job por job; no se repite aquí el detalle línea a línea de cada script, que ya está documentado en la matriz de recursos de la auditoría y se detalla en el Anexo B del documento de anexos técnicos.

Esta separación por job permite distinguir con precisión qué parte de la infraestructura corresponde a cada etapa del trabajo, evitando presentar el conjunto como una única ejecución monolítica. En particular, la validación cruzada de cinco particiones se ejecutó como un array SLURM de cinco subtareas (una por fold) configurado para ejecutarse de forma secuencial, una subtarea activa a la vez, y no como cinco ejecuciones simultáneas en paralelo. La persistencia y copia de artefactos al finalizar cada job tampoco se implementó de forma idéntica en todos los scripts inspeccionados, lo que se señala aquí como una característica del entorno de ejecución, sin que ello afecte a la validez de los resultados ya persistidos y verificados en los capítulos 9 y 10.

Esta organización en jobs con funciones diferenciadas, documentados mediante sus logs y, cuando corresponde, mediante sus artefactos de salida, permite, en conjunto con el protocolo de evaluación fijado por escrito y las verificaciones de

hash ya descritas en los capítulos 9 y 10, hablar de una reproducibilidad a nivel de procedimiento: la posibilidad de documentar y verificar qué se ejecutó, en qué orden y sobre qué recursos, sin que ello implique una garantía de reproducir de forma idéntica, bit a bit, cada cifra concreta obtenida en cada ejecución.

12.4 Recursos computacionales observados por etapa

El hardware de GPU confirmado varía según el job, y esta variación debe respetarse en lugar de generalizarse. En los jobs con uso de GPU para los que existe evidencia explícita —ya sea mediante información registrada en los logs de ejecución o mediante la directiva de solicitud de recursos del correspondiente script SLURM— se documenta el uso de una NVIDIA A30. Entre estas ejecuciones se encuentran el run de reconstrucción patient-level y control multiclase, los jobs de benchmark y fine-tuning de arquitecturas, la evaluación del holdout final, la generación de Grad-CAM y el array de la 5CV. En cambio, hay jobs para los que no existe una GPU confirmada en ningún artefacto disponible —la fusión por promedio y la exportación de resultados—, y jobs de preparación o agregación de datos que no solicitan GPU en absoluto, por tratarse de tareas de partición o de agregación de ficheros JSON. Los nombres de nodo (gpu01, gpu02, gpu04, gpu06) identifican máquinas del clúster, no modelos de tarjeta gráfica; el modelo de GPU solo se afirma cuando está confirmado de forma independiente para ese job concreto.

El modelo finalmente seleccionado (ResNet50-FT) tiene 23.850.371 parámetros totales, de los cuales 14.691.331 son entrenables durante la fase de fine-tuning. Estas cifras describen la magnitud del modelo entrenado; esta auditoría no las ha traducido en una estimación de memoria de GPU, de FLOPs, de consumo eléctrico o de coste, porque ninguna de esas magnitudes ha sido medida directamente en los artefactos disponibles, y una estimación indirecta introduciría supuestos no verificados.

Para el array de la 5CV conviene además distinguir dos magnitudes temporales que no deben confundirse entre sí: el tiempo de pared (wall-clock) del array completo, que no se ha podido calcular por falta de una marca de fin explícita de la última subtarea, y el tiempo de cómputo de GPU efectivamente registrado por el propio pipeline dentro de cada fold, que sí está documentado por separado para cada uno de los cinco folds y se detalla en el Anexo B del documento de anexos técnicos. El segundo no incluye necesariamente la inicialización del entorno, la carga de datos, el arranque de TensorFlow/XLA, las operaciones de entrada/salida ni el tiempo de espera entre subtareas del array, por lo que no debe interpretarse como una medida de la duración total de esa etapa.

12.5 Viabilidad técnica

La evidencia disponible permite responder afirmativamente a si el proyecto fue técnicamente viable con los recursos utilizados, entendida esta viabilidad en sentido técnico y experimental, no clínico: los experimentos se ejecutaron efectivamente sobre la infraestructura HPC descrita, existen logs y artefactos que documentan esa ejecución, y se completaron la selección de arquitectura, la evaluación de holdout final y la validación cruzada de cinco particiones. En las ejecuciones de entrenamiento completadas para las que existe evidencia explícita de hardware GPU, una única NVIDIA A30 por job o subtarea permitió ejecutar el protocolo previsto. Esta constatación se limita a la ejecutabilidad del proyecto con los recursos empleados y no implica eficiencia u optimización: no implica que la configuración de recursos utilizada fuera la más eficiente posible, ni que pudiera reducirse sin afectar a los resultados. No se dispone de una comparación de rendimiento frente a configuraciones alternativas de hardware, por lo que este trabajo no formula ninguna afirmación sobre eficiencia u optimización de los recursos empleados.

12.6 Viabilidad económica y limitaciones de estimación

No se dispone de un registro económico suficientemente completo para realizar una estimación monetaria defendible del coste total del proyecto. Además, una parte relevante de la infraestructura computacional utilizada corresponde a recursos institucionales, por lo que asignarles un precio comercial equivalente introduciría supuestos externos no presentes en la documentación del TFG. Por este motivo, la viabilidad económica se aborda de forma cualitativa y no mediante una cifra total de coste.

Esto no equivale a afirmar que el proyecto tuviera coste cero: los recursos institucionales empleados —el propio clúster HPC y el tiempo de cómputo consumido en él— tienen un coste real, aunque no exista un pago directo documentado por parte del alumno ni un registro que permita cuantificarlo. La formulación correcta, y la única que sostiene la evidencia disponible, es que no se dispone de un coste monetario total documentado, ni de los datos necesarios (tarifa del clúster, coste energético, coste de personal) para construirlo de forma defendible. Por la misma razón, tampoco se calcula aquí ningún coste asociado a servicios de computación en la nube: el Dockerfile conservado de ScanIA documenta una intención de diseño de despliegue compatible con Cloud Run, no un uso real y facturado de esa infraestructura.

La clasificación completa del software y la infraestructura empleados entre categorías de origen abierto, institucional y de diseño histórico de servicios cloud queda pendiente de una revisión más exhaustiva, no realizada en su totalidad en esta

auditoría. Puede afirmarse, de forma prudente, que el trabajo emplea herramientas software de uso habitual en investigación, pero no se ha realizado una clasificación exhaustiva de las licencias de todos los componentes empleados dentro del alcance de este capítulo. Esta limitación no bloquea las conclusiones de viabilidad técnica de la sección anterior, que se apoyan en la evidencia de ejecución ya documentada, no en la clasificación pendiente de licencias.

12.7 Síntesis

El proyecto fue técnicamente ejecutable mediante una combinación de software científico de uso común en investigación y recursos HPC institucionales, sin que fuera necesario desplegar una infraestructura de producción propia. El gestor de colas SLURM permitió automatizar las ejecuciones y mantener una separación clara entre las distintas funciones del trabajo experimental: diagnóstico de entorno, entrenamiento y benchmark de arquitecturas, evaluación de holdout, generación de Grad-CAM, exportación de resultados y las tres fases de la validación cruzada de cinco particiones.

En las ejecuciones finales completadas para las que existe evidencia de uso de GPU, el protocolo se ejecutó con una única NVIDIA A30 por job o subtarea. La evidencia disponible no permite, sin embargo, generalizar ese mismo hardware a la totalidad de los 16 jobs documentados, algunos de los cuales no solicitan GPU o no la confirman en ningún artefacto accesible, ni permite concluir que la configuración empleada fuera la más eficiente posible.

Finalmente, no existe una base económica suficientemente completa para calcular un coste monetario total defendible de este trabajo, y el Dockerfile histórico conservado de ScanIA debe leerse como evidencia de un diseño de despliegue compatible con Cloud Run, nunca como prueba de un uso real de infraestructura en la nube; el uso real y verificado de Cloud Run en una fase posterior del proyecto (servicio run-model) se documenta por separado en el apartado 12.8 y en el capítulo 5, sin relación de continuidad de código confirmada con este Dockerfile histórico. Estas conclusiones se limitan estrictamente a la viabilidad técnica y económica de los recursos empleados, y no constituyen una valoración general del proyecto, que se aborda en los capítulos siguientes.

Tabla 12.1 — Resumen de jobs SLURM documentados

Job	Función	Nodo	GPU	Duración documentada	Estado/ observación
70436	Reconstrucción patient-level + tarea binaria + control multiclase	gpu01	NVIDIA A30 24GB (confirmada)	Sí (00:28:32)	Verificado
70751	Diagnóstico de entorno	gpu01	Sin GPU física detectada (no solicita gres)	Sí (00:01:24)	Job de diagnóstico, no de ejecución del pipeline
70752	Test de GPU (diagnóstico)	gpu02	NVIDIA A30 (confirmada)	No impresa	Job de diagnóstico
70754	Test de GPU (diagnóstico)	gpu02	NVIDIA A30 (confirmada)	No impresa	Job de diagnóstico
70885	Benchmark multiclase (intento)	gpu02	NVIDIA A30 (confirmada)	No aplica	Job fallido (NotJSONError, notebook vacío); no productivo
70886	Benchmark multiclase (reproducción baseline)	gpu02	NVIDIA A30 (confirmada)	No calculada (sin marca de fin)	Detenido por RuntimeError del chequeo de regresión
70887	Benchmark multiclase (arquitecturas, fase frozen)	gpu02	NVIDIA A30 (confirmada)	Sí (00:21:26)	Verificado
70888	Benchmark multiclase (fine-tuning)	gpu02	NVIDIA A30 (confirmada)	Sí (00:09:24)	Verificado
70891	Benchmark multiclase (continuación)	gpu02	NVIDIA A30 (confirmada)	Sí (00:09:36)	Verificado
70929	Fusión por promedio (postproceso sobre OOF)	No confirmado en ningún artefacto	No aplica (sin evidencia de uso de GPU)	No calculada	Verificado con reserva (nodo no confirmable)
70930	Evaluación de holdout final	gpu02	A30 (soportada por directiva)	No calculada (solo marca)	Verificado

Job	Función	Nodo	GPU	Duración documentada	Estado/observación
			SLURM --gres=gpu:a30:1)	de inicio)	
70931	Generación de Grad-CAM	gpu02	A30 (soportada por directiva SLURM)	No calculada	Verificado
70932	Exportación de resultados	gpu04	No confirmada	No calculada (log truncado)	Exportación documentada con reserva
71101	Preparación/partición de la 5CV	gpu06	No aplica (sin solicitud de GPU)	No calculada	Verificado
71102	Array 5CV (entrenamiento, 5 folds, secuencial %1)	gpu02	NVIDIA A30 (confirmada en los .err de las 5 subtareas)	No calculada (wall-clock del array; tiempo de cómputo por fold documentado aparte en el Anexo B del documento de anexos técnicos)	Verificado
71107	Agregación de resultados de la 5CV	gpu06	No aplica	No calculada	Verificado

Tabla 12.2 — Recursos por categoría

Recurso	Uso en el proyecto	Evidencia/alcance
HPC / SLURM (Universidad de Sevilla)	Entrenamiento y evaluación automatizada de los experimentos	Documentado mediante 16 jobs y sus logs .out/.err
NVIDIA A30	Entrenamiento y evaluación en los jobs	Confirmada por log de ejecución o por directiva

Recurso	Uso en el proyecto	Evidencia/alcance
	donde su uso está confirmado	SLURM, según el job; no generalizable a todos
Jupyter (nbconvert --execute)	Ejecución automatizada de notebooks en el clúster, sin edición manual celda a celda	Confirmado en los scripts .slurm de envío
Artefactos JSON/CSV	Persistencia de métricas y trazabilidad de resultados	Resultados ya utilizados y verificados en los capítulos 9 y 10
Checkpoints .keras	Persistencia de checkpoints de modelo	24 checkpoints documentados mediante listado y hash en la etapa de selección de arquitectura
Dockerfile ScanIA	Diseño histórico de despliegue compatible con Cloud Run	No demuestra un despliegue ni un uso real de infraestructura cloud

12.8 Despliegue operativo del servicio y material demostrativo

Con posterioridad al cierre de la redacción de este documento, en el marco de trabajo adicional de despliegue operativo de un servicio de inferencia en la nube, se detectó y corrigió una incidencia de disponibilidad en un servicio Cloud Run identificado como run-model, que sirve el modelo de clasificación descrito en este trabajo. El servicio devolvía errores de tipo 503 al recibir peticiones de inferencia; el diagnóstico atribuyó el fallo al agotamiento de memoria del contenedor, provocado por la ejecución de varios procesos trabajadores que cargaban, cada uno, una copia completa del modelo en memoria. La corrección aplicada ajustó la memoria asignada al contenedor y limitó la concurrencia de procesos por instancia, y el correcto funcionamiento del servicio quedó verificado mediante una revisión desplegada posteriormente y una comprobación de estado. El detalle técnico completo de esta incidencia se recoge en el Anexo J del documento de anexos técnicos.

Como material complementario de difusión del proyecto se ha producido, además, un vídeo demostrativo de corta duración (aproximadamente dos minutos, con subtítulos en español e inglés) que muestra el funcionamiento de una aplicación cliente de escritorio conectada a este mismo servicio — aplicación cuyo código fuente se ha inspeccionado directamente y se documenta en el capítulo 5—, incluyendo la clasificación tanto de la tarea binaria histórica como de la tarea multiclase que constituye el resultado final de este trabajo. El vídeo está disponible en <https://youtu.be/QgTOD3KOTpl> y se referencia únicamente como material adicional de divulgación del proyecto; no sustituye ni amplía ninguno de los resultados experimentales documentados en los capítulos 8 a 11, que siguen siendo la evidencia cuantitativa de referencia de este trabajo.

Esta actualización documenta trabajo operativo y de material audiovisual posterior al cierre experimental del proyecto, y no reabre ni modifica la caracterización del prototipo histórico de interfaz presentada en el capítulo 5 ni la clasificación de los componentes de infraestructura del apartado 12.2. La existencia de un servicio run-model operativo en la fecha de esta actualización es una observación distinta, verificada de forma directa en una sesión de trabajo posterior, que este documento no relaciona de forma confirmada con los artefactos de despliegue históricos ya descritos en el capítulo 5. Con posterioridad a esta actualización, la inspección directa del código fuente del cliente de escritorio y del backend (capítulo 5) confirmó de forma independiente que ambos artefactos están conectados entre sí y con este mismo servicio run-model, reforzando con evidencia de código lo que hasta entonces solo se había observado operativamente.

13. Consideraciones éticas y limitaciones

13.1 Procedencia de los datos y consideraciones éticas

13.1.1 Dataset Cheng

El conjunto de imágenes utilizado para la tarea multiclase de este trabajo procede íntegramente del dataset de Cheng et al., distribuido públicamente en Figshare. El artículo original que acompaña a este dataset declara explícitamente que los registros e información de los pacientes fueron anonimizados y de-identificados antes del análisis, por lo que puede afirmarse que los datos de origen llegaron a este proyecto ya sometidos a un proceso de anonimización documentado por los autores del dataset. El mismo artículo declara también que el estudio fue aprobado por los comités de ética del Nanfang Hospital y del General Hospital de la Tianjin Medical University, los dos centros de procedencia de las imágenes. Esta aprobación ética y la anonimización declarada son dos hechos distintos, verificados de forma independiente, y ninguno de los dos implica el otro.

Distinto es el caso del consentimiento informado de los pacientes de origen. En el texto del artículo recuperado durante esta auditoría no se ha localizado una mención explícita al consentimiento informado, aunque sí se mencionan por separado la anonimización y la aprobación ética. La aprobación por parte de un comité de ética no permite, por sí sola, dar por hecho que existió y quedó documentado un consentimiento informado explícito de los pacientes. En consecuencia, este aspecto se mantiene como no verificado: no se ha localizado en la evidencia revisada una mención explícita al consentimiento informado de los pacientes de origen, por lo que este aspecto se mantiene como no verificado, sin que ello implique afirmar ni descartar que dicho consentimiento existiera.

En cuanto a la licencia de distribución, la página oficial del dataset en Figshare indica una licencia CC BY 4.0. Esta designación se estableció mediante verificación externa directa de la página del dataset; en una revisión posterior dentro de esta misma auditoría, el bloque específico de licencia no pudo volver a extraerse en texto plano, por lo que el dato se mantiene como verificado con reserva y no se eleva a una verificación completa. Finalmente, más allá de la anonimización, la aprobación ética y la licencia ya comentadas, no se han revisado en detalle otras condiciones de uso específicas que pudiera imponer Figshare sobre este dataset, por lo que este aspecto permanece pendiente de fuente original y no se afirma ni se descarta la existencia de condiciones adicionales.

13.1.2 Dataset IXI

El dataset IXI, empleado únicamente en la tarea histórica de clasificación binaria descartada en este trabajo (véanse los capítulos 2 y 6), se distribuye bajo una licencia distinta: CC BY-SA 3.0, con obligación de citar la fuente de los datos. Esta licencia es diferente de la CC BY 4.0 que rige el dataset Cheng y ambas no deben tratarse como equivalentes.

Para IXI, esta auditoría ha verificado la licencia de distribución, pero no ha reconstruido con el mismo nivel de detalle documental las condiciones éticas originales de adquisición. No se dispone, por tanto, de evidencia equivalente a la de Cheng respecto a anonimización explícita, aprobación por comité de ética o consentimiento informado de los sujetos de IXI, y no debe asumirse por analogía que las mismas garantías documentadas para Cheng se extienden a IXI. Como en el caso de Cheng, tampoco se han revisado en detalle eventuales condiciones de uso adicionales de IXI más allá de la licencia y la obligación de cita.

La Tabla 13.1 resume el estado documental de ambos datasets en los aspectos considerados.

Tabla 13.1 — Estado documental de los datos

Aspecto	Cheng	IXI	Estado/alcance
Anonimización	Declarada explícitamente en el artículo fuente	No verificada con el mismo detalle documental	Verificado solo para Cheng
Aprobación ética	Declarada explícitamente (dos comités identificados)	No verificada con el mismo detalle documental	Verificado solo para Cheng
Consentimiento informado	No se ha localizado mención explícita	No verificado	No verificado en ninguno de los dos casos
Licencia	CC BY 4.0 (verificada con reserva)	CC BY-SA 3.0, con obligación de cita (verificada)	Licencias distintas, no equivalentes
Condiciones de uso adicionales	No revisadas en detalle	No revisadas en detalle	Pendiente de fuente original en ambos casos

13.2 Sesgos y limitaciones metodológicas

Este trabajo incluyó, en una fase histórica ya descartada como enfoque principal (véanse los capítulos 2 y 6), una tarea de clasificación binaria tumor/no-tumor que combinaba el dataset Cheng (clase tumor) con el dataset IXI (clase no-tumor). Esta combinación introdujo un sesgo de procedencia confirmado: la clase estaba asociada de forma determinista al dataset de origen, lo que permitía a un clasificador basado únicamente en descriptores de bajo nivel alcanzar una exactitud muy por encima del azar sin utilizar representaciones aprendidas por una CNN. Este resultado demuestra la existencia de información discriminativa de bajo nivel asociada a las etiquetas, pero no permite determinar qué parte de esa información está relacionada o no con características tumorales. Este comportamiento fue uno de los motivos que llevó a descartar dicha tarea como enfoque principal del proyecto, en favor de la tarea multiclase sobre una única cohorte, ya discutida en el capítulo 11.

Este sesgo de procedencia debe distinguirse con claridad de los controles de fuga de información (data leakage) aplicados de forma transversal en el proyecto: disjunción de `patient_id` entre particiones, verificación de unicidad por SHA-256 de cada imagen y agrupación por paciente en toda validación cruzada. Estos controles no detectaron solapes de `patient_id` entre particiones ni coincidencias de SHA-256 entre imágenes, y el procedimiento de búsqueda de casi-duplicados mediante pHash y SSIM no confirmó candidatos de casi-duplicado en el universo de imágenes auditado. Estos resultados corresponden a los procedimientos concretos aplicados y no permiten afirmar una ausencia total de fuga de información en un sentido absoluto; son, en todo caso, un tipo de comprobación distinto del sesgo de procedencia descrito en el párrafo anterior, y ambos no deben confundirse entre sí.

En la tarea multiclase final, sobre la que se basan los resultados discutidos en el capítulo 11, las tres etiquetas proceden de la cohorte Cheng, por lo que desaparece la asociación determinista entre clase y dataset externo que caracterizaba la tarea binaria, sin que ello permita descartar otros sesgos residuales dentro de la propia cohorte. Por eso, la evaluación final del proyecto no incluyó una validación externa sobre una cohorte independiente: la selección de arquitectura se realizó sobre 198 pacientes de Cheng, el holdout final sobre 35 pacientes de la misma cohorte, y la validación cruzada de cinco particiones reagrupa a esos mismos 233 pacientes de Cheng en un nuevo particionado, sin incorporar una cohorte externa independiente. Por tanto, este protocolo no evalúa la generalización a centros, escáneres, protocolos de adquisición o poblaciones externas a la cohorte Cheng. No puede afirmarse, por tanto, que el modelo generalice a datos de otras procedencias; la evidencia obtenida se limita a lo observable dentro de la cohorte Cheng.

El holdout, descrito y discutido con más detalle en los capítulos 10 y 11, constituye el resultado válido del protocolo pre-5CV; la validación cruzada de cinco particiones se incorporó posteriormente como una evolución metodológica orientada a caracterizar la variabilidad interna del resultado, sin invalidar ni sustituir al holdout. Además, este procedimiento no constituye una validación cruzada anidada: la arquitectura se seleccionó primero, sobre los 198 pacientes de la Etapa B, y solo después se ejecutó la 5CV con el modelo y el protocolo ya fijados, sin reabrir la selección de arquitectura dentro de cada fold de la 5CV. La 5CV aporta así una caracterización de la variabilidad del resultado bajo distintos particionados de la misma cohorte, pero no equivale a una estimación completamente independiente obtenida mediante anidamiento.

Como limitación adicional de reproducibilidad, el determinismo estricto de GPU no se activó de forma explícita durante los entrenamientos, lo cual fue una decisión documentada del proyecto. Esta configuración constituye una posible fuente de variabilidad entre ejecuciones, pero no se dispone de evidencia que permita atribuirle de forma demostrada la diferencia observada entre el resultado baseline y su reproducción, discutida en el capítulo 11.

Por último, el análisis Grad-CAM realizado en este trabajo tiene un alcance estrictamente cualitativo y post-hoc. El dataset Cheng original incluye anotaciones de máscara tumoral para cada imagen, además de la imagen y la etiqueta de clase; sin embargo, el análisis Grad-CAM realizado en este trabajo no incorporó ni utilizó esas anotaciones de máscara como referencia para una evaluación cuantitativa de la superposición. En consecuencia, los mapas de Grad-CAM deben interpretarse como evidencia cualitativa de las regiones que más influyen en la predicción del modelo, y no como segmentaciones ni como prueba de localización tumoral.

Tabla 13.2 — Limitaciones principales

Limitación	Consecuencia para la interpretación
Sesgo de procedencia en la tarea binaria histórica	Los resultados de esa tarea, ya descartada, no pueden interpretarse como evidencia de reconocimiento tumoral
Evaluación final restringida a la cohorte Cheng	No hay evidencia de generalización a otros hospitales, escáneres, protocolos o poblaciones
Ausencia de validación externa	Los resultados no pueden extrapolarse

Limitación	Consecuencia para la interpretación
	fuera de la cohorte Cheng auditada
Validación cruzada interna y no anidada	La 5CV caracteriza variabilidad interna del modelo ya seleccionado, no sustituye una estimación independiente anidada
Determinismo de GPU no activado explícitamente	Introduce una posible fuente de variabilidad entre ejecuciones no completamente controlada
Grad-CAM cualitativo sin evaluación cuantitativa frente a máscara	Los mapas no constituyen prueba de localización tumoral ni sustituyen una segmentación

13.3 Límites de interpretación y aplicabilidad

Este trabajo no constituye un dispositivo médico, una herramienta de diagnóstico automático ni un sistema clínico validado. El sistema desarrollado realiza una clasificación experimental de los pacientes de la cohorte entre las tres clases tumorales estudiadas a partir de sus cortes de resonancia magnética, mediante agregación de las probabilidades obtenidas por corte, dentro del protocolo y la cohorte descritos en los capítulos anteriores; no debe describirse como un sistema que “diagnostica” glioma, meningioma o pituitario, ni emplearse un vocabulario clínico que sugiera una validación asistencial que no se ha realizado. La clasificación obtenida tampoco equivale a una detección, una segmentación o una estimación de volumen tumoral; el análisis Grad-CAM, al ser cualitativo, no modifica esta distinción ni añade una capacidad de localización validada.

La evidencia obtenida en este trabajo se restringe a la cohorte de 233 pacientes de Cheng utilizada en la tarea multiclase, sin que de este trabajo pueda concluirse si dicho tamaño de cohorte es grande, pequeño, suficiente o insuficiente en términos generales; esa valoración excede lo que este trabajo puede establecer.

13.4 Síntesis

La documentación disponible permite describir con una precisión razonable el marco ético declarado por los autores del dataset Cheng —anonimización y aprobación por comité de ética— así como las licencias de distribución de ambos datasets, manteniendo como no verificados el consentimiento informado explícito y las condiciones de uso adicionales, sin que ello se traduzca en una afirmación de que estos aspectos no existieran.

La principal limitación científica del proyecto es que la evaluación final permanece interna a la cohorte Cheng, sin una validación sobre una cohorte externa independiente. La incorporación de la validación cruzada de cinco particiones mejora la caracterización de la variabilidad interna del resultado bajo distintos particionados, pero no sustituye a una cohorte externa ni a un esquema de validación cruzada anidada. El análisis Grad-CAM añade una capa de explicabilidad cualitativa sobre las regiones que influyen en la predicción, sin constituir una validación espacial de la localización tumoral.

En conjunto, el sistema desarrollado debe entenderse como un artefacto experimental de clasificación, evaluado dentro de los límites descritos en este capítulo, y no como una herramienta clínica. El capítulo de conclusiones y trabajo futuro retoma, cuando corresponda, qué extensiones del trabajo permitirían abordar algunas de estas limitaciones.

14. Conclusiones y trabajo futuro

14.1 Conclusiones

Este TFG se planteó, en su formulación final, desarrollar y evaluar un procedimiento experimental trazable y reproducible para la clasificación multiclase de tumores cerebrales mediante RM, utilizando identificación real de paciente en el protocolo final y conservando la trazabilidad de las principales decisiones y artefactos experimentales. Las conclusiones de este capítulo se apoyan en esa formulación final, no en la exploración inicial del proyecto, cuya evolución ya se ha documentado en los capítulos anteriores.

El desarrollo del trabajo mostró, desde una fase temprana, que una métrica de rendimiento elevada no es por sí sola una garantía suficiente si el diseño experimental que la produce contiene factores de confusión no controlados. La tarea binaria explorada inicialmente —distinguir entre presencia y ausencia de tumor— estaba condicionada por una asociación estructural entre la etiqueta de clase y el dataset de procedencia de las imágenes, lo que impide utilizar su resultado como evidencia central de capacidad de clasificación tumoral. Por esa razón, ese resultado se trata en esta memoria como un antecedente histórico y como una lección metodológica: la tarea binaria histórica mostró que un resultado aparentemente favorable puede quedar metodológicamente comprometido cuando la clase y la procedencia del dataset están estructuralmente asociadas.

La reorientación del proyecto hacia la clasificación multiclase dentro de una única cohorte de referencia eliminó esa asociación determinista entre la etiqueta y un dataset externo, aunque sin que ello permita descartar otros sesgos residuales dentro de la propia cohorte. A partir de esa reorientación, el trabajo final se desarrolló con separación y evaluación sistemáticas a nivel de paciente, comparando varias arquitecturas bajo un protocolo común antes de fijar por escrito, con verificación por huella criptográfica, la configuración que después se evaluaría.

El modelo finalmente seleccionado, ResNet50 con fine-tuning (ResNet50-FT), fue el candidato con mejor balanced accuracy (BA) sobre las predicciones out-of-fold empleadas en la selección, con un valor del 88,54 %. Con esa configuración fijada antes de abrir el conjunto de evaluación, el modelo alcanzó una BA del 93,89 % sobre un holdout previamente reservado dentro de la misma cohorte Cheng. Una validación cruzada posterior sobre el conjunto completo de pacientes de esa cohorte permitió caracterizar con más detalle la variabilidad interna del modelo: la BA media entre particiones fue del 89,44 % $\pm$ 4,64 puntos porcentuales, con una BA calculada sobre las predicciones agregadas de todas las particiones del 88,68 %. La 5CV mostró que

el resultado del holdout se encontraba cerca de la parte alta de la variabilidad observada entre particiones, aportando una estimación más completa del comportamiento interno del modelo que la que ofrecería una única evaluación aislada; la variabilidad entre particiones, además, no fue despreciable.

El rendimiento observado debe interpretarse dentro de la cohorte estudiada y no permite, por sí solo, establecer conclusiones sobre generalización externa ni sobre validez clínica: no se realizó una validación externa sobre una cohorte independiente, y la evaluación patient-level empleada se limita a caracterizar el comportamiento del modelo dentro de los datos disponibles. Los controles de bajo nivel aplicados sobre la tarea binaria histórica confirmaron, además, la existencia de información discriminativa asociada a la procedencia de los datos, un hallazgo que refuerza la necesidad de interpretar cualquier resultado de clasificación de imágenes médicas junto con la composición del conjunto de datos que lo produce.

14.2 Cumplimiento de los objetivos

El capítulo 2 estableció los objetivos específicos de este trabajo, articulados en torno a la trazabilidad de los datos, la metodología de evaluación y la automatización de la ejecución experimental. Los objetivos principales definidos para el TFG se alcanzaron, aunque algunos presentan limitaciones explícitas que condicionan su alcance, recogidas en la Tabla 14.1.

La trazabilidad patient-level de cada imagen se considera cumplida: el conjunto de datos final permite identificar, para cada corte utilizado, el paciente real del que procede, y las particiones del protocolo final respetan esa identidad en las etapas documentadas de selección y evaluación. El pipeline experimental se considera cumplido con limitaciones: la reproducibilidad lograda es a nivel de procedimiento — los artefactos conservados permiten documentar y verificar las principales configuraciones, datos, decisiones y resultados del proceso experimental—, no un determinismo numérico exacto entre ejecuciones distintas, y la recuperación de algunos artefactos de fases históricas del proyecto no es completa.

La clasificación multiclase sobre la cohorte Cheng se considera cumplida como núcleo experimental final del trabajo, con la comparación de varias arquitecturas de Deep Learning bajo un protocolo común igualmente cumplida. La validación mediante 5CV se considera cumplida como caracterización de la variabilidad interna del modelo, con la salvedad de que se trata de una evaluación interna sobre la misma cohorte, no anidada con la selección de arquitectura y no equivalente a una validación externa. La explicabilidad mediante Grad-CAM se considera cumplida con limitaciones: se trata de un análisis cualitativo y post-hoc, sin una evaluación cuantitativa de la superposición

entre los mapas de activación y una referencia de segmentación. Por último, la automatización de la ejecución experimental mediante infraestructura de computación de altas prestaciones se considera cumplida, entendida como automatización del proceso experimental y no como una contribución científica independiente de por sí.

Tabla 14.1 — Cumplimiento de los objetivos

Objetivo	Evidencia de cumplimiento	Estado
Trazabilidad patient-level	Identificación real de paciente en el conjunto de datos final; particiones agrupadas por paciente en el protocolo final	Cumplido
Pipeline reproducible	Configuración, hashes y artefactos versionados que permiten documentar y verificar el procedimiento experimental	Cumplido con limitaciones
Clasificación multiclase (Cheng)	Modelo final entrenado y evaluado sobre las tres clases dentro de la cohorte de referencia	Cumplido
Comparación de arquitecturas	Varias arquitecturas evaluadas bajo un protocolo común antes de la selección final	Cumplido
Validación interna (5CV)	Validación cruzada de cinco particiones agrupada por paciente sobre el conjunto completo de la cohorte	Cumplido
Explicabilidad (Grad-CAM)	Análisis cualitativo post-hoc sobre una selección de casos	Cumplido con limitaciones
Automatización (HPC)	Ejecución de los experimentos automatizada mediante	Cumplido

Objetivo	Evidencia de cumplimiento	Estado
	infraestructura de computación de altas prestaciones	
Demostrador (aplicación cliente-servidor)	software cliente-backend de inferencia implementados y verificados por código (capítulo 5), integrados en una fase posterior del proyecto	Cumplido (aportación secundaria)

14.3 Principales aportaciones del trabajo

Más allá del cumplimiento de los objetivos específicos, este trabajo puede resumirse en cuatro aportaciones principales.

La primera es la reconstrucción y trazabilidad de los datos a nivel de paciente. Frente a un punto de partida en el que no era posible reconstruir de forma fiable la identidad real de cada paciente, el conjunto de datos se reconstruyó a partir de fuentes que permiten recuperar una identificación real de paciente o sujeto. Sobre esa base se incorporaron controles de duplicados exactos y de candidatos perceptualmente similares antes de utilizar el conjunto reconstruido en el protocolo experimental.

La segunda es un pipeline experimental reproducible y auditable, construido como un conjunto de responsabilidades separadas y reutilizables, con configuración centralizada, versionado de artefactos mediante huella criptográfica y validaciones automáticas que comprueban, antes o durante la ejecución, condiciones como la ausencia de solapes entre pacientes. Esta reproducibilidad, como ya se ha señalado, se entiende a nivel de procedimiento y no como una garantía de determinismo numérico exacto.

La tercera es un protocolo de evaluación patient-level aplicado de forma coherente en las etapas finales documentadas: la arquitectura se seleccionó antes de abrir el conjunto de holdout, ese conjunto permaneció reservado hasta que el protocolo quedó fijado por escrito, y la validación cruzada posterior se realizó sobre el conjunto completo de pacientes sin reabrir la selección de arquitectura. La unidad utilizada para las métricas principales de selección y evaluación fue el paciente, mediante agregación de las probabilidades obtenidas en sus distintos cortes.

La cuarta aportación es el análisis crítico del propio proceso experimental y la documentación explícita de sus limitaciones. El proyecto no ocultó que la tarea binaria histórica estaba metodológicamente comprometida, que la reproducción de un resultado histórico difirió del valor original sin que la causa quedara completamente confirmada, que la variabilidad entre particiones de la 5CV no fue despreciable, que no se realizó una validación externa, que el protocolo final no constituye una validación cruzada anidada, ni que el análisis de Grad-CAM tiene limitaciones cuantitativas conocidas. La documentación explícita de estas limitaciones forma parte de la transparencia metodológica del trabajo y delimita las conclusiones que pueden extraerse de sus resultados.

De forma secundaria y complementaria a estas cuatro aportaciones científicas y metodológicas, este trabajo incorpora también una aportación de Ingeniería del Software: un demostrador cliente-servidor —cliente de escritorio y backend de inferencia en Cloud Run—, desarrollado y verificado por código en una fase posterior del proyecto (capítulo 5), que permite ejecutar los modelos entrenados y visualizar sus resultados. Esta aportación no forma parte del núcleo científico del TFG ni modifica ninguna de sus conclusiones; se documenta con el mismo nivel de exigencia y transparencia que el resto del trabajo, incluyendo sus limitaciones de seguridad y de arquitectura.

14.4 Trabajo futuro

Las líneas de trabajo futuro que se proponen a continuación se derivan directamente de las limitaciones descritas en el capítulo 13 y sintetizadas en el apartado anterior, y no de una lista genérica de posibles mejoras.

La prioridad principal es la validación externa: evaluar el modelo y el protocolo desarrollados sobre una cohorte independiente de la empleada en este trabajo, idealmente procedente de otro centro, con otro dataset o con otros protocolos de adquisición. El objetivo de esa validación sería medir la capacidad de generalización externa del sistema, sin que ello permita anticipar si los resultados obtenidos en esa cohorte adicional serían comparables a los presentados aquí.

Una segunda línea, de naturaleza metodológica, consistiría en extender el protocolo actual hacia una validación cruzada anidada, en la que un bucle externo se dedicara a la evaluación y un bucle interno a la selección de arquitectura e hiperparámetros. Esto no corrige un error del diseño actual, que es válido para el propósito con el que se planteó, sino que constituiría una extensión metodológica más

estricta, en la que la propia selección de modelo quedaría integrada dentro de la estimación de su rendimiento.

Una tercera línea consistiría en comparar los mapas de Grad-CAM obtenidos con las anotaciones de máscara tumoral disponibles para el dataset Cheng, no utilizadas en este trabajo como referencia de una evaluación cuantitativa, mediante métricas de solapamiento o localización adecuadas. Esto permitiría complementar el análisis cualitativo ya realizado con una evaluación cuantitativa de la correspondencia entre las regiones destacadas por el modelo y las anotaciones espaciales de referencia.

Una cuarta línea consistiría en investigar de forma explícita, en la medida en que los metadatos disponibles lo permitan, posibles señales residuales asociadas al centro, al protocolo o al escáner de adquisición dentro de la propia cohorte, sin asumir de antemano que dichos sesgos existen. En una dirección relacionada, cabría también evaluar otras estrategias de agregación de las predicciones a nivel de paciente, más allá de la media aritmética de probabilidades por corte empleada en este trabajo, sin que ello implique que esta última sea incorrecta.

Por último, en el plano de la reproducibilidad, sería deseable reducir la variabilidad no controlada entre ejecuciones mediante un determinismo de GPU más estricto y una captura más completa del entorno de ejecución —versiones de librerías, imagen de contenedor reproducible—, así como avanzar hacia un empaquetado más integral del código, los datos permitidos y la configuración empleada. De forma secundaria y desacoplada del pipeline de entrenamiento, el demostrador desarrollado (capítulo 5) ya permite seleccionar estudios, ejecutar inferencia, visualizar probabilidades y consultar los mapas de Grad-CAM. Queda como línea de trabajo futura incorporar trazabilidad persistente de cada inferencia individual (registro de qué imagen, modelo y resultado se generó y cuándo), hoy no implementada; una eventual evaluación asistencial de esta o de otras líneas requeriría, en todo caso, una validación clínica específica y un análisis de los requisitos regulatorios aplicables, aspectos que quedan fuera del alcance de este TFG.

14.5 Cierre

La calidad de un experimento de Deep Learning no se resume en una única cifra de rendimiento. Como ha mostrado la evolución de este proyecto, los datos empleados, su procedencia, la unidad a la que se realiza la partición, la trazabilidad del proceso y el protocolo de evaluación forman parte del resultado tanto como la propia arquitectura del modelo.

Este TFG deja, como resultado final, un clasificador experimental para tres tipos de tumor cerebral, un pipeline reproducible a nivel de procedimiento, un protocolo de evaluación patient-level aplicado de forma consistente en las etapas finales documentadas, y una documentación explícita de los límites dentro de los cuales deben interpretarse sus resultados. El principal resultado de este trabajo no es, por tanto, únicamente ese modelo, sino un procedimiento experimental en el que los resultados obtenidos pueden interpretarse junto con la procedencia de los datos, las decisiones de partición y las limitaciones del propio protocolo.

Sobre esa base, el siguiente paso no consiste en buscar una cifra de rendimiento mayor, sino en evaluar la capacidad de generalización del sistema sobre una cohorte externa independiente de la empleada en este trabajo, antes de plantear cualquier ampliación de su ámbito de aplicación.

Referencias

- Arghistani, A. M., Erbay, H., & Özdil, A. (2026). A Multi-Scale Attention and Efficient Convolution Framework for Brain Tumor Classification in Medical Imaging Analytics. *Healthcare Analytics*, 10, 100481. <https://doi.org/10.1016/j.health.2026.100481>
- Bulla, P., Anantha, L., & Peram, S. (2020). Deep Neural Networks with Transfer Learning Model for Brain Tumors Classification. *Traitement du Signal*, 37(4), 593-601. <https://doi.org/10.18280/ts.370407>
- Cawley, G. C., & Talbot, N. L. C. (2010). On Over-Fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation. *Journal of Machine Learning Research*, 11, 2079-2107.
- Cheng, J. (2024). *Brain Tumor Dataset*. <https://doi.org/10.6084/m9.figshare.1512427.v8>
- Cheng, J., Huang, W., Cao, S., Yang, R., Yang, W., Yun, Z., ... Feng, Q. (2015). Enhanced Performance of Brain Tumor Classification via Tumor Region Augmentation and Partition. *PLOS ONE*, 10(10), e0140381. <https://doi.org/10.1371/journal.pone.0140381>
- Efron, B. (1979). Bootstrap Methods: Another Look at the Jackknife. *The Annals of Statistics*, 7(1), 1-26. <https://doi.org/10.1214/aos/1176344552>
- Hassan, S. A., Hathah, A., Elnokity, O. E., Morfeq, A., Abdelfattah, W., Ateya, A. A., & Elsayy, N. (2026). Enhancing Brain Tumor Detection Through Deep Learning and Explainable AI Techniques. *Scientific Reports*, 16, 20558. <https://doi.org/10.1038/s41598-026-60334-y>
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770-778.
- IXI Dataset*. (s. f.). Recuperado de <https://brain-development.org/ixi-dataset/>
- Nalawade, S., Murugesan, G. K., Vejdani-Jahromi, M., Fisicaro, R. A., Bangalore Yogananda, C. G., Wagner, B., ... Maldjian, J. A. (2019). Classification of Brain Tumor Isocitrate Dehydrogenase Status Using MRI and Deep Learning. *Journal of Medical Imaging*, 6(4), 046003. <https://doi.org/10.1117/1.JMI.6.4.046003>

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 4510-4520.

scikit-learn developers. (s. f.). *Cross-validation iterators for grouped data*. Documentación oficial de scikit-learn. Recuperado de https://scikit-learn.org/stable/modules/cross_validation.html#cross-validation-iterators-for-grouped-data

Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization. *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 618-626.

Simonyan, K., & Zisserman, A. (2014). *Very Deep Convolutional Networks for Large-Scale Image Recognition*. Recuperado de <https://arxiv.org/abs/1409.1556>

Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., & Wojna, Z. (2016). Rethinking the Inception Architecture for Computer Vision. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2818-2826.

Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 97, 6105-6114.

Las referencias recogidas en esta sección corresponden a las fuentes citadas formalmente en el cuerpo de la memoria. Los estudios adicionales de la revisión bibliográfica cuya fuente original no ha sido verificada se mantienen identificados como tales en el Anexo I del documento de anexos técnicos.

Anexos

Los anexos técnicos de este trabajo (Anexo A, inventario de artefactos del proyecto; Anexo B, entorno de ejecución de alto rendimiento y relación completa de ejecuciones; Anexo C, protocolos experimentales; Anexo D, tabla maestra de resultados y resultados completos por partición; Anexo E, galería de mapas de explicabilidad; Anexo F, inventario técnico del prototipo histórico ScanIA; Anexo G, instrucciones de reproducibilidad; Anexo H, fragmentos

esenciales de configuración y código; Anexo I, tabla completa de los estudios de la revisión bibliográfica; y Anexo J, incidencia de despliegue en Cloud Run) se recogen íntegros, con el mismo nivel de verificación aplicado al resto de este trabajo, en el documento complementario de anexos técnicos, al que la memoria remite mediante referencias cruzadas de la forma «ver Anexo X del documento de anexos técnicos». Esta separación en un documento aparte responde a la extensión conjunta de los anexos y no afecta a ninguno de los resultados científicos, tablas o figuras presentados en los capítulos anteriores de esta memoria.